



Funded by the
European Union
NextGenerationEU

**RECOVERY
AND RESILIENCE
PLAN**

Final Project Report

Deliverable D3

Reported by: Evolveum, s.r.o.

5/2026



Table of Contents

Revision History 5

1. Introduction 6

 1.1 Purpose 6

 1.2 Document Overview 6

 1.3 Target Audience 7

 1.4 Acronyms and Abbreviations 7

 1.5 Changed Terms 9

2. Executive Summary 10

 2.1 Project Context 10

 2.2 Summary of the Delivered Solution 10

 2.3 Project Milestones and Timeline 11

 2.4 Key Results 11

3. Project Scope, Goals, and Delivered Outcomes 12

 3.1 Project Goals 12

 3.2 Planned Solution Areas 12

 3.3 Delivered Outcomes by Area 12

 3.4 Deviations, Scope Adjustments, and Open Items 18

4. AI-Assisted Solution Components 19

 4.1 Connector Code Generator 19

 4.1.1 Purpose 19

 4.1.2 Implemented Approach 19

 4.1.3 Final Component Design 20

 4.1.4 Integration into MidPoint and Related Tooling 21

 4.1.5 Delivered Functionality 21

 4.1.6 Evaluation and Validation 22

 4.1.7 Current Limitations 28

 4.2 Model Mapping Recommendation 29

 4.2.1 Purpose 29

 4.2.2 Implemented Approach 29

 4.2.3 Final Component Design 30

 4.2.4 Integration into MidPoint and Related Tooling 30

 4.2.5 Delivered Functionality 31

 4.2.6 Evaluation and Validation 31

 4.2.7 Current Limitations 36

 4.2.8 Complex Attributes: Experiments and Initial Design Direction 36

4.3 Correlation Recommendation	38
4.3.1 Purpose	38
4.3.2 Implemented Approach	38
4.3.3 Final Component Design	38
4.3.4 Integration into MidPoint and Related Tooling	38
4.3.5 Delivered Functionality	39
4.3.6 Evaluation and Validation	39
4.3.7 Current Limitations	40
4.4 Delineation and Focus Type Suggestion	40
4.4.1 Purpose	40
4.4.2 Delineation	40
4.4.3 Focus Type Suggestion	41
4.4.4 Final Component Design	41
4.4.5 Integration into MidPoint and Related Tooling	41
4.4.6 Delivered Functionality	42
4.4.7 Evaluation and Validation	42
4.4.8 Current Limitations	45
4.5 AI Development and Testing Tooling	45
4.5.1 Purpose	45
4.5.2 Implemented Toolchain	45
4.5.3 Integration into the Solution Lifecycle	46
4.5.4 Model and Prompt Tuning	46
4.5.5 Delivered Functionality	47
4.5.6 Current Limitations	47
4.6 UX and User-Facing AI Workflows	47
4.6.1 Onboarding workflow overview	48
4.6.2 Connector Generator Interface	56
4.6.3 MidPoint Studio Plugin	63
4.6.4 Association Suggestions Interface	64
4.6.5 Data Mapping Interface	65
4.6.6 Guiding UX Principles	72
4.7 Complete Pipeline Evaluation and Test Scenarios	73
4.7.1 Purpose	73
4.7.2 Evaluation Scope and Methodology	73
4.7.3 Connector Code Generator Evaluation	73
4.7.4 Configuration Assistant Test Environments	94
4.7.5 Configuration Assistant End-to-End Test Scenarios	97
4.7.6 Configuration Assistant Quantitative Results	102

4.7.7 Configuration Assistant Results By Environment	103
4.7.8 Key Findings and Limitations	107
5. Architecture and Design	109
5.1 Component Structure	109
5.2 Interface and API Specifications	111
5.3 Connector Framework and Code Generation Architecture	112
5.4 Mapping Recommendation Architecture	113
5.5 Correlation Recommendation Architecture	113
5.6 Delineation Architecture	113
5.7 User Interface Architecture	114
5.8 Integration Catalog	114
5.9 Validation Services	115
5.10 Supporting Internal Tooling	116
5.11 Security Considerations	117
5.12 Changes to MidPoint	117
5.13 Changes to ConnId Framework	118
5.14 Changes to midPoint Studio	118
5.15 MEL - MidPoint Expression Language	119
6. DevOps and Operational Setup	120
6.1 Hardware and Infrastructure	120
6.1.1 Current Hardware Setup	121
6.1.2 GPU Performance Testing	123
6.2 Kubernetes and Network Infrastructures	124
6.3 CI/CD Pipeline	125
6.4 Operational Architecture	127
6.4.1 Observability Architecture	127
6.4.2 Workload Organization and Namespace Strategy	127
6.4.3 API Gateway and Ingress Management	128
6.4.4 Workload Standards and Best Practices	128
6.5 Security Operations	129
6.6 Backup and Disaster Recovery	130
6.7 GPU Setup Checklist	130
6.8 Deployment Flexibility	131
7. Compliance, Security, and Governance	134
7.1 Cybersecurity Considerations	134
7.2 AI Governance	135
7.3 EU AI Act Alignment and Obligations	136
7.4 Personal Data Protection	137

7.5 Implemented Security, Privacy, and Fairness Measures 139

7.6 Risks, Limitations, and Remaining Open Issues 140

8. Open-Source Outputs and Project Deliverables 143

8.1 Delivered Artifacts Overview 143

8.2 Source Code and Repositories 143

8.3 Architecture, Developer, and User Documentation 144

8.4 DevOps and Deployment Artifacts 145

8.5 Licensing and Publication 146

8.6 Upstream Contributions 146

9. Project Evaluation, Lessons Learned, and Future Work 148

9.1 Evaluation Against Project Goals 148

9.2 Main Technical Outcomes and Their Practical Value 148

9.3 Community Presentation and Validation 149

9.3.1 MidPoint Community Meetup 149

9.3.2 Identity at the Center 151

9.3.3 AI Awards 151

9.3.4 Gartner IAM Summit 2026 151

9.4 Lessons Learned 152

9.5 Future Roadmap and Follow-Up Opportunities 153

9.5.1 Productizing the Onboarding Pipeline 153

9.5.2 Application Onboarding 154

9.5.3 Application and Resource Dashboards and Setup Assistant 155

9.5.4 Integration Catalog in MidPoint 157

9.5.5 Update of Learning Materials 157

9.5.6 Native Integration of Secure Expression Language 158

9.5.7 Activities in Areas of Compliance and Regulatory Alignment 158

Revision History

Revision	Date	Author	Description
0.1	2026-03-16	V.Kresnakova, M.Mares	Initial structure for the final project report
0.2	2026-03-18	V.Kresnakova	Filled the main report chapters and prepared a buildable draft
0.3	2026-03-24	V.Kresnakova	Extended and refined the priority mapping, matching, and correlation chapters
0.4	2026-03-31	V.Kresnakova	Prepared and completed the code generator chapter, including its structure and description
0.5	2026-04-07	V.Kresnakova	Prepared the first draft of chapters 5 and 7 based on the implemented solution
0.6	2026-04-10	V.Kresnakova	Prepared the first draft of chapters 8 and 9 and refined their structure and wording
0.7	2026-04-17	V.Kresnakova	Expanded the AI-assisted solution component descriptions and aligned them with the implemented MidPoint smart integration workflow
0.8	2026-05-12	V.Kresnakova	Updated evaluation sections with component benchmark results for schema matching, mapping generation, and delineation
0.9	2026-05-25	R. Richter	DevOps section updated with initial HW overview and background of the procurement process
0.10	2026-05-28	M. Mares	Updated sections in chapter 9 Evaluation

1. Introduction

1.1 Purpose

This document is the final report for the VAIA project under project code 09I05-03-V04-00101. It summarizes the final state of the delivered solution and captures the main technical, operational, and project outcomes achieved during the project.

The report builds on the earlier milestone documentation, especially the Milestone 1 architecture and design report, but shifts the focus from research, analysis, and target design to implemented solution capabilities, delivered artifacts, operational setup, and overall project evaluation.

The document covers the AI-assisted solution areas developed within the project, the final architecture and design of the solution, DevOps and operational aspects, compliance and governance considerations, and the open-source outputs and documentation produced as part of the project.

1.2 Document Overview

This report is organized into a set of chapters that together provide a complete overview of the final project results.

- **Chapter 1: Introduction** presents the purpose of the report, its organization, target audience, acronyms and abbreviations, and terminology notes.
- **Chapter 2: Executive Summary** summarizes the project scope, timeline, delivered outputs, and key results.
- **Chapter 3: Project Scope, Goals, and Delivered Outcomes** connects the planned project areas with the outcomes delivered by the end of the project.
- **Chapter 4: AI-Assisted Solution Components** describes the main AI-assisted capabilities developed within the project, including connector code generation, mapping recommendation, correlation recommendation, delineation, AI tooling, and user-facing AI workflows.
- **Chapter 5: Architecture and Design** documents the final technical architecture, component interactions, interfaces, supporting services, and modifications to related open-source components.
- **Chapter 6: DevOps and Operational Setup** describes the infrastructure, deployment model, CI/CD pipelines, observability, and operational aspects of the delivered solution.
- **Chapter 7: Compliance, Security, and Governance** covers the applied governance approach, regulatory considerations, and implemented security and privacy measures.
- **Chapter 8: Open-Source Outputs and Project Deliverables** provides an overview of

published code, documentation, deployment artifacts, licenses, and upstream contributions.

- **Chapter 9: Project Evaluation, Lessons Learned, and Future Work** concludes the report with an assessment of project outcomes, limitations, lessons learned, and recommended follow-up work.
- **Bibliography** lists the sources referenced in the report.

1.3 Target Audience

This final project report is intended for the following audiences:

- **VAIA stakeholders and funding authorities** who need a clear summary of the project outcomes, delivered solution areas, and published outputs.
- **Solution architects and technical leads** who need an overview of the final architecture, design decisions, and integration model of the delivered solution.
- **Developers and integrators** working with midPoint and related project outputs, including AI-assisted configuration and integration capabilities.
- **DevOps and operations teams** responsible for building, deploying, and operating the delivered solution and related infrastructure.
- **Security, privacy, and compliance reviewers** evaluating governance, security controls, and regulatory considerations associated with the project.
- **Open-source adopters and contributors** interested in understanding the outputs of the project and their relation to the wider midPoint ecosystem.

1.4 Acronyms and Abbreviations

AI

Artificial Intelligence

AI/ML

Artificial Intelligence / Machine Learning

API

Application Programming Interface

CI

Continuous Integration

CI/CD

Continuous Integration / Continuous Delivery

DR

Disaster Recovery

DevOps

Development and Operations

GDPR

General Data Protection Regulation

IDE

Integrated Development Environment

IGA

Identity Governance and Administration

ISO

International Organization for Standardization

ISO/IEC 42001

Artificial Intelligence Management Systems standard

LLM

Large Language Model

MCM

MidPoint Community Meetup

MLOps

Machine Learning Operations

SQL

MidPoint Query Language

MVP

Minimum Viable Product

PoC

Proof of Concept

SCIM

System for Cross-domain Identity Management

UI

User Interface

UX

User Experience

WP

Work Package

1.5 Changed Terms

This report follows the terminology established in the earlier project documentation where possible in order to preserve continuity across milestones and final outputs.

Where naming evolved during the project, the final report uses the final agreed terminology consistently. Earlier terms are referenced only where necessary to maintain traceability to milestone documentation or previously published materials.

2. Executive Summary

The VAIA project addressed the challenge of accelerating application onboarding into the midPoint platform by combining AI-assisted capabilities with supporting architectural, operational, and open-source deliverables. The project focused on reducing manual effort in connector development and configuration, improving the usability of integration workflows, and preparing the resulting solution for practical reuse and further development.

2.1 Project Context

Modern organizations depend on a growing number of applications, services, and identity-related data sources. As this landscape becomes more complex, onboarding new applications into an identity governance and administration platform becomes increasingly costly and time-consuming. This is especially visible in cases where new connectors, schema mappings, correlation rules, and related configuration logic must be designed and implemented manually. The public midPilot project description frames this problem in security terms as well: identity-related weaknesses are involved in a large share of cyberattacks, and slow or incomplete onboarding of applications into centralized IGA contributes to weak visibility, shadow IT, and a larger attack surface.

The project was initiated to explore and implement a more efficient onboarding approach around the midPoint platform. The central idea is to support selected integration tasks with AI-assisted methods while preserving practical usability, integration quality, human oversight, and compatibility with the broader open-source midPoint ecosystem. The intended social and operational impact is therefore not only faster integration work, but also improved security posture, better identity governance, and reduced barriers for organizations with limited specialist capacity.

2.2 Summary of the Delivered Solution

The delivered solution consists of several complementary areas.

At the functional level, the project has developed AI-assisted capabilities for connector code generation, model mapping recommendation, correlation recommendation, and delineation-related support. These capabilities are designed to assist integrators in selected onboarding tasks and to reduce manual effort in configuration and implementation workflows.

At the system level, the project has defined and implemented an architecture that connects these capabilities with the wider midPoint environment and related tooling, including user-facing workflows, validation services, and supporting internal components.

At the operational level, the project has also covered DevOps and deployment aspects required to build, run, observe, and maintain the resulting solution in a practical environment.

At the delivery level, the project has contributed to open-source components in the midPoint ecosystem and produced documentation, source-code outputs, and related artifacts intended for reuse, publication, and future continuation.

2.3 Project Milestones and Timeline

The project was structured into three major milestones covering the progression from architecture and design, through implementation of the end-to-end solution, to final testing, documentation, and publication of the project outputs.

- Milestone 1 focused on research, analysis, experiments, and target architecture and design.
- Milestone 2 focused on integrating the main solution components into a working end-to-end implementation.
- Milestone 3 focused on finalizing the delivered solution, documenting its architecture and operations, evaluating project outcomes, and preparing published outputs and related artifacts.

2.4 Key Results

The main result of the project is a delivered AI-assisted solution for faster and more structured application onboarding in the midPoint ecosystem, supported by the final architecture and design documentation, DevOps and operational guidance, and published project outputs.

The project has also produced practical knowledge about the strengths and limitations of using AI-assisted approaches in connector generation, mapping, correlation, and related configuration support. These findings informed the final implementation choices and helped establish a basis for future continuation of this work in open-source and product-related contexts.

By the project end, the solution was not only implemented at the technical level, but also framed as a reusable and publishable outcome through documentation, operational artifacts, trustworthy-AI-oriented governance considerations, and contributions to the surrounding open-source ecosystem.

3. Project Scope, Goals, and Delivered Outcomes

3.1 Project Goals

The project goal was to make application onboarding into the midPoint ecosystem faster, more structured, and less dependent on fully manual connector and configuration work. This goal was pursued through a combination of AI-assisted capabilities, platform improvements, supporting tooling, and operational deliverables.

At the functional level, the project focused on reducing manual effort in four main onboarding areas: connector creation, schema and mapping preparation, correlation setup, and delineation-related object type configuration. At the platform level, it also aimed to improve the surrounding ecosystem, including the connector framework, midPoint itself, midPoint Studio, validation services, and user-facing workflows.

An equally important project objective was to ensure that the results are not limited to isolated prototypes. The expected outcome was a reusable solution supported by open-source publication, developer and user documentation, DevOps guidance, and contributions back to the upstream components on which the solution depends.

3.2 Planned Solution Areas

The planned project scope covered a connected set of technical and delivery areas.

The primary solution areas were connector code generation, mapping recommendation, correlation recommendation, and delineation support. These capabilities were expected to be accessible through user-facing workflows in midPoint and related tooling, rather than as stand-alone research artifacts.

The scope also included the integration catalog, improvements in user experience and navigation for onboarding scenarios, supporting AI development and validation tooling, and platform-level changes in midPoint, ConnId, and midPoint Studio. Operational topics such as deployment architecture, CI/CD, observability, and security were part of the scope as well.

The closing project commitments further included publication-related outputs: source code, architecture and design documentation, DevOps documentation, and contributions intended for reuse in the wider open-source ecosystem.

3.3 Delivered Outcomes by Area

The table below summarizes the relationship between the planned areas and the delivered outcomes by the project end.

Planned area	Delivered outcome	Status
Connector generation	Connector generation is delivered as a standalone session-oriented microservice pipeline implemented in the <code>midpilot-connector-gen</code> repository and integrated into midPoint through the Java-side <code>SmartIntegrationService</code> API and connector development backend. The implemented pipeline covers target system documentation discovery, scraping, and digesting, as well as connector generation steps. The digester extracts object classes, attributes, endpoints, relations, authentication information, and metadata, while the code-generation layer produces native schema, ConnId structures, and operation-specific code for search, create, update, delete, and relation handling. The current implementation explicitly supports REST- and SCIM-oriented generation paths.	Delivered
Mapping recommendation	Mapping recommendation is delivered as a two-step capability covering schema matching and schema mapping. The <code>midpilot-smart-integration</code> microservice provides dedicated <code>matching</code> and <code>mapping</code> modules, while midPoint adds orchestration for inbound and outbound suggestions, representative data sampling, heuristic matching rules, hallucination filtering, deduplication, validation, and quality-based selection among <code>as-is</code> , heuristic, and AI-generated mapping variants. The resulting suggestions are integrated into the resource wizard UI through dedicated inbound and outbound mapping flows.	Delivered

Planned area	Delivered outcome	Status
Correlation recommendation	Correlation recommendation is delivered as a pragmatic combination of AI-assisted schema matching and midPoint-side correlator selection and scoring. The matching module in midpilot-smart-integration explicitly prioritizes recall for correlator-plausible attributes, while midPoint then selects likely correlators according to the focus type, reuses existing inbound mappings where available, derives draft correlation mappings from schema matches, and evaluates candidates on sampled focus and shadow data using coverage- and ambiguity-based scoring. The resulting suggestions are integrated into the correlation workflow in the resource wizard and are intended to be refined together with the mapping configuration.	Delivered
Delineation support	Delineation support is delivered as an assisted object type suggestion workflow combining AI-generated delineation rules with midPoint-side statistics and validation. The object_type module in midpilot-smart-integration suggests kind , intent , and delineation filters from schema and computed statistics, while midPoint provides the supporting statistics services, filter and base-context validation, corrective retry with structured feedback, and GUI support for reviewing suggested object type partitions and their underlying statistics.	Delivered

Planned area	Delivered outcome	Status
Integration catalog	The integration catalog is delivered as a standalone application-centric catalog implemented with a Spring Boot backend, Angular frontend, and PostgreSQL persistence layer. The final implementation no longer models the domain as a flat list of connectors. Instead, it distinguishes Application , Integration method , and Connector , allowing one application to be associated with multiple integration methods with specified connectors and multiple published versions of each integration method. The delivered functionality includes application and integration method metadata management, upload of connector implementations, Jenkins-driven build callbacks, download of published artifacts, download statistics, application tags, country-of-origin metadata, lifecycle-state tracking, and logo management. This means the final result is a working catalog and publication workflow for reusable onboarding artifacts, not only a design proposal.	Delivered
UI and UX improvements	UI and UX improvements are delivered primarily in the midPoint web onboarding workflow. The final solution integrates mapping, correlation, and object type suggestions directly into review-oriented resource onboarding flows, instead of exposing the AI components only as separate technical services. Compared with milestone 1, the UI direction has become more pragmatic: it focuses on concrete onboarding steps in midPoint and on guided review of suggestions, validation results, and generated artifacts.	Delivered

Planned area	Delivered outcome	Status
AI tooling and validation	AI development and testing tooling is delivered as a practical toolchain built around Langfuse and ClearML . Langfuse is used for tracing and observability of AI-assisted microservice execution, while ClearML is used for experiment tracking and management of evaluation-oriented notebook and script workflows used during development and validation of the AI components. This area therefore realized the milestone 1 AI Studio direction as a concrete working toolchain rather than as a single standalone product.	Delivered
MidPoint, ConnId, and Studio improvements	MidPoint received substantial project-specific implementation work in the dedicated smart-impl area, the Java-side orchestration layer for smart integration features inside MidPoint. Separate project work was also delivered in the connector development backend. The delivered changes include orchestration of schema matching, mapping suggestion, correlation suggestion, object type suggestion, statistics computation, validation and retry handling, heuristic mapping support, and activity handlers for asynchronous execution of suggestion workflows. MidPoint also contains a dedicated ConnectorDevelopmentType persistence model and backend logic for storing documentation sources, detected schema information, generated artifacts, connector project state, and generated manifest content. Framework-facing work is also reflected in the generation of ConnId-related schemas and operation scripts. MidPoint Studio was extended on the midpilot branch with expert-oriented smart-suggestion actions for resource object types, mappings, correlation rules, and associations, together with a smart-suggestion tool window, review tables, diff-based application of suggestions, and initial connector-generator workflow support.	Delivered

Planned area	Delivered outcome	Status
DevOps and operations	DevOps and operational outputs are delivered in practical and reusable form across the main repositories. The connector generation and smart integration services provide Docker-based local deployment, environment-based configuration, OpenAPI endpoints, and documented runtime setup. The integration catalog adds composed deployment with backend, frontend, PostgreSQL, Docker secrets, and build and publication integration points for GitHub and Jenkins. Operational observability for AI-assisted services is implemented through Langfuse , while development and evaluation workflows use ClearML . These materials provide a real build-and-run baseline for further reuse and product development.	Delivered
Compliance, governance, and security	Compliance, governance, and security are implemented primarily through architectural constraints and operational practices rather than through a separate autonomous AI control layer. The delivered solution keeps users in the review loop, validates generated scripts and filters before use, and separates runtime configuration and secrets from the source code using environment variables and secret files. The final architecture therefore reduces risk by treating AI outputs as assisted draft configurations that need to be reviewed and validated, and integrates the outputs in the controlled midPoint workflows. Some broader governance topics such as final long-term organizational controls and a complete threat-model package remain as follow-up work beyond the delivered implementation scope.	Delivered

Planned area	Delivered outcome	Status
Open-source outputs and publication	The project produced multiple open-source outputs: the connector generation microservice, the smart integration microservice, the integration catalog, and project-related changes in midPoint. These outputs include source code, API surfaces, deployment materials, database schemas, and accompanying project documentation. The repositories are publicly accessible and use open-source licensing compatible with the project objective of making the resulting tooling reusable in the wider midPoint ecosystem.	Delivered

3.4 Deviations, Scope Adjustments, and Open Items

Compared with milestone 1, the final delivered scope is narrower in some exploratory directions and stronger in practical integration. The most important adjustment is that the project did not end with one separate monolithic **AI Studio** product. Instead, it produced a set of working components integrated into midPoint onboarding flows, supported by dedicated microservices, validation logic, operational tooling, and publication infrastructure.

Some originally broader exploratory directions were simplified into more robust review-oriented workflows. This is especially visible in mapping and correlation flows, where the final result relies on a combination of AI suggestions, midPoint-side heuristics, validation, deduplication, and scoring rather than on unrestricted generation alone. The user-facing scope also converged toward the midPoint web onboarding workflow as the primary final interaction surface.

At the same time, some important directions remain open because they go beyond the practical scope of the current project phase, even though groundwork for them has been prepared during the project. One important example is richer support for complex and multi-valued attributes. This includes cases such as structured names, e-mail addresses, phone numbers, addresses, and other business-significant composite values that cannot be handled well enough by a purely flat simple-attribute model. This area was already analyzed and partially prepared during the project, but full end-to-end support would require broader changes in schema serialization, matching, mapping, correlation, validation, and user interaction.

These topics should therefore be understood as follow-up directions that build directly on the current project results, not as abandoned ideas. Further hardening is also still needed in areas such as broader scenario coverage, final end-to-end evaluation on representative datasets, additional production-readiness work, and continued upstreaming of project results into the wider product ecosystem.

4. AI-Assisted Solution Components

This chapter describes the AI-assisted capabilities delivered within the project in their final form. Compared with the Milestone 1 report, the focus here is no longer on research and baseline experiments, but on the implemented solution, its integration, and its practical maturity.

All experiments in this chapter that use the `gpt-oss-120b` were executed on Evolveum's local GPU infrastructure, where this model was deployed and accessed through the project's LLM endpoint. This is relevant for interpreting the results, because these runs did not depend on a publicly hosted model endpoint.

4.1 Connector Code Generator

4.1.1 Purpose

The connector code generator addresses one of the most expensive onboarding tasks in the midPoint ecosystem: preparing connector implementation artifacts for applications that are not already covered by an existing reusable connector. Its purpose is to reduce the amount of manual work needed to move from application documentation to a technically usable connector definition and operation-specific implementation pieces.

In the delivered solution, connector generation is not treated as a single one-shot generation step. It is implemented as a structured workflow in which documentation is discovered or provided, transformed into structured intermediate data, and then used to generate specific parts of the connector implementation.

4.1.2 Implemented Approach

The implemented solution is a session-oriented microservice pipeline in the `midpilot-connector-gen` repository. At the API level, it is divided into five parts: `session`, `discovery`, `scrape`, `digester`, and `codegen`.

The `session` layer provides the stateful context for the whole workflow. It stores session data, uploaded documentation, intermediate results, and job references for asynchronous processing. In practice, this is not just an in-memory convenience layer: the implementation persists sessions, jobs, uploaded documentation items, and chunk-level relevance metadata in a PostgreSQL-backed storage model so that the workflow can survive asynchronous execution and state can be reused between stages. Documentation can be supplied either by upload or indirectly later through discovery/scraping steps, and uploaded files are split into chunks and processed in background by jobs tied to the session.

The `discovery` module is responsible for finding candidate documentation URLs for the target application. The `scrape` module then retrieves relevant documentation content from those URLs.

This allows the workflow to start either from discovered documentation or from documentation provided directly by the user.

The **digester** module transforms documentation into structured intermediate artifacts. Its implemented extraction areas include object classes, attributes, endpoints, relations, authentication information, and metadata. These outputs are stored in the session and become the input for subsequent code generation steps. The digester is not a single monolithic pass: object classes are extracted first, then per-object-class attribute and endpoint extraction can focus only on relevant documentation chunks, while relation, authentication, and metadata extraction are handled as separate background jobs with their own progress tracking.

The **codegen** module generates individual connector artifacts from the structured intermediate data. The implemented generation targets include native schema, ConnId-related structures, search code, create code, update code, delete code, and relation-handling code. Each of these artifacts has its own API entry point and its own asynchronous job lifecycle instead of being collapsed into one final "generate connector" endpoint. The implementation also contains separate prompt and documentation sets for REST and SCIM scenarios, and the code-generation service explicitly detects the protocol family from the session metadata before selecting operation-specific assets. For operation-level generation, the service also accepts user-selected preferred endpoints and a repair context containing the current script and midPoint validation or runtime errors. This allows the generator to refine an already generated or user-edited script instead of always starting from a blank generation request.

4.1.3 Final Component Design

The final component design combines a standalone Python/FastAPI microservice with a session-centric orchestration model and multiple specialized internal modules. The design is intentionally modular. The service does not try to infer everything in one step, but instead separates discovery, scraping, structured extraction, and artifact generation into explicit stages.

This design has two important consequences. First, intermediate outputs can be reviewed, overridden, or reused in the session before continuing to the next phase. Second, the generated result is not a monolithic connector blob, but a set of specific implementation parts that correspond to the connector schema and operation logic.

Internally, the code generation layer is also split into dedicated **core**, **selection**, **prompts**, **documentations**, and **utils** parts. This confirms that the delivered implementation is not just a thin API wrapper over prompts, but an actual software component with internal selection logic, reusable prompt sets, and domain-specific supporting materials. An especially important design choice is the use of chunk-level relevance propagation. Digest stages record which documentation chunks were relevant to object classes, attributes, or endpoints, and later codegen stages reuse these **relevantDocumentations** metadata to narrow the context supplied to the generator for a given operation. This makes the pipeline both more explainable and more robust than a design that would pass the full scraped documentation corpus into every generation step.

Another notable design element is the background job subsystem. Each major step is scheduled as a persisted job with `queued`, `running`, `finished`, and `failed` states, progress counters, stage messages, and recovery of stale running jobs at the application startup. This is important for a real onboarding workflow because scraping, digestion, and generation are multi-step operations over multiple documents and object classes, and they therefore need explicit execution state rather than a purely synchronous request-response style.

4.1.4 Integration into MidPoint and Related Tooling

The connector generation service was integrated into midPoint through the Java-side `SmartIntegrationService` API and connector development backend. The connector generation endpoint is configured separately from the general smart integration service. In the current midPoint system configuration, the `smartIntegration` section contains `serviceUrl`, which points to the `midpilot-smart-integration` microservice used by the recommendation components, while `connectorGenerationUrl` points to the connector generation microservice. The same configuration section also contains `connectorFrameworkUrl`, currently used by the connector development backend to obtain the base connector framework/template for the editable connector project.

At the midPoint level, the integration is not limited to generic remote invocation. The implementation contains a dedicated connector development backend that drives the external microservice through session-scoped `discovery`, `scrape`, `digester`, and `codegen` calls and then maps the returned results back into midPoint-side connector development objects. This includes discovery of candidate documentation links, scraping and synchronization of documentation, extraction of application metadata and authorization information, discovery of object classes, per-object-class extraction of endpoints and attributes, and generation of concrete connector artifacts, such as native schema scripts, ConnId mappings, search, create, update, and delete scripts, and relation-handling code.

The generated artifacts are not treated as transient text only. MidPoint preserves them in its `ConnectorDevelopmentType` model, writes the generated files to the editable connector project, and recomputes the connector manifest so that the produced pieces become part of the evolving connector structure instead of remaining detached AI outputs. This makes the integration substantially deeper than a one-off API bridge: the external generation service is embedded into a stateful connector development workflow that can progressively populate application metadata, detected schema, documentation sources, and generated implementation artifacts.

This means that connector generation is not delivered as a standalone research service only. It was designed to participate in the broader midPoint onboarding flow and to provide input for subsequent configuration steps.

4.1.5 Delivered Functionality

The delivered functionality includes:

- A standalone connector generation microservice implemented with FastAPI.

- Session management backed by persistent storage for workflow state, documentation items, jobs, and chunk-level relevance metadata.
- Discovery of candidate documentation links.
- Scraping of relevant documentation.
- Upload-based ingestion of user-provided documentation with chunking and asynchronous processing.
- Structured extraction of object classes, per-object-class attributes, per-object-class endpoints, relations, authentication data, and metadata.
- Code generation for native schema, ConnId, search, create, update, delete, and relation logic, each as a separate artifact-oriented operation.
- Separate implementation support for REST and SCIM-oriented generation paths, including protocol-aware asset selection.
- Status and override endpoints for intermediate and generated results, making the workflow reviewable and correctable between stages.
- Repair-oriented code generation using the current script and midPoint-side validation or runtime errors as additional context.
- Support for preferred endpoint selection for operation-specific code generation.
- Integration hooks in midPoint through the connector development backend, midPoint tasks, editable connector project files, and related GUI flow.

At the architectural level, this is stronger than a prototype focused only on prompts. It is a multi-stage software component with explicit API surfaces and persistent workflow state.

4.1.6 Evaluation and Validation

The connector generation capability was validated in three different ways during the project. These validation activities have different scope and should not be read as one single evaluation.

First, the implementation was checked at the repository and service level. This includes unit and integration tests, asynchronous job handling, persisted session and job state, and session-based status tracking for the individual pipeline stages.

Second, the midPoint integration was validated through midPoint-side test and setup materials. These checks focus on communication between midPoint and the connector generation service, and on whether the generated state can be represented in the midPoint connector development workflow.

Third, the generator outputs were evaluated experimentally on selected REST and SCIM-oriented applications. This section reports that component-level experimental evaluation. The evaluation focused on the structured data that drives code generation: object classes, application metadata, authentication methods, attributes, and endpoints. These are the most

important intermediate results because they determine whether the generator can later produce meaningful native schema, ConnId structures, and artifacts for search, create, update, delete, and relation handling.

The end-to-end validation of the complete connector generator workflow is reported separately in [Section 4.7](#). Unlike the component-level evaluation in this section, the end-to-end evaluation includes the full guided workflow, human review, correction, generated artifacts, and executable behavior in test environments.

The final evaluation used ten runs per evaluated application. The main datasets were:

- OpenProject, evaluated as a REST application and covering **user**, **group**, **membership**, **principal**, and **project** object classes.
- Codeberg-Forgejo, evaluated as a REST application and covering **user**, **repository**, **organization**, and **team** object classes.
- Nextcloud provisioning API, evaluated as a REST application and covering provisioning **user** and **group** object classes.
- Zendesk, evaluated as a REST application and covering **user**, **group**, **organization**, **organization_membership**, and **group_membership** object classes.
- Atlassian, evaluated as a SCIM application and covering **user** and **group** resources.

This shows that the evaluation was not limited to REST APIs only. OpenProject, Codeberg-Forgejo, Nextcloud, and Zendesk represent the REST-oriented part of the evaluation, while Atlassian represents the SCIM-oriented part. The metrics should be interpreted as a development benchmark for the extraction stages. The ground truth datasets were prepared earlier in the project and were used to compare prompt versions, extraction behavior, and model output stability during development. They were not continuously re-curated after every change in the documentation, evaluated application version, or prototype scope. For example, a ground truth dataset may have been prepared against one application version, while a later evaluation or local test may have used a newer version with a changed API documentation, object model, or endpoint behavior. Some mismatches can therefore be caused by version or documentation drift rather than by extraction errors. For the final assessment of operational usefulness, the more important evidence is therefore the end-to-end validation in [Section 4.7](#), where the generator workflow is evaluated in test environments with human review, correction, and executable behavior.

Rather than evaluating every possible object exposed by each API, the evaluation used a connector-oriented sample: one account-like object class, one or more role-like object classes, and, where available, relation-bearing membership data. The quantitative scoring focused on the structured intermediate outputs needed to generate ConnId definitions, native schema, search variants, such as **getAll** and **getId**, and filter-based search, create, update, delete, and relation support.

The following tables separate the evaluated extraction stages. This avoids mixing very different

result types into one wide table and makes precision, recall, F1-score, and accuracy clearly distinguishable and comparable across the stages.

Object Class Extraction

Application	Object-class precision	Object-class recall	Object-class F1-score	Object-class accuracy	Metadata score
OpenProject	11.65 %	98.33 %	20.33 %	11.13 %	96.67 %
Codeberg-Forgejo	15.97 %	95.00 %	27.34 %	15.83 %	100.00 %
Zendesk	23.19 %	80.00 %	35.96 %	21.92 %	83.33 %
Atlassian	6.23 %	22.22 %	9.73 %	5.12 %	60.00 %
Nextcloud	21.51 %	100.00 %	35.40 %	21.51 %	76.67 %

The object-class results mostly show a recall-oriented behavior. For OpenProject, Codeberg-Forgejo, and Nextcloud, the generator usually found the expected object classes, but it also produced many additional candidates. This produced high recall, from **95.00 %** to **100.00 %**, but low precision and therefore low F1-score. Zendesk reached lower but still useful recall at **80.00 %**. In this case, **group** and **organization** were detected consistently, while **organization_membership** and **group_membership** were detected in only part of the runs. The **user** class was also predicted consistently, but the ground truth called the class **user_for_admin**, so this naming difference was counted as both a false positive and a false negative. Atlassian was the exception: its object-class recall was only **22.22 %**, so this case should be treated as an unresolved validation gap rather than a successful extraction result.

For the recall-oriented cases, this is a useful first draft for connector development, because missing an important object class is usually more damaging than presenting extra candidates for review. However, the result also confirms that object-class extraction must remain reviewable and should not be applied automatically without filtering or user confirmation.

Application metadata was the most stable extraction category:

- Codeberg-Forgejo reached a full **100.00 %** score across the evaluated runs.
- OpenProject reached **96.67 %**, with the failures concentrated in **applicationVersion**.
- Zendesk reached **83.33 %**, also with the failures concentrated in **applicationVersion**.
- Nextcloud reached **76.67 %**, with failures mostly around **base_api_endpoint** fields and one **apiVersion** miss.
- Atlassian reached **60.00 %** and the main practical failure was the base API URL.

This indicates that metadata extraction is usable when documentation exposes clear product and endpoint information, but it is still sensitive to how API base URLs and versioning are presented.

Authentication Extraction

Application	Auth precision	Auth recall	Auth F1-score	Auth accuracy	Main observation
OpenProject	71.33 %	65.00 %	67.46 %	52.00 %	Expected methods were often found, but runs repeatedly added <code>api_token_bearer</code> and sometimes missed <code>session</code> or <code>oauth2</code> .
Codeberg-Forgejo	65.71 %	76.67 %	70.77 %	54.76 %	Authentication extraction was usable, but OAuth, OpenID Connect, and OTP-related variants were often ambiguous or over-generated.
Zendesk	58.82 %	100.00 %	74.07 %	58.82 %	All expected authentication mechanisms were found, but precision was reduced by extra candidates.
Atlassian	23.83 %	100.00 %	38.33 %	23.83 %	The expected bearer/API-key token mechanism was found in every run, but precision was low because the model also produced extra authentication candidates.
Nextcloud	76.00 %	95.00 %	84.44 %	73.08 %	The bearer token was found in all but one run, but precision was reduced by extra candidates.

Authentication extraction reached moderate quality on OpenProject and Codeberg-Forgejo, stronger recall-oriented results on Zendesk and Nextcloud, and a lower but still interpretable result on Atlassian. Nextcloud achieved **84.44 %** on the averaged `getAuth` result, having an average per-run F1-score similar at **84.67 %**, which indicates a stable high-recall result rather than a single-run outlier.

The recurring issue was not complete failure to detect authentication, but ambiguity and over-generation: for example, an extra bearer-token, OAuth/OpenID Connect, or OTP-related variants appeared in several runs.

These results again support the final design choice that authorization information should be presented as a candidate set for expert review rather than silently converted into final connector behavior.

Endpoint Extraction

Application	Endpoint precision	Endpoint recall	Endpoint F1-score	Endpoint accuracy
OpenProject	47.18 %	92.29 %	62.44 %	45.39 %
Codeberg-Forgejo	16.02 %	99.64 %	27.60 %	16.01 %
Zendesk	19.47 %	65.00 %	29.77 %	17.65 %
Atlassian	100.00 %	100.00 %	100.00 %	100.00 %
Nextcloud	84.08 %	44.00 %	57.77 %	40.62 %

Endpoint extraction behaved differently across applications:

- OpenProject reached a balanced but still recall-oriented result with **92.29 %** recall and **62.44 %** F1-score for exact method-and-path matching.
- Codeberg-Forgejo found almost all expected endpoints with **99.64 %** recall, but precision was only **16.02 %**, meaning that the extracted endpoint set was too noisy for direct use without filtering.
- Nextcloud showed the opposite profile: precision was high at **84.08 %**, but recall was only **44.00 %**.
- Zendesk endpoint extraction reached **65.00 %** recall, but precision was low at **19.47 %**. This is slightly lower than in other applications, but Zendesk has a large number of endpoints that it uses primarily with the **User** object class, which the LLM was unable to detect.
- Atlassian reached **100.00 %** after normalizing endpoint paths relative to the SCIM base URL. The normalization point is important because a strict evaluator that compares full raw paths can incorrectly report endpoint extraction as wrong when the real issue is an incomplete **baseApiEndpoint** in metadata.

Together, the results confirm that endpoint extraction quality depends heavily on documentation structure, API publication format, and correct separation between the base API URL and object-class endpoint paths.

The SCIM-oriented path should be interpreted as a space for further active development promising better outcomes rather than a closed result. SCIM is a standardized provisioning protocol with a defined resource model, schema structure, and conventional resource endpoints, which makes it suitable not only for LLM-based extraction but also for deterministic heuristics. Follow-up work therefore continues on heuristics that do not rely on LLM calls. For endpoints, this approach has already been partially validated, which is reflected in the strong normalized endpoint result. For object-class and attribute extraction, similar improvements remain in progress.

Attribute Extraction

Attribute extraction was evaluated with two strictness levels. The first level compared attribute

names only. The second compared name, type, and format.

Application	Name-only precision	Name-only recall	Name-only F1-score	Name + type + format F1-score	Interpretation
OpenProject	89.11 %	83.72 %	86.33 %	64.75 %	Strong attribute-name extraction, weaker strict type and format extraction.
Codeberg-Forgejo	55.09 %	56.35 %	55.72 %	48.09 %	Usable but noisy attribute discovery, especially for repository-oriented schema.
Zendesk	100.00 %	100.00 %	100.00 %	73.04 %	Perfect attribute-name match in the evaluated set, weaker but still strong type and format agreement.
Atlassian	38.10 %	16.00 %	22.54 %	20.42 %	Correct SCIM endpoints, but weak attribute coverage and poor type and format-level extraction.
Nextcloud	19.61 %	90.00 %	32.21 %	18.69 %	Recall-oriented extraction with many extra or weakly specified attributes.

These attribute results are important because they show how much the evaluation strictness changes the apparent quality of the generator.

- When only names are compared, OpenProject performs well and Codeberg/Forgejo remains usable as a review draft.
- Zendesk performed strongest on attributes, reaching **100.00 %** F1-score for name-only matching and **73.04 %** for strict matching.
- The Atlassian SCIM case shows the opposite pattern from its endpoint result: after base-URL normalization, the endpoint paths match perfectly, but attribute extraction remains weak with F1-score of only **22.54 %** for name-only and **20.42 %** for strict matching.

The per-object-class results show that SCIM **user** attributes were detected better than SCIM **group** attributes, but both remained below the level needed for direct schema generation. When type and format agreement is required, quality drops compared with name-only matching. In practical terms, the generated native schema should therefore be treated as a strong starting point for attribute discovery, not as a final schema contract.

Conclusion

The evaluation confirms that the generator is more mature as a structured assistant than as a fully autonomous connector factory. It can identify many of the right object classes, endpoints, attributes, and authentication options, and it produces the input needed for connector artifacts, such as native schema, ConnId mappings, search variants, create, update, delete, and relation support. At the same time, the quantitative results show that review, filtering, preferred endpoint selection, and repair-oriented regeneration are not optional convenience features. They are part of the necessary control layer around the AI-generated connector development.

The overall conclusion is that the delivered connector generator provides a useful acceleration layer for REST-like and SCIM-like connector development, especially for preparing a first-draft schema and operation inputs. It reduces the amount of manual documentation analysis and gives engineers concrete artifacts to inspect and refine. The measured limitations are mainly precision, over-generation, incomplete strict schema metadata, and sensitivity to base URL extraction, which means that the final workflow must remain expert-reviewed and integrated with midPoint-side validation rather than presented as fully automatic connector creation.

4.1.7 Current Limitations

The current implementation is clearly strongest for REST and SCIM-oriented integrations, which is also visible in the structure of the codegen prompts and bundled documentation examples. It should not be presented as a universal generator for all connector categories.

The midPoint integration also still contains a transitional dependency on an explicitly configured connector framework or template URL. This is sufficient for the delivered connector development workflow, but the final distribution path should be described as a connector-framework or integration catalog responsibility rather than as a permanent hard-coded template source.

The implementation also depends on the quality and coverage of available documentation. Where documentation is incomplete, noisy, or inconsistent, the quality of extracted intermediate data and generated code tends to degrade. For that reason, the pipeline is intentionally structured around intermediate outputs and override capabilities instead of assuming that every step is fully reliable.

Finally, although the component is already integrated into midPoint-side smart integration flows, the empirical results show that production readiness depends on keeping the workflow review-oriented. The current generator is useful for preparing first drafts and reducing manual analysis effort, but the final connector quality still depends on expert review, validation against the real target system, and broader scenario coverage beyond the evaluated REST/OpenAPI and SCIM-oriented applications. The SCIM-oriented Atlassian results also show that protocol support in the implementation does not automatically translate into reliable extraction quality for every documentation style.

4.2 Model Mapping Recommendation

4.2.1 Purpose

Model mapping recommendation reduces the amount of manual work needed to prepare attribute mappings between an application schema and the midPoint object model. Its role in the final solution is to support one of the most time-consuming parts of application onboarding: deciding which attributes correspond to each other and how should be the values transformed between the two sides.

In the delivered solution, this area is intentionally split into two related but distinct steps. The first step is schema matching which identifies candidate attribute correspondences between the application schema and the midPoint schema. The second step is schema mapping which suggests the actual mapping logic for these attribute pairs.

The delivered implementation also explicitly distinguishes inbound and outbound mappings. Inbound mappings describe transformations of attribute values from the application side to the midPoint side. Outbound mappings describe transformations from the midPoint side to the application side.

4.2.2 Implemented Approach

The implemented approach combines AI-assisted microservices with additional orchestration, heuristics, validation, and quality assessment in midPoint itself. This is important because the final solution does not rely on raw LLM output alone.

For schema matching, the `midpilot-smart-integration` microservice exposes a dedicated `matchSchema` endpoint. Its purpose is to suggest candidate attribute pairs between the application schema and the midPoint schema. The request contains the application schema and the midPoint schema with attribute metadata, and the response contains a list of matched attribute pairs.

For schema mapping, the same microservice exposes a `suggestMapping` endpoint. This endpoint operates on a selected application attribute and a selected midPoint attribute together with example input and output values. The request explicitly contains the mapping direction through the `inbound` flag, making the difference between application-to-midPoint and midPoint-to-application mapping a first-class part of the implementation. For validation-driven retries, the request can also carry the previous generated script and the validation error log as a remedial context.

On the midPoint side, these microservice calls are only one part of the delivered solution. `SchemaMatchingOperation` combines well-known schema knowledge with AI-produced schema matches, merges the results, removes duplicates, and filters out hallucinated matches that reference attributes not present in the real schemas.

`MappingsSuggestionOperation` then builds on the schema match and performs the actual mapping suggestion process. This includes collecting representative owned shadows, preparing separate samples for LLM prompting and validation, checking whether an `as-is` mapping is already sufficient, evaluating heuristic transformations, calling the AI service when appropriate, retrying once with validation feedback if the generated script fails, and rejecting very low-quality suggestions.

The implemented mapping heuristics are a substantial part of the delivered solution. The current code contains heuristic rules such as trimming, lowercasing, uppercasing, stripping diacritics, and several combinations of these operations. These heuristics are evaluated and scored, and `midPoint` chooses the best available strategy among `as-is`, heuristic, and AI-generated variants.

4.2.3 Final Component Design

The final design is a layered recommendation component embedded into the smart integration flow. At the service level, the production `midPoint` flow uses the `matching` and `mapping` microservice modules. At the platform level, `midPoint` wraps these services in operations that add schema knowledge, preprocessing, deduplication, scoring, and fallback logic.

This means that the final component design has three levels:

- Schema matching which proposes candidate attribute correspondences.
- Mapping suggestion which proposes the actual transformation logic.
- `MidPoint`-side orchestration which decides whether to use `as-is`, heuristic, or AI-generated mappings and how to present the result.

This layered design is stronger than a pure prompt-based approach because it separates concerns and adds `midPoint`-side control over quality and applicability.

4.2.4 Integration into `MidPoint` and Related Tooling

Mapping recommendations are integrated directly into the `midPoint` smart integration workflow and resource wizard UI. The implementation contains dedicated GUI support for both inbound and outbound mapping suggestions, including separate panels and tables for the two directions.

The user-facing flow therefore does not expose mapping suggestions only as a generic API call result. It presents them in the context of object type configuration where users can inspect, accept, reject, modify, and later validate the suggested mappings.

At the implementation level, the integration also includes wrapper utilities and smart integration support code that translate suggestion results into the underlying mapping configuration structures used by `midPoint`. The currently wired `midPoint` client methods for this area call `matching/matchSchema` and `mapping/suggestMapping`.

4.2.5 Delivered Functionality

The delivered functionality includes:

- Schema matching between application schema and midPoint schema.
- Mapping suggestion for selected attribute pairs.
- Explicit support for both inbound and outbound mapping directions.
- AI-assisted mapping script generation based on attribute metadata and example values.
- Heuristic mapping suggestions implemented directly in midPoint.
- Comparison of **as-is**, heuristic, and AI-generated mapping variants based on measured quality.
- Progressive fallback from direct **as-is** mapping, through deterministic heuristics, to AI-generated transformations only when simpler options are not sufficient.
- Retry of AI-generated mapping with backend validation feedback.
- Remedial context for retries containing the previous script and validation error log.
- Controlled use of representative data samples for prompting and validation with separate samples used to generate suggestions and to verify them.
- Simulation-oriented validation of generated inbound mappings on representative data before suggestions are accepted.
- Hallucination filtering and deduplication in schema matching.
- Filtering of duplicate suggestions against already existing mappings or already accepted suggestions.
- Dedicated GUI support for mapping suggestions in the resource wizard.

This means that the delivered functionality is not limited to suggesting "as-is" mappings or mappings with an expression snippets. It covers the whole path from matching candidate attributes to surfacing direction-specific mapping suggestions in the user workflow. The expression language we used in most experiments is Groovy, as it has been the default language for midPoint mappings. In parallel with the midPilot work, Evolveum also invested effort into a new, more constrained, and safer alternative to Groovy. This related midPoint platform activity was motivated in part by the security needs exposed by AI-generated mappings, but it should be understood as an associated platform hardening activity rather than as a standalone midPilot component. The new safe expression language is called MEL (MidPoint Expression Language). After MEL support became available in midPoint, selected final evaluation scenarios were repeated using MEL.

4.2.6 Evaluation and Validation

The delivered implementation already contains important validation mechanisms inside the runtime design. On the schema matching side, hallucinated AI matches are removed if they

reference attributes that do not actually exist. On the mapping side, mapping candidates are evaluated using representative data samples, quality assessment, and validation of generated expressions. When AI-generated scripts fail validation, the implementation retries once with the validation error log and the previous script as additional correction context.

The codebase also contains dedicated tests for mapping suggestion behavior, including tests for:

- Preference of heuristics when AI is disabled.
- Fallback to **as-is** mapping when no suitable heuristic exists.
- Explicit outbound heuristic handling.
- Preference of heuristics when they outperform AI.
- Concrete heuristic families such as diacritics removal, casing changes, and trim-based combinations.

The final end-to-end evaluation of this capability belongs to the complete pipeline evaluation in [Section 4.7](#). This is important because the practical behavior of mapping recommendation is shaped not only by the language model itself, but also by midPoint-side preprocessing, heuristic enrichment, post-processing, validation, deduplication, and filtering of invalid or hallucinated suggestions.

Schema Matching Experiments

For the schema-matching part of the pipeline, the project maintained a dedicated benchmark focused on the raw LLM matching stage. This benchmark did not evaluate the full delivered onboarding pipeline end to end. In particular, it did not yet include the later midPoint-side post-processing steps, such as schema-aware filtering, deduplication, validation, ranking, and the downstream use of matching results in mapping and correlation workflows.

The benchmark used midPoint schema CSV files split by focus type together with resource schema CSV files and manually prepared ground truth label files. The evaluated dataset was assembled from the refined schema collection prepared during the project. This collection was intentionally built from a broad mix of sources, including top-priority cloud systems, such as Entra ID, AWS, and Salesforce, middle-tier systems, such as Google Apps, Zoom, Atlassian, GitHub, KnowBe4, and Slack, European and open-source systems, such as Nextcloud, OpenProject, Univention, and Shopware, and several domain-oriented datasets, such as healthcare, aviation, and academy. The refined schema collection was created as CSV-based datasets with literal property names, types, and descriptions copied from source documentation, together with manually curated generalized match tables that captured all relevant 1:1 candidate correspondences between resource attributes and midPoint properties.

In the evaluated benchmark subset, the schema-matching experiments covered 46 schemas in total: 31 **User** schemas, 14 **Role** schemas, and 1 **Service** schema. The evaluation tracked not only standard matching metrics such as precision, recall, F1, and MCC, but also hallucination count and correlator recall. The correlator-specific metric was important because the practical

onboarding objective was to avoid missing plausible correlation candidates, even if this meant accepting somewhat lower precision. The final report tables therefore show the main decision metrics only: precision, recall, F1, correlator recall, and average inference time where applicable.

The final schema-matching evaluation run for the delivered direction used the locally deployed `gpt-oss-120b` model on Evolveum's GPU infrastructure. The run used the final single-template prompt, covered all 46 schemas, and completed without failed runs. The aggregate result was:

Model	Precision	Recall	F1	Correlator recall	Avg. inference time
<code>gpt-oss-120b</code> local GPU	0.519	0.835	0.609	0.854	53.175 s

The same local run also produced the following per-focus results:

Focus	Precision	Recall	F1	Correlator recall	Avg. inference time
User	0.575	0.823	0.647	0.876	60.087 s
Role	0.408	0.868	0.535	0.767	39.577 s
Service	0.333	0.750	0.462	1.000	29.273 s

The result reflects the intended design trade-off. The final schema-matching configuration prefers recall, especially correlator recall, over strict precision. This is acceptable for the delivered workflow because midPoint performs later validation, filtering, scoring, and human review of broader candidate sets.

The experimentation in this area was not limited to a single static prompt. The prompt evolved incrementally based on analysis of repeated false negatives, comparison of TP/FP/FN behavior across datasets, and targeted prompt refinements for semantically indirect matches. Additional experiments tried passing the focus type explicitly to the model and also splitting runs into separate `User` and `Non-User` prompt variants. These changes did not bring measurable improvements, so the final iteration kept a single-template approach and focused on better recall of semantically plausible candidates, especially for correlator-related attributes.

The earlier prompt iterations are not reported as final benchmark results. They were used internally to guide prompt tuning and to confirm that recall-oriented matching was more useful for downstream midPoint review than stricter precision-oriented prompting. The final report therefore focuses on the final evaluated configuration and on the model comparison below.

The project also ran a multi-model benchmark on the same version setup to compare how the recommended local model behaves against hosted alternatives under the same schema matching prompt:

Model	Precision	Recall	F1	Correlator recall	Avg. inference time per schema
<code>gpt-oss-120b</code> local GPU	0.519	0.835	0.609	0.854	53 s
<code>openai/gpt-oss-20b</code>	0.510	0.822	0.600	0.847	15 s
<code>google/gemini-3-flash-preview</code>	0.410	0.920	0.543	0.927	75 s
<code>openai/gpt-5.4</code>	0.310	0.962	0.456	0.971	201 s
<code>nvidia/nemotron-3-nano-30b-a3b</code>	0.516	0.598	0.528	0.628	161 s

The benchmark showed that the best model depended on the optimization target. The recommended delivered model is the locally deployed `gpt-oss-120b`: it had the strongest F1-score in this comparison, the highest precision among the evaluated options, solid correlator recall, and the practical advantage of running on Evolveum-controlled GPU infrastructure. `openai/gpt-5.4` achieved the highest recall and correlator recall, but at a substantial latency cost and with clearly lower precision. `google/gemini-3-flash-preview` provided a recall-oriented profile with lower latency than `openai/gpt-5.4`, while `openai/gpt-oss-20b` remained the strongest hosted baseline by F1 the among the listed hosted benchmark models. `nvidia/nemotron-3-nano-30b-a3b` showed weaker recall and also produced the largest number of hallucinations, so it was not considered a strong candidate for the delivered solution.

At the same time, these numbers should be interpreted strictly as LLM-stage matching results. They were useful for steering the prompt design and model selection, but they are not yet a substitute for final solution-level evaluation. This supported the final design decision not to rely on raw LLM output alone, but to combine schema matching with midPoint-side filtering, validation, and downstream correlation scoring.

Mapping Script Experiments

For mapping generation, the project maintained a separate experiment. This experiment focused on whether an LLM can infer a deterministic Groovy transformation script from a small number of source-target examples. Compared with older mapping experiments, this one moved closer to the production flow used by the delivered solution. It evaluated not only the generated script itself, but the full component-level chain consisting of prompting, structured response parsing, Groovy sandbox execution, and output comparison against expected results.

The dataset design changed during the project. The earlier datasets were mostly focused on **N:1** mapping tasks, where multiple input attributes are transformed into a single output value. Later experiments also needed to cover simpler but common **1:1** transformations.

The current benchmark therefore combines several data sources. A smaller challenge-oriented part was created manually to reproduce concrete data issues observed during development,

including deliberately injected anomalies such as person-name shortening, typos in numeric identifiers, and inconsistent source-target pairs. Most of the remaining datasets were generated with AI from domain-specific source scripts extracted from exported mapping collections. The data generation used GPT-5.4 to synthesize approximately 40 source-target pairs per script, after which the datasets were manually reviewed and adjusted.

The first production-style run was executed with the locally deployed `gpt-oss-120b` model on Evolveum's GPU infrastructure. The measured LLM generation latency for this local GPU run was `448.23 s` in total, or `11.21 s` per dataset on average. The measured end-to-end runtime, including generation, parsing, sandbox execution, and evaluation, was `93.51 s`, or `2.34 s` per dataset on average. There were two error cases of execution-level failures: one `comment_only_script` classification and one Groovy runtime failure. The one incorrect case was an incorrect but executable transformation.

The same 40-dataset benchmark was also executed across several hosted models to compare script-generation quality and runtime behavior under the same prompting, parsing, sandboxing, and evaluation flow. The resulting aggregate scores are:

Model	Datasets	Correct	Incorrect	Error	Success rate	Avg. LLM latency	Avg. end-to-end time
<code>gpt-oss-120b</code> local GPU	40	37	1	2	92.5 %	11.21 s	2.34 s
<code>openai/gpt-oss-20b</code>	40	31	7	2	77.5 %	28.94 s	6.45 s
<code>openai/gpt-5.4</code>	40	35	5	0	87.5 %	2.34 s	1.68 s
<code>nvidia/nemotron-3-nano-30b-a3b</code>	40	19	2	19	47.5 %	82.31 s	15.19 s
<code>google/gemini-3-flash-preview</code>	40	32	6	2	80.0 %	1.93 s	1.58 s

In this benchmark comparison, the locally deployed `gpt-oss-120b` achieved the highest success rate overall. Among the listed hosted models, `openai/gpt-5.4` achieved the highest success rate and produced no execution errors. `google/gemini-3-flash-preview` was the fastest model in this run, while `nvidia/nemotron-3-nano-30b-a3b` showed a high number of execution-level failures on this prompt and sandbox setup.

These results are useful as a validation of the mapping-generation component and of the production-style prompting-and-validation flow. However, they still do not represent the final evaluation of the full delivered mapping solution in midPoint.

The delivered solution also includes midPoint-side heuristics, quality comparison between `as-is`, heuristic, and AI-generated variants, sampling of representative data, validation feedback, and retry behavior. The complete end-to-end evaluation of schema matching and mapping inside the delivered onboarding pipeline is consolidated in [Section 4.7](#).

4.2.7 Current Limitations

The current implementation already goes beyond naive prompt-based mapping, but it still has practical limits. Schema matching quality depends on the quality of available schema metadata and attribute descriptions. Schema mapping quality depends on having representative correlated data and examples from which useful transformation logic can be inferred and based on which it can be validated.

The current microservice-side mapping endpoint is built around single-attribute mapping jobs. This is sufficient for many practical cases, but more complex many-to-one or context-heavy mappings remain harder and still require expert review and manual refinement. The single-attribute constraint is enforced by the service even though the request schema uses lists for application and midPoint attributes. The midPoint-side quality assessment also focuses on comparable single-valued samples; multi-valued sample pairs are skipped during scoring.

The final solution also intentionally keeps humans in the loop. Users are expected to review suggestions, and the broader process may require repeated iterations between mappings and correlation to stabilize the result. This is consistent with the contract documented in the smart integration API itself: mapping and correlation suggestions are meant to be developed together, not treated as one final automatic pass.

4.2.8 Complex Attributes: Experiments and Initial Design Direction

Complex attributes were not incorporated into the delivered production mapping workflow, but they remained an important research and design topic throughout the project. This section therefore documents project learning and future design direction, not a production feature that should be evaluated as part of the delivered midPoint mapping workflow.

The topic was already recognized in Milestone 1, which identified structured and multi-valued attributes such as names, emails, phone numbers, and addresses as a major practical limitation of the traditional simple-attribute model. The motivation was straightforward: if such data had to be handled only through an ad hoc JSON or string encodings, the resulting mappings and connector logic would become substantially more complex, less transparent, and harder to validate and maintain.

For that reason, part of the project effort focused on preparatory work that could reduce this future complexity. On the midPoint side, the codebase already contains architectural groundwork for richer handling of structured attributes, although this capability is not yet part of the delivered smart integration production workflow. On the smart integration side, the mapping API and surrounding design discussions also reflected the broader ambition to support more than single flat attribute-to-attribute transformations.

The intended direction of this work is more specific than simply "support nested data". In the long run, the smart integration pipeline should be able to work not only with flat single-attribute transformations, but also with structured and multi-valued data, such as **name**,

`emails[*]`, `phoneNumbers[*]`, or `addresses[*]`, including their sub-attributes. Instead of forcing connectors and mappings to serialize such values into opaque strings, the platform should be able to represent them natively in schema definitions, item paths, and smart integration requests.

From the pipeline perspective, this would require three connected layers. First, schema serialization would need to expose structured attributes and their subfields to the smart integration service instead of flattening or omitting them. Second, matching would need to reason over whole structures or grouped subfields, for example, recognizing that `emails[*]/value` and `contact/email[*]/address` are the primary value-bearing parts of corresponding records, while `type`, `category`, or `primary` provide additional context. Third, mapping would need to evolve from producing only a transformation script for a single flat attribute to suggesting declarative transformations between structured values, including rules for pairing individual entries within multi-valued collections.

Milestone 1 also identified another prerequisite for such support: more expressive item paths with natural or business keys. For complex multi-valued data, it is not sufficient to address entries by position alone; practical mappings often need to refer to concepts such as "work email", "home phone", or "primary address". This is why the complex-attribute direction is closely related to future work on more human-readable addressing of values and, eventually, richer path or filter expressions over structured data.

This direction can be illustrated by selectors such as `emails/[primary=true]/value` or `emails/[type='work']/value`, where the path does not simply address an indexed value but selects a complex entry by its business meaning. That is important both for "complex-to-simple" cases, such as selecting a primary email into a legacy single-valued property, and for "simple-to-complex" cases, where a single source value should create or update one entry inside a structured multi-valued collection.

Another important part of the proposed model is to separate reusable structure-level translation logic from object-type-level mappings. In such a model, details of how one structured type maps to another would live in a dedicated reusable definition, while concrete object types would only declare that, for example, `ri:permanentAddress` maps to `permanentAddress` and delegate the internal field-level translation to the shared type definition. This would reduce duplication and make complex mappings more declarative and easier to reuse across different resource object types.

The same design direction is relevant for multi-source scenarios in which several systems contribute values into the same complex collection. Examples include separate sources of work and personal emails, provenance-aware updates of existing entries, and conflict-resolution rules around flags such as `primary`. Once complex attributes are represented natively, future matching and mapping suggestions would no longer be limited to one flat value at a time, but could instead help identify, merge, preserve, or replace whole structured entries within a collection.

4.3 Correlation Recommendation

4.3.1 Purpose

Correlation recommendation helps determine how imported accounts or other resource objects should be matched to existing identities in midPoint. This is a critical onboarding step because poor correlation can create either missed matches or unsafe joins.

4.3.2 Implemented Approach

The final implementation uses a pragmatic two-layer approach instead of the more ambitious logic originally considered during the earlier project phase. The first layer reuses schema matching with explicit emphasis on correlator recall. In the `midpilot-smart-integration` service, the matching prompt explicitly prioritizes avoiding missed correlator candidates and distinguishes between `User` and `Non-User` correlator sets.

The second layer is implemented in midPoint itself. MidPoint does not try to generate arbitrary correlation strategies from scratch. Instead, it starts from a focus-type-specific set of known likely correlators and checks how well these candidates are supported by schema matches and by real data. For `User`-like objects, this includes paths such as `name`, `personalNumber`, and `emailAddress`, while for non-user focus types it includes `name`, `identifier`, and `emailAddress`, according to the applicable focus class.

4.3.3 Final Component Design

The final design is intentionally simple and explainable. MidPoint first obtains schema matches for correlator-relevant focus properties. For each candidate correlator path, it then checks whether there is already an applicable inbound mapping. If such a mapping exists, the existing mapping can directly support correlation suggestion. If not, midPoint derives a draft correlation mapping from the schema match and wraps it into a correlation suggestion.

The resulting candidates are not accepted blindly. They are evaluated by sampling focus objects and resource shadows and by scoring the resulting links. The implemented evaluator measures whether the candidate provides useful shadow-to-focus coverage while penalizing ambiguity in both directions. This means that the final recommendation is based on the combined effect of AI-supported matching, focus-type-specific correlator selection, existing configuration reuse, and data-driven scoring in midPoint.

4.3.4 Integration into MidPoint and Related Tooling

Correlation recommendation is integrated directly into the smart integration API and into the resource object type wizard in midPoint. The API exposes synchronous and task-based operations for suggesting correlation and for checking the status of submitted suggestion jobs. In the GUI, correlation suggestions are surfaced through the correlation wizard flow and

through the smart integration status-handling utilities that wrap and manage suggested correlator items.

The capability is also explicitly linked to mapping recommendation. The smart integration API documents that mappings and correlation suggestions should be created together and may need to alternate iteratively until the configuration stabilizes. This reflects the implemented design, where useful correlation often depends on available mappings and where suggested correlation may itself require a missing inbound mapping to be added.

4.3.5 Delivered Functionality

The delivered functionality includes:

- Schema matching prompts that prioritize high recall for correlator-plausible attributes.
- Focus-type-aware selection of known probable correlators in midPoint.
- Reuse of already existing inbound mappings when they can support correlation.
- Creation of draft correlation mappings when schema matches indicate a viable correlator path.
- Simulation-oriented evaluation of candidate correlators on sampled focus and shadow data.
- Use of existing and draft inbound mappings as part of correlation simulation and scoring.
- Scoring of suggested correlators on sampled focus and shadow data using coverage and ambiguity penalties.
- Filtering that keeps only the highest-scoring suggestion for each correlator path.
- Integration of the resulting suggestions into the midPoint resource configuration workflow.

This design is simpler than some of the earlier milestone ideas, but it is also more robust and easier to reason about. It avoids overcomplicating the correlation problem while still combining AI assistance with concrete product logic and data-based validation.

4.3.6 Evaluation and Validation

The delivered implementation already contains product-level validation mechanisms. Suggested correlators are not ranked only by schema semantics, but by their observed behavior on sampled focus and shadow objects. The score rewards useful coverage and penalizes ambiguous links, which makes the recommendation more representative of practical correlation quality than schema-only reasoning. This simulation-oriented step is also important for inbound mappings: a candidate correlator can be evaluated together with the inbound mapping that would feed the relevant focus attribute, including draft mappings derived from schema matches.

The codebase also contains dedicated tests for correlation suggestion and correlator evaluation. These tests cover behavior such as selecting the best correlator candidate, excluding weak

suggestions, and evaluating the effect of uniqueness, ambiguity, and missing values on the resulting correlation score.

At the component level, the most relevant experimental evidence for correlation recommendation comes from the schema-matching benchmark described above in the [Schema Matching Experiments](#) section. This is because the first step of the delivered correlation workflow depends on whether the matching stage is able to surface plausible correlator candidates for the relevant focus-type-specific paths. For that reason, the reported correlator-recall results from the schema-matching experiments should be read as an upstream evaluation of correlator discovery quality, not only as a generic schema-matching metric.

The final end-to-end evaluation of correlation recommendation belongs to the complete pipeline evaluation in [Section 4.7](#). That evaluation should capture the combined effect of schema matching, focus-type-specific correlator selection, existing mappings, draft inbound mappings, and midPoint-side data-based scoring.

4.3.7 Current Limitations

The current implementation is intentionally pragmatic and does not attempt to synthesize arbitrarily complex correlation strategies. It works best when relevant correlators are already represented among the known focus-type-specific paths and when the available data support meaningful uniqueness and coverage analysis.

Correlation quality remains sensitive to real data quality, missing mappings, and resource-specific identity conventions. The implementation also currently prefers simple item-based correlator suggestions; this is appropriate for many onboarding scenarios but does not eliminate the need for expert review in more complex environments.

4.4 Delineation and Focus Type Suggestion

4.4.1 Purpose

Delineation support addresses the problem of identifying and configuring object types and their boundaries in imported resource data. Its intended purpose in the project was to make object type setup more systematic and to reduce manual trial-and-error-based setup flow during onboarding.

4.4.2 Delineation

The delivered delineation approach combines AI-assisted object type suggestion with midPoint-side statistical analysis and validation. In the `midpilot-smart-integration` service, the `object_type` module receives the schema of a single object class together with computed statistics and asks the language model to suggest `kind`, `intent`, and delineation rules expressed as MQL filters and optional `baseContextFilter` clauses.

The prompting logic is not based only on schema names. It also uses statistical characteristics of the data, such as missing-value ratios, unique-value ratios, frequent literal values, extracted value patterns, and cross-attribute tuple statistics. The prompt explicitly requires mutually exclusive, stable, IGA-relevant partitions and prefers simple, robust delineations over needlessly complex rule sets.

Compared with milestone 1, the final implementation is narrower and more pragmatic. The milestone text described a broader exploratory direction with richer statistical profiling and more open-ended rule discovery experiments. The delivered solution kept the central idea of statistics-assisted delineation, but reduced it to a more controlled workflow centered on stable value distributions, pattern extraction, crosstabs, executable MQL generation, and backend validation.

4.4.3 Focus Type Suggestion

The smart integration pipeline also includes a dedicated focus-type predictor. In the current implementation, midPoint sends the object type kind and intent, the serialized application schema, and optional base-context delineation to the `midpilot-smart-integration focusType/suggestFocusType` endpoint. The microservice uses a lightweight LLM prompt constrained to the finite set `UserType`, `RoleType`, `OrgType`, and `ServiceType`, and returns a single suggested focus type. This process remains a human-in-the-loop workflow, where the suggested focus type must be reviewed and confirmed by the user during the integration process.

4.4.4 Final Component Design

The final design is a closed loop between statistics, AI suggestion, and backend validation. MidPoint computes statistics for object classes and object types and sends these statistics to the smart integration service together with the serialized schema. The service returns suggested object types that may contain one or more delineation rules for the same object class.

MidPoint then parses and validates the returned filters and base-context definitions. If validation fails on the first attempt, structured validation feedback is sent back to the service for a corrective retry. If some issues still remain after the retry, midPoint keeps partial results and marks invalid parts explicitly instead of discarding the whole suggestion set.

This makes delineation a practical assisted capability rather than a free-form LLM output. The delivered behavior depends on the combined effect of statistics computation, prompt constraints, filter validation, retry with backend feedback, and user review in the wizard flow. This is also the main difference from the milestone 1 vision: instead of continuing toward increasingly complex exploratory rule synthesis, the final design converged on a simpler and more robust suggestion-validation loop that better fits real product workflows.

4.4.5 Integration into MidPoint and Related Tooling

The capability is integrated into the smart object type suggestion workflow in midPoint. The

backend exposes object type suggestion operations that consume resource object class statistics, while the GUI provides dedicated smart object type suggestion panels and statistics views for object classes, object types, and focus types.

The statistics part is not auxiliary but integral to the delivered delineation design. MidPoint contains dedicated statistics services and activity handlers for computing and persisting object class, object type, and focus object statistics. The GUI can display and regenerate these statistics, which makes the delineation workflow inspectable and iterative instead of opaque.

4.4.6 Delivered Functionality

The delivered functionality includes:

- An `object_type` microservice module for suggesting object types and delineation rules from schema and statistics.
- Support for `kind`, `intent`, `displayName`, `description`, `filter`, and `baseContextFilter` suggestions.
- Prompt guidance for stable IGA-relevant delineation using value counts, pattern counts, DN suffixes, and cross-attribute crosstabs.
- Backend validation of suggested MQL filters and base-context filters in midPoint.
- Retry of invalid suggestions with structured validation feedback returned to the microservice.
- Preservation and explicit marking of partially invalid suggestions when full validation does not succeed.
- GUI support for reviewing suggested object types and inspecting the underlying statistics.

The resulting capability is more concrete than a pure research prototype. At the same time, it remains an assisted configuration workflow, not a fully autonomous delineation engine.

4.4.7 Evaluation and Validation

The delivered solution contains validation directly in the product flow. Suggested delineation filters are parsed and checked against the relevant object class definitions, base-context filters are validated separately, and invalid suggestions can be retried with structured backend feedback. This means the final behavior is already constrained by executable validation, not only by prompt instructions.

The statistics layer itself is also part of the validation story. MidPoint computes concrete frequency and pattern information from real shadow data, including value counts, missing values, DN suffixes, token-like prefixes and suffixes, and object-type-specific statistics. These statistics make delineation suggestions less dependent on naming intuition alone and more grounded in observed data distributions.

As part of the development of delineation support, we performed a series of experiments to find

the optimal combination of input statistics and LLM prompts that would lead to stable and IGA-relevant rules.

Delineation Statistics: Overview of Experiments

Originally, we experimented with a wide range of signals, including:

- **Cardinal and missing values:** `unique_count`, `unique_ratio`, `missing_ratio`.
- **Value distribution:** Top-N most frequent values (without constraints, it proved to be too noisy).
- **Patterns:** fixed-length prefixes/suffixes, n-grams, regex buckets (digits only, hex, etc.).
- **Numerical analysis:** quantiles and histograms.
- **Dependencies (Crosstabs):** Cramér's V, χ^2 (chi-square) tests.

Rationale for the Final Delineation Configuration

Experiments showed that "heavy" statistical dependency metrics (such as Cramér's V) caused practical issues:

- **Low IGA relevance:** Strong statistical ties often reflected only technical artifacts of migrations, not stable business partitions.
- **LLM confusion:** Models tended to combine statistically "linked" attributes into brittle rules that were not robust.
- **Token pressure:** Complex crosstabs unnecessarily inflated the prompt without adding value to the final quality.

We therefore narrowed the final solution down to **low-noise and interpretable signals**:

- `uniqueValueCount` and `missingValueCount` (cardinal metrics).
- `valueCount` (Top 10 values, only if they repeat at least twice).
- `valuePatternCount` (DNsuffix for LDAP structure, delimiter-aware tokens for technical identifiers).

Explanation of Evaluation Metrics

To understand the model performance results, we used the following metrics:

- **Coverage:** The fraction of all records for which the model was able to assign a prediction (a specific `intent`). High coverage is critical for end-to-end usability, as a model that only predicts for a small subset of data is not practically useful even if its accuracy on that subset is high.
- **V-measure:** A harmonic mean between **Homogeneity** and **Completeness**. It provides a balanced measure of clustering quality, ensuring that the model does not just put everything

into one big group (low homogeneity) or create a unique group for every single record (low completeness).

- **Completeness:** Measures whether records from the same true category are assigned to the same suggested group. A high score means that one real intent is not unnecessarily split across many generated object types.
- **Homogeneity:** Measures "cluster purity". A high score means that each suggested group (intent) contains only records that belong to a single true category in the original data. In IGA, high homogeneity ensures that the suggested rules do not mix different types of identities or entitlements together.
- **Predicate Errors:** The number of runtime exceptions encountered when executing the generated MQL filters. A value of 0 is essential for stability, confirming that the LLM-generated rules are syntactically and semantically valid for the midPoint backend.

Delineation Model Evaluation Results

Final benchmarking on six representative delineation datasets showed high stability and quality of the proposed rules. The final run for the delivered solution used the locally deployed **gpt-oss-120b** model on Evolveum's GPU infrastructure. The generated MQL rules were applied back to the datasets, evaluated against expected **intent** and **kind** labels, and checked for predicate execution errors and overlapping rule matches. For comparability with the earlier model benchmark, the local **gpt-oss-120b** result is shown using the same main metrics:

Model	Coverage	V-measure	Homogeneity
gpt-oss-120b local GPU	~0.994	~0.806	~0.972
openai/gpt-5.2	~0.994	~0.816	~0.994
openai/gpt-oss-20b	~0.994	~0.762	~0.951
qwen/qwen3-vl-32b	1.000	~0.788	~0.871
google/gemma-3-27b	~0.802	~0.825	~0.980

On the three reported metrics, **openai/gpt-5.2** remains the strongest model overall: it combines near-complete coverage with the highest V-measure among the high-coverage models and the highest homogeneity. The locally deployed **gpt-oss-120b** is very close in coverage and only slightly lower in V-measure and homogeneity, while also producing executable rules with no predicate errors in the final local run. For the delivered solution, this makes local **gpt-oss-120b** a strong practical choice even though **openai/gpt-5.2** is still the best pure benchmark result.

Key findings:

- **openai/gpt-5.2** is the best model by the combined benchmark view of coverage, V-measure, and homogeneity.
- The local **gpt-oss-120b** run provided a stable final local result with near-complete coverage

and no predicate execution errors.

- Coverage remained a key practical metric: a delineation approach that leaves many rows unlabeled is not useful in onboarding, even if the labeled subset looks clean.
- Delineation has proven in practice to be a successful assisted workflow, where statistics ground the LLM in the reality of the data instead of relying solely on intuition from attribute names.

The final evaluation of focus type suggestion is included in the complete pipeline evaluation in [Section 4.7](#), where it can be assessed together with object type delineation.

4.4.8 Current Limitations

Delineation quality still depends strongly on the quality and representativeness of the available statistics. If the observed data do not expose stable partitions, the best result may legitimately be a single default rule instead of a richer decomposition.

The current design also intentionally restricts itself to simple, stable, IGA-relevant filters. That improves safety and interpretability, but it means the capability will not capture every domain-specific partitioning pattern automatically. Expert review is therefore still required, especially for complex or highly organization-specific object type taxonomies.

4.5 AI Development and Testing Tooling

4.5.1 Purpose

The role of AI development and testing tooling in the project was to make experimentation, tracing, validation, and review of AI-assisted behavior manageable. Without such tooling, prompt-based and heuristic-based features would have been difficult to evaluate and maintain.

4.5.2 Implemented Toolchain

The project assembled a practical toolchain rather than delivering a single all-in-one AI platform. The key implemented tools were **Langfuse** and **ClearML**. **Langfuse** was used for tracing and observability of AI-assisted service execution, including prompt flows, retries, and runtime debugging of the smart integration microservices. **ClearML** was used for experiment tracking, evaluation runs, and management of notebooks and scripts used during development and validation of the AI components.

Compared with milestone 1, the final outcome in this area is best described as a practical realization of the **AI Studio** concept in the form of selected working tools, not as one separate integrated product named **AI Studio**.

4.5.3 Integration into the Solution Lifecycle

These tools were used during feature development, prompt iteration, evaluation of recommendation quality, and debugging of AI-assisted service interactions. **Langfuse** supported inspection of traces from running services and helped analyze retry loops, prompt behavior, and runtime failures. **ClearML** supported repeatable experiment execution and tracking of evaluation-related development work around notebooks and service-level experiments.

4.5.4 Model and Prompt Tuning

During the experiments, the project tried different model families, prompt variants, and runtime settings to understand their effect on quality, latency, and stability. Model choice was not driven only by raw benchmark quality. An important requirement was to use a model that could be operated safely in the project environment, preferably an open-source or open-weights model, ideally European, that could run on Evolveum-controlled infrastructure. For this reason, the project tested several European and open model alternatives before converging on the **gpt-oss** model family for the final evaluated direction.

After that decision, many experiments were performed with **gpt-oss-20b**, different prompt versions, and different settings for reasoning effort and temperature. Reasoning effort had a visible effect on both quality and latency: the smaller **gpt-oss-20b** model often needed higher reasoning effort to approach the quality of the larger model, which increased runtime. In repeated experiments, the locally deployed **gpt-oss-120b** proved more advantageous for the final workflow because it was more stable and reached stronger results without relying on the smaller model's extra reasoning budget. For final runs, the project mostly used high reasoning effort and temperature **1**, following the recommended/default setup for the evaluated OpenAI model family.

Prompt tuning focused on making the task boundaries explicit, requiring structured outputs, preventing invented schema attributes, and feeding validation errors back into retry attempts. Hosted frontier models were used as quality references, and the local **gpt-oss-120b** results were continuously compared against these top benchmark models. The comparison showed that its performance was comparable for the target tasks and metrics.

The locally deployed **gpt-oss-120b** therefore became the recommended delivered option because it combined sufficient quality, better repeatability in the project experiments, acceptable latency, predictable operating cost, and the security advantage of running on local GPU infrastructure.

Granular data control was part of the same tuning process. The implemented workflows avoid sending arbitrary raw data when smaller representative samples or derived statistics are sufficient, and they keep separate data samples for prompting, validation, simulation, and scoring.

4.5.5 Delivered Functionality

The delivered result in this area is a usable engineering toolchain for experimentation, tracing, and validation of AI-assisted onboarding features. This includes:

- Trace-level observability of AI-assisted microservice execution through **Langfuse**.
- Experiment and evaluation tracking through **ClearML**.
- Practical support for notebook- and script-based development workflows.
- A concrete operational basis for debugging and evaluating the delivered AI-assisted onboarding components.

4.5.6 Current Limitations

The project does not converge on a single finalized **AI Studio** product. Instead, it produced a practical but still evolving toolchain centered on **Langfuse** and **ClearML**. This is sufficient for project delivery, but there is clear room for broader standardization, tighter integration, and a more unified user and operations model in the future.

4.6 UX and User-Facing AI Workflows

The web user interface of midPoint represents the primary user-facing entry point for AI-assisted workflows in the project. In the final solution, AI capabilities are not exposed as a separate free-form assistant interface but rather as guided actions embedded into concrete configuration steps. This approach is particularly relevant for onboarding a new resource and for configuring key parts of the integration in a structured and reviewable way.

This UX section should be read as documentation of the user-facing design direction around the delivered AI capabilities, not as a list of independent backend components. It combines three kinds of material: the delivered onboarding-oriented web workflows, expert workflows in midPoint Studio, and design work that applies the same model of reviewable suggestions to related configuration areas. The end-to-end evaluation in [Section 4.7](#) focuses on the main onboarding pipeline, especially object type, correlation, mapping, simulation, and connector generation workflows. Other UX subsections, such as association suggestions and the broader mapping-interface redesign, document how the same interaction principles are intended to extend to adjacent midPoint configuration areas.

From the perspective of the delivered smart integration workflow, the most important user-facing web capabilities are object type suggestion, mapping recommendation, and correlation recommendation. These capabilities are designed to help the user accelerate resource onboarding by generating candidate configurations from schema information, statistics, and contextual metadata, while still keeping the user in control of the final decision. Instead of directly applying generated results, the UI workflow exposes them as suggestions to be reviewed, confirmed, modified, or rejected by the user.

A key property of the final web workflow is the balance between automation and validation. The purpose of AI assistance is to reduce the amount of manual configuration work, especially in repetitive or heuristically recognizable tasks, but not to bypass product-side control mechanisms. For that reason, the generated results are tied to validation and correction loops instead of being treated as authoritative output. In practice, this means that the web UI serves as a guided onboarding environment in which AI generates candidate configurations, the backend validates them, and the user remains responsible for the final acceptance of the result.

Compared with the earlier concepts, the final solution is therefore more pragmatic and product-oriented. The web workflow does not attempt to present AI as a fully autonomous configuration engine. Instead, it converges on a model in which AI is used as a structured assistant within explicit wizard-like or step-based flows. This makes the resulting UX more predictable, easier to validate, and more compatible with midPoint's existing configuration and security model.

4.6.1 Onboarding workflow overview

The delivered web UI onboarding flow extends the existing resource creation wizard rather than replacing it. The basic wizard still covers the elementary resource configuration, connector setup, connection configuration, discovery and schema creation. Its UX changes focus on clearer navigation, consistent wizard actions, and clearer distinction between available and locked creation options.

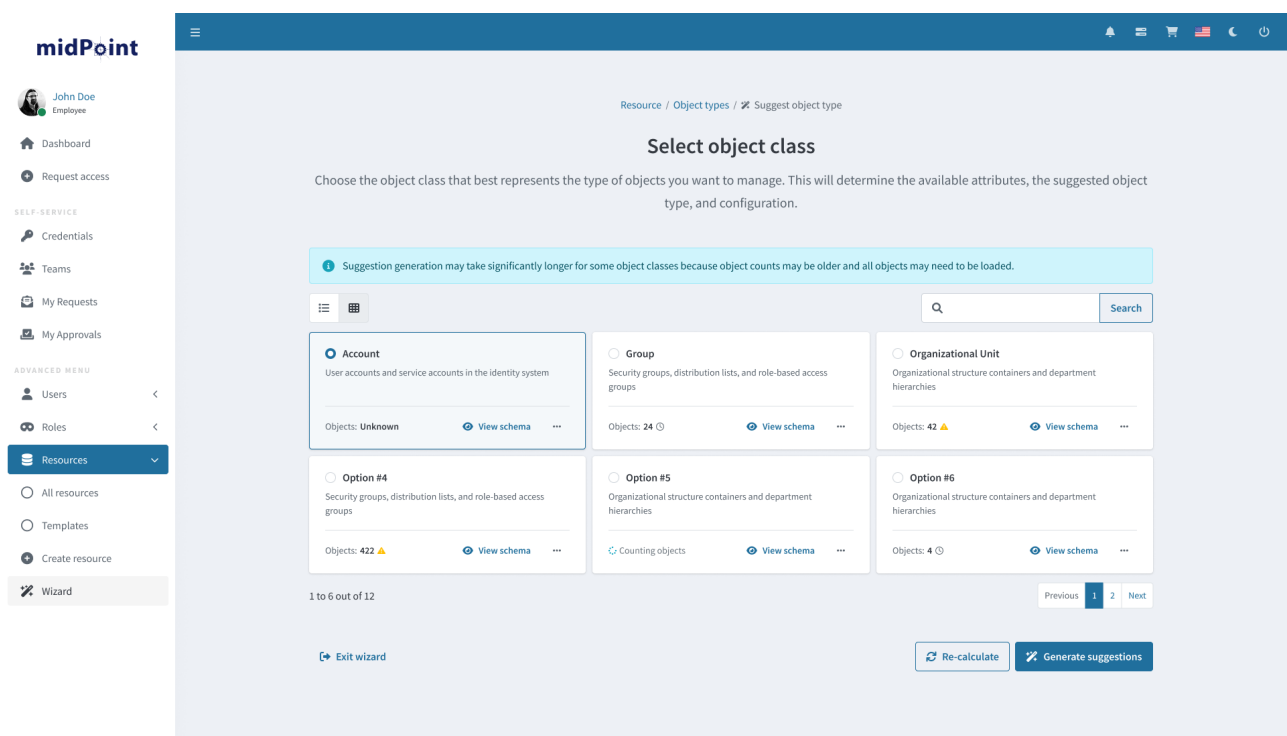
The main UX shift starts after the resource is created. The user is no longer left with a completed resource object and a broad configuration surface. Instead, the UI continues the onboarding journey through tiles with recommended actions, disabled states, resource data preview, statistics, and AI-assisted configuration actions.

This model keeps AI assistance tied to concrete configuration tasks. AI helps prepare candidates for object type, focus type, mapping, or correlation setup, but the user still reviews, edits, and explicitly saves the configuration. Resource onboarding therefore becomes a guided review flow: create the resource, inspect the data, configure object types, continue into related configuration areas, and validate the result before the resource becomes operationally ready.

4.6.1.1 Object Types and Focus Type Suggestions

Object type configuration is the first major smart integration branch after a resource is created. The resource connection and schema already exist, but the user still needs to define how resource objects should be interpreted in midPoint terms and how they should be delimited from available resource data.

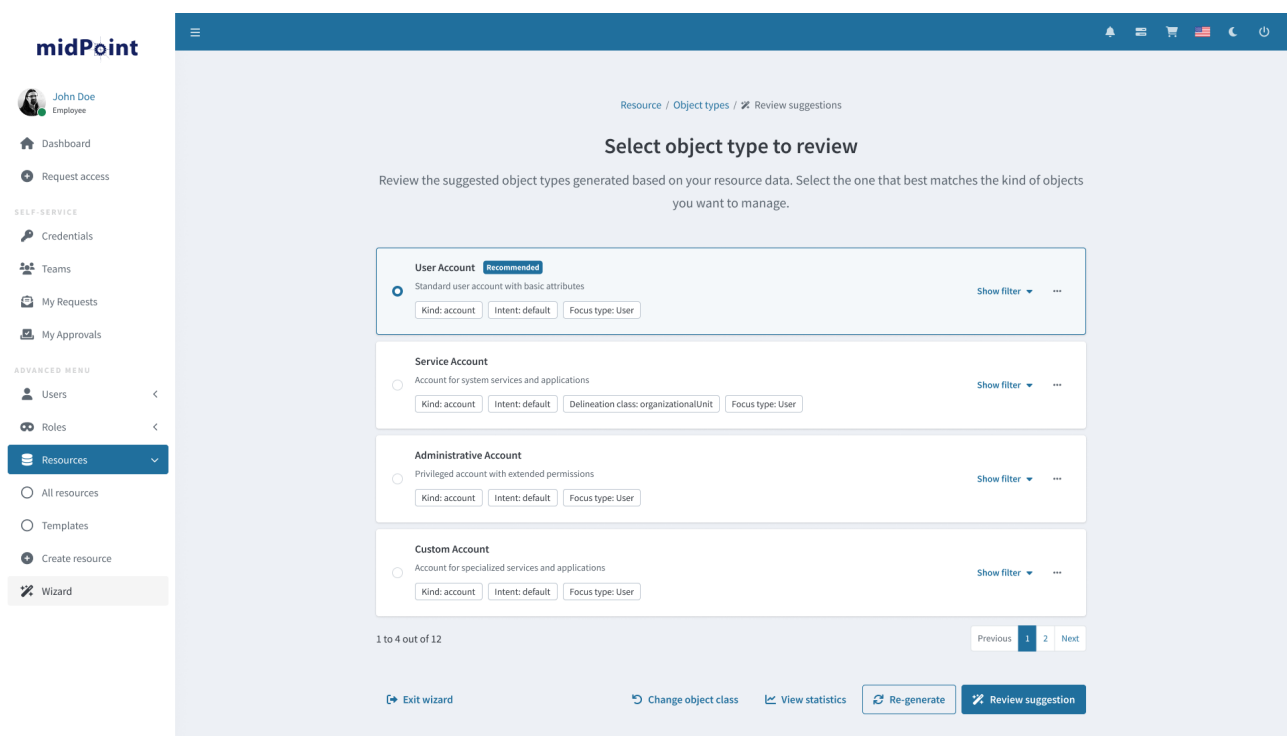
The implemented flow supports both manual object type creation and AI-assisted suggestions. When no object types exist yet, the Object Type Manager lets the user either add one manually or generate suggestions. The suggestion flow starts with object class selection and gives access to schema and resource statistics before the generation starts.



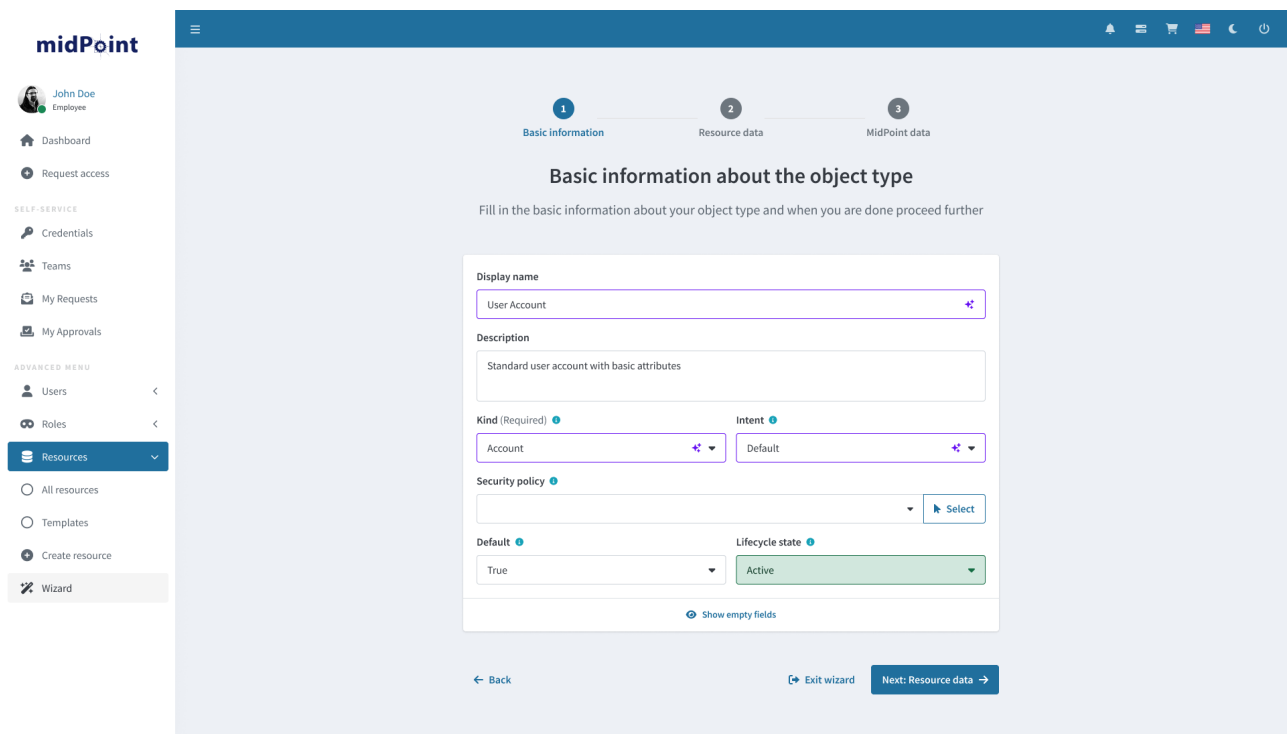
The AI-assisted generation is explicit and controlled. Before suggestions are created, the user decides whether the schema and statistical data can be used by the AI service. The UI also explains that authentication credentials and user passwords are not shared.

The generation process is shown as a visible task. The user can wait for completion, suspend the task, or run it in the background. If background processing is used, the Object Type Manager shows an information banner until generated suggestions are available.

Generated object types are presented as reviewable candidates. The user can inspect suggested names, descriptions, kind, intent, object class, focus type, and filters. Suggestions can be reviewed, refreshed, discarded, or compared with existing object types before any configuration is saved.



Reviewing a suggestion opens the regular object type wizard with generated values prefilled and visually marked by an AI badge and purple highlighting. The user still reviews basic information, resource data, and midPoint data before saving. The focus type suggestion is part of the same flow, either as a generated value in the suggestion or as a separate action when the object type is created manually.

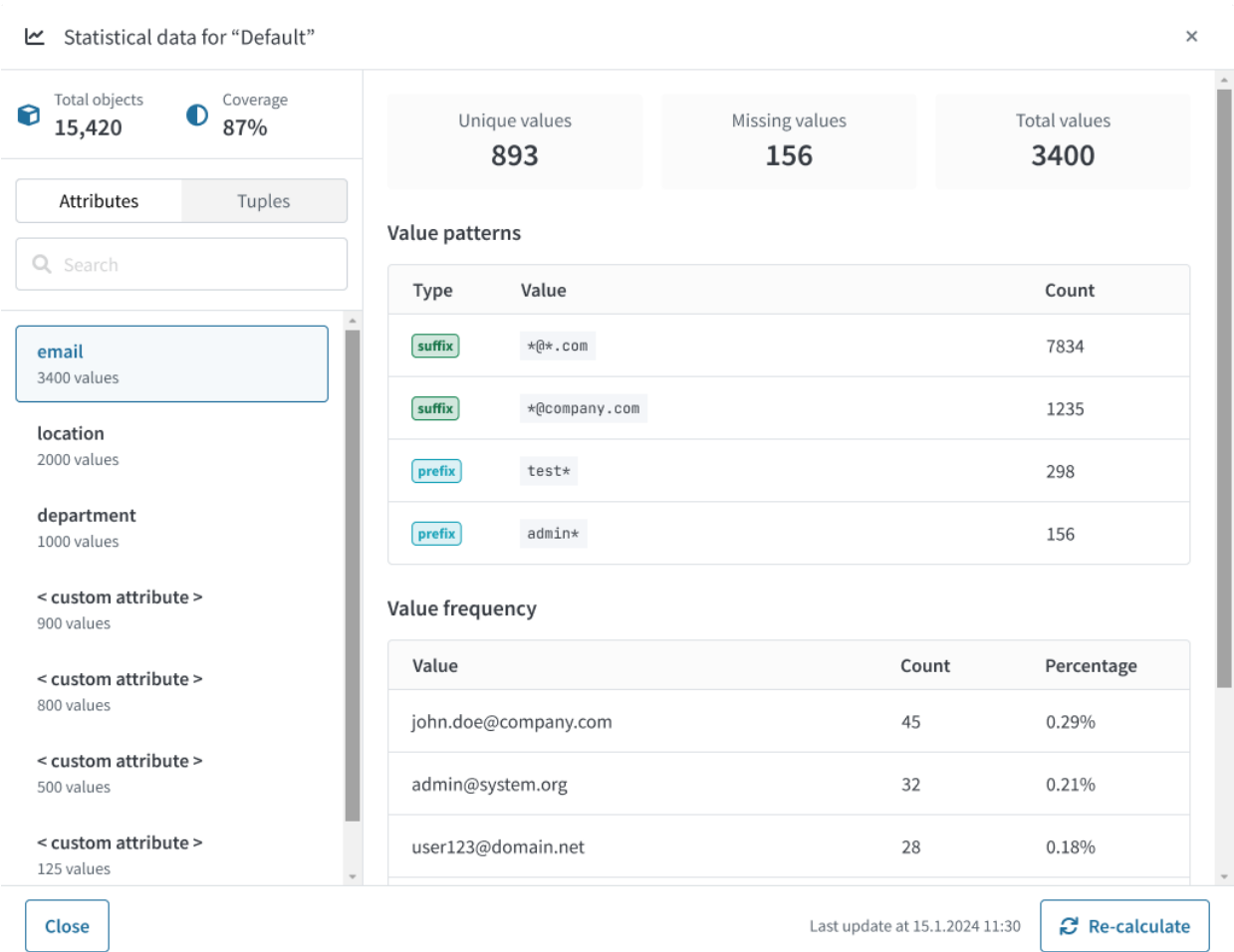


UX Challenges

- Object type configuration is conceptually difficult. Users need to understand the difference between object class, object type, kind, intent, focus type, and delineation filter.
- AI-generated object type suggestions can look convincing even when they still require a domain expert review.
- A generated filter can be syntactically valid but semantically wrong for the intended business use case.
- Long-running AI operations need visible state because suggestion generation may involve statistics, schema processing, service-side generation, validation, refresh, or retry.
- Users need to understand which values were generated by AI and which values are an existing, manual, or already accepted configuration.
- The UI must avoid making the AI output look inherently authoritative, especially because accepted object types become structural resource configuration.

UX Decisions

- Use a review-oriented suggestion model. Object types are not created automatically from the AI output, they are first shown as candidates and then reviewed through the regular object type wizard.
- Use resource data statistics as an evidence layer. Attribute distributions, value counts, unique values, empty values, and value patterns help users understand why the particular object type filters are suggested.



- Make AI data usage explicit. The permission dialog lets users decide whether the schema and statistical data can be used by the AI service.
- Visually mark generated values. AI-filled fields use an AI badge and purple highlighting, while suggested rows can be marked as AI-generated and ready for review.
- Support asynchronous work. Users can run the suggestion generation in the background and later review the generated propositions in the Object Type Manager.
- Preserve manual control. Users can add object types manually, edit generated values, discard suggestions, refresh suggestions, compare generated object types with existing ones, or turn suggestions off entirely.
- Keep the focus type suggestion inside the object type configuration flow so that it supports the user at the point where the decision is needed instead of becoming a separate AI feature.

4.6.1.2 Correlation Suggestions

After an object type is created, the object type wizard guides the user to correlation. This step is recommended because the flow already knows which resource objects are managed and which midPoint focus type they should match.

The correlation screen supports manual rule creation and AI-assisted suggestions. When no

rules exist, the empty state offers Create new and Generate suggestions buttons. Suggestion generation uses the same explicit permission model, where the user confirms which resource data can be used and is informed that credentials stay excluded.

Generated rules appear as reviewable cards. The user can inspect the proposed correlator name, reason, correlation item, match method, weight, tier, and efficiency. Each suggestion can be accepted, discarded, regenerated, or opened through the View rule button before it is saved.

If an accepted rule needs a mapping that does not exist yet, midPoint warns the user and explains what mapping will be added. The accepted rule still remains part of the working configuration until the user saves the correlation settings.

UX Challenges

- Correlation is safety-sensitive because poor matching can cause missed matches, duplicates, or incorrect links between resource objects and midPoint focus objects.
- Users need to understand why a suggested rule is plausible, not only that it was generated.
- Suggested correlation may depend on mappings that do not exist yet; this causes a cross-step dependency inside the onboarding flow.
- A rule can be technically valid while still being risky because of low coverage, ambiguous values, or weak real-world data quality.
- The interface needs to show enough detail for review without forcing users into low-level configuration too early.
- Accepting a suggestion and saving the correlation settings are separate actions, so the UI must make the unsaved state visible.

UX Decisions

- Place correlation immediately after object type creation as the recommended continuation.
- Present correlation rules as reviewable suggestion cards with accept, discard, regenerate, and view-rule actions.
- Show a reason for each suggestion through readable text and concrete rule items rather than only an AI label.
- Expose key evaluation values such as weight, tier, and efficiency to support review.
- Allow users to inspect the full rule before accepting it.
- Detect missing supporting mappings and explain that they will be added during acceptance instead of failing later.
- Keep the final save action separate so users can review accepted rules before committing correlation settings.

4.6.1.3 Synchronization Reactions

After correlation is configured and saved, the object type wizard recommends synchronization. This step moves the object type closer to operational readiness because it defines how midPoint reacts to resource object situations.

This flow is not AI-based. The user can create reactions manually or open a guided reaction generator that asks for synchronization direction and basic operational choices, such as what should happen when a new resource object is detected or deleted.

The generator creates editable default reactions in the synchronization table. The rows expose situation, action, and lifecycle state so that the user can review the generated behavior before saving it.

Generated reactions still require explicit save. After saving, synchronization can be marked as configured and the wizard can recommend the next configuration area.

UX Challenges

- Synchronization has operational consequences, so users need to understand the effect of generated reactions before saving them.
- The underlying configuration combines situations, actions, lifecycle states, and synchronization direction, which can be difficult to configure from scratch.
- Users may not know which default reactions are appropriate for source or target resources.
- The generator is not AI-based, so the UI needs to distinguish guided product logic from AI-assisted suggestions.
- Generated reactions can look complete, but they still require user review and an explicit confirmation.

UX Decisions

- Recommend synchronization only after correlation is configured because synchronization behavior depends on object matching having been defined.
- Use a guided form to collect the minimum decisions needed for default reactions.
- Keep lifecycle state visible in each generated row so that users can see whether the reaction affects production or not.
- Require user to explicitly save the configuration after generation to preserve user control.
- Treat synchronization as a step toward operational readiness rather than a hidden advanced configuration area.
- Avoid AI styling for reaction generation when the mechanism is rule-based or methodology-based rather than AI-generated.

4.6.1.4 Mapping Suggestions

After the object type, correlation, and synchronization are configured, the object type wizard recommends mappings. This step connects resource attributes with midPoint properties and moves the resource configuration from structural setup toward data flow design.

The mapping screen supports manual configuration and generated suggestions. If a previously suggested and accepted correlation rule requires a supporting mapping, the user may already see one accepted mapping in the table. Otherwise, the user can start from an empty mapping setup and generate suggestions based on the current object type context.

Suggestion generation for mappings follows the same controlled permission model as the other AI-assisted steps. The user confirms which data can be used, such as schema or raw sample data. During generation, a visible progress banner shows that mappings are being suggested and gives the user a way to stop the process.

Generated mappings are added directly into the mapping table. The UI distinguishes inbound and outbound mappings, shows whether a suggestion comes from AI or from system logic, and lets the user accept, discard, or inspect suggestions in context. If several suggestions target the same midPoint property, they are grouped so that the user can select only one applicable option.

Before accepting a mapping, the user can use simulation and statistics to check whether the suggested mapping makes sense. After a suggestion is accepted, it changes from a proposed item into an accepted mapping row. The configuration still has to be saved explicitly. After saving, the basic onboarding configuration can be considered ready for initial operation.

UX Challenges

- Mapping configuration is complex because users must understand source attributes, target midPoint properties, expression logic, direction, and lifecycle state.
- AI-generated mappings can create many candidates at once, which can overwhelm users if the suggestions are not grouped and visually differentiated.
- Multiple suggestions can compete for the same target property, so the UI must prevent users from accepting conflicting mappings accidentally.
- Suggestions from AI and suggestions from system logic need different visual treatment because they have different sources and trust models.
- Users need evidence before accepting mappings, especially when the mapping affects identity data, activation data, or lifecycle behavior.
- A mapping may look correct in the UI but still behave incorrectly when applied to real objects.

UX Decisions

- Keep mappings inside the active object type context so that the user understands which

resource object population is being configured.

- Insert generated suggestions into the same table as existing mappings so that users review them in the final configuration context.
- Use purple for AI-generated suggestions and blue for system-generated suggestions to distinguish the sources of the recommendations.
- Group competing suggestions for the same target property and allow only one to be applied.
- Keep accept and discard actions on each suggestion so that review can happen item by item.
- Provide the simulation option and show statistics so that the user can validate mapping behavior against data before accepting it.
- Require an explicit Save mappings action after acceptance so that accepted suggestions do not become committed configuration without an action by user.

4.6.1.5 Outcome of the Onboarding Workflow

Overall, the delivered onboarding flow establishes a review-first configuration model. AI and rule-based generation reduce the amount of manual setup, but generated output is shown as editable suggestions that must be reviewed, accepted, and saved by the user. The flow also uses recommendations, locked states, statistics, and simulation to make the next steps visible and to keep configuration decisions grounded in resource data.

4.6.2 Connector Generator Interface

Connector generation in the midPoint UI is an optional AI-assisted path for creating a connector. It is not part of every resource onboarding flow but it becomes relevant when the connector catalogue does not offer a suitable option for the application the user wishes to connect. Its purpose is to provide AI-assisted connector building wizard which helps users decide but keeps the control over the process in their hands.

The connector generation workflow can be started from Integrations → Connectors → Generate Connector screen, from the All Connectors view, or from resource onboarding when a missing connector is detected.

The workflow guides the user through connector creation as a sequence of controlled checkpoints:

1. Identify the target application.
2. Review discovered documentation.
3. Define connector metadata.
4. Configure and test the connection.
5. Create object classes.
6. Validate schema and search behavior.

7. Manage relationships.
8. Return to the connector summary serving as the main signpost point whence various parts of the connector can be accessed adjusted.

AI assists at each stage:

1. Discover documentation.
2. Suggest configuration values.
3. Generate scripts.
4. Recommend object classes.
5. Identify endpoints.

Every AI output remains reviewable and editable until the user confirms it as part of the connector.

This assisted connector-building process is especially beneficial for users whose specialty is not connector development:

- **Manual connector creation is too demanding:** Creating a connector without assistance requires technical knowledge, API documentation analysis, scripting, validation, and repeated testing. This makes connector creation difficult for users who are not connector developers.
- **Documentation discovery creates friction:** Users have to find relevant API documentation, judge its quality, and decide whether it is sufficient for connector implementation. This is time-consuming and error-prone, especially for older, custom, or poorly documented applications.

UX Challenges

- **Fixed wizard patterns do not fit AI-assisted generation:** A traditional wizard can suggest that the full path is known upfront. In connector generation, the next step may depend on AI findings, missing documentation, validation results, or errors encountered during script execution.
- **Generated technical output can appear too authoritative:** AI-generated values, scripts, attributes, or object class structures may look correct even when they are incomplete or wrong. Without review and editing options, users could accept unreliable output too easily.
- **Validation before progression:** Connector generation cannot rely only on plausible-looking output. Connection settings, generated scripts, search behavior, returned data, and capability operations need to be tested before the user continues to the next steps.
- **The generator needs execution feedback before it can continue:** The generator cannot reliably decide the next step based solely on a filled form, generated script, or selected endpoint. Important configuration parts have to be executed against the real resource so

that the system can confirm that the current path works and use that result as input for later steps. This is especially important for capabilities such as create, update, and delete, where one validated operation can support another. For example, the generator can use a working create capability to prepare a test object for the delete operation validation.

- **Complex connector state is hard to track:** A generated connector may contain documentation sources, connection information, object classes, capabilities, relationships, and uncategorized scripts. Without summary, users can get lost during long-running work.
- **Completed branches need to stay accessible without clutter:** Object class configuration can contain several sub-steps, such as discovery, schema validation, endpoint selection, and search validation. If every completed sub-step list remained expanded, the navigation would become difficult to scan after finishing configuration of several object classes or relationships.

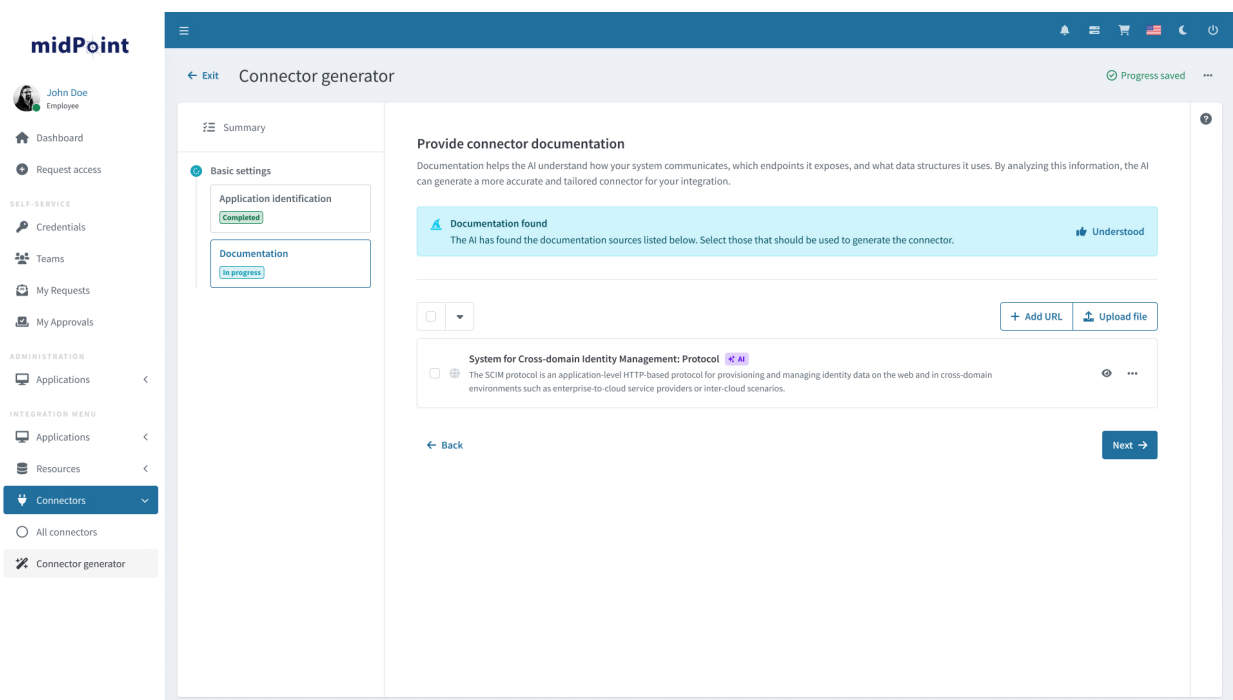
Design Challenges

- **Controlled AI assistance:** AI has to help with documentation discovery, recommendations, pre-filled values, script generation, validation, and repair without turning the workflow into an uncontrolled prompt interface.
- **Adaptive progress model:** The UI has to show progress without promising future steps that might change. The user needs to understand the current step and completed history, while the system remains free to branch, ask for more information, or enter a fix path.
- **Balance between forms, recommendations, and technical scripts:** Some users need simple guided inputs while others need access to generated scripts and technical details. The design therefore combines form-based decisions, selectable recommendations, tables, and script views.
- **Reviewable generated output:** Every important AI result has to remain open to inspection, editing, and reversion. This is necessary to protect the workflow against hallucinations, incomplete scripts, or weak documentation.
- **Readable object class configuration:** Generated schema and attributes have to be understandable for both technical and business users. A script view is necessary for precision, while a table view is needed for easier scanning and editing the attributes.
- **Flow continuity after branches:** After completing a branch, such as object class creation, the user needs a clear continuation point instead of being forced into a single linear path.
- **Validation as part of the generation logic:** Validation has to work as an input for the generator, not only as reassurance for the user. Successful connection tests, search execution, and capability checks create verified states on which the later steps can build.
- **Progressive disclosure of complexity:** The interface has to keep the main task focused while providing access to help, warnings, error details, and advanced repair actions when needed.
- **Recovery without losing context:** When generated output looks inaccurate or validation

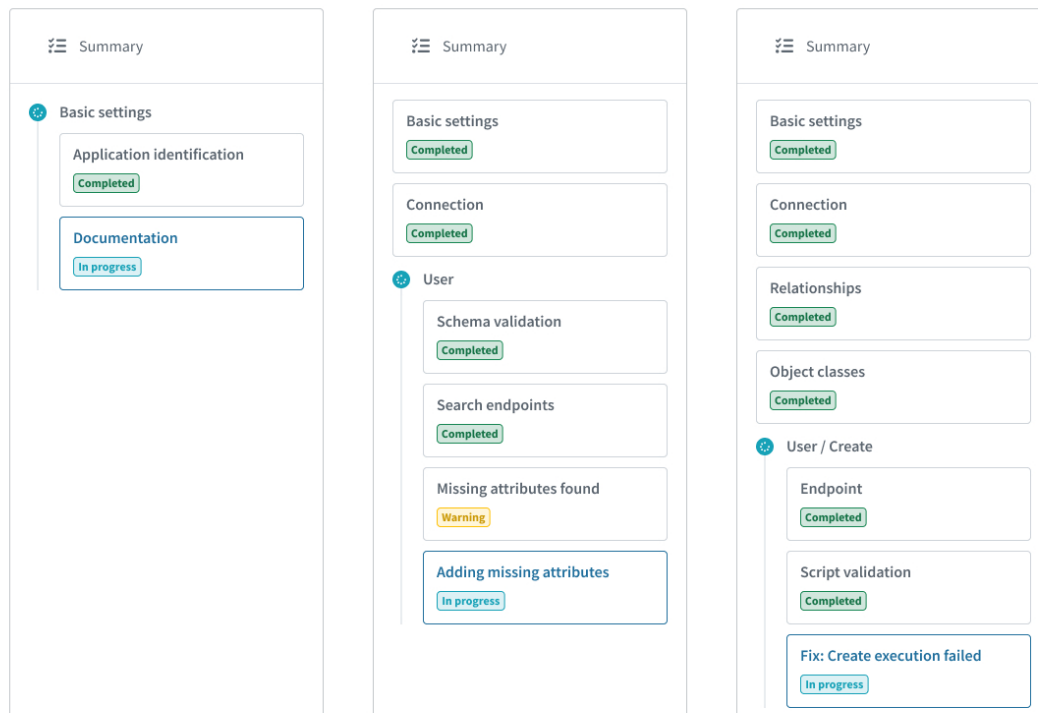
fails, the user needs a way to add better documentation and regenerate the affected script while staying in the same workflow area.

UX Decisions

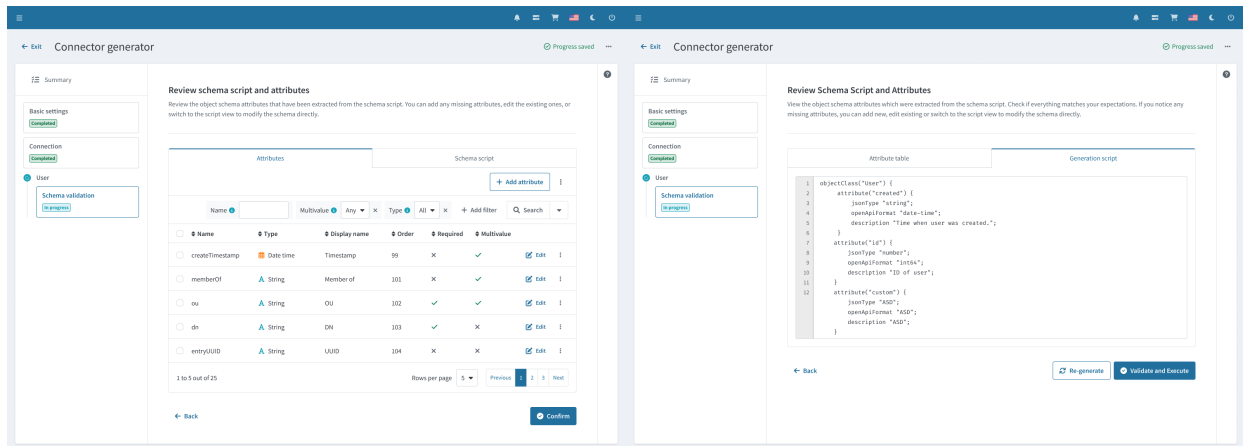
- **Guided AI workflow instead of a general prompt:** The prompt-like interaction was replaced by structured steps. AI assistance is applied inside concrete tasks where it can pre-fill, recommend, generate, validate, or correct specific connector parts.
- **Application identification as the starting point:** The generator starts with a focused form where the user identifies the target application, integration type, and relevant context. This gives the system enough information to search for documentation and prepare the initial connector path.
- **AI-assisted documentation discovery:** After the application identification form is completed, AI helps find relevant documentation. The user can review selected sources, remove unsuitable ones, or add additional links or files before the generation continues.



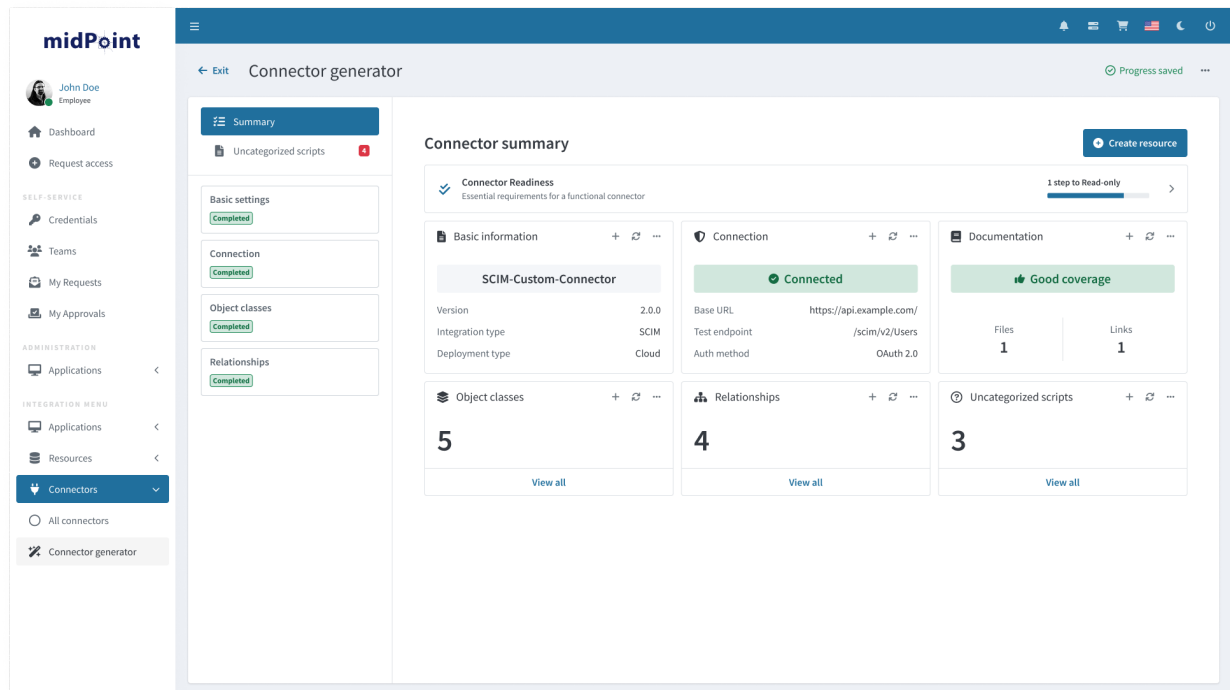
- **Basic connector information separated from target discovery:** Connector metadata, such as group id, artifact id, display name, description, and version, are configured after documentation review. This separates target application discovery from connector package definition.
- **Current step and history instead of a fixed future path:** The progress area shows the active step and completed history. Future steps are not promised upfront, because AI may need more information, validation may fail, or an error may create a repair branch.



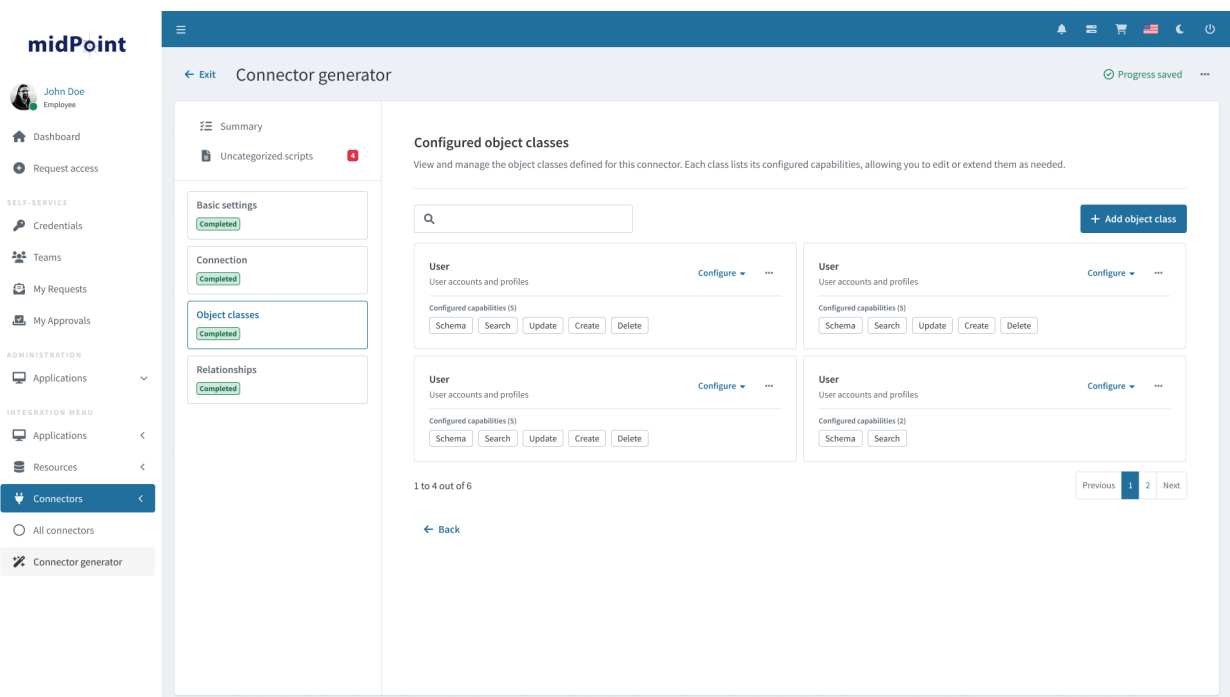
- **Connection configuration built from documentation and user input:** The generator identifies possible base URLs, supported authentication methods, and test endpoints from the selected documentation where possible. The user can use suggestions or enter values manually when documentation is missing, ambiguous, or incorrect.
- **Authentication split into supported methods and testing method:** The interface first lets the user decide which authentication methods the connector should support. It then asks which method should be used for the connection test and shows the corresponding credential fields.
- **Connection test as a required checkpoint:** The connector must reach the target system and receive a valid response before the generator can safely continue. This confirms that the basic connection is usable and gives the generator a verified base for object classes, search behavior, and other capabilities.
- **Script and table representation for attributes:** During object class definition, generated attributes can be reviewed in a table-oriented representation and connected to the underlying schema script. The table makes attributes easier to scan and edit, while the script representation preserves technical control. Complex attributes remain a subject of prudent UX design in the future.



- **Recommended object classes with fallback exploration:** When adding an object class, the generator recommends likely object classes based on documentation and discovery. If the expected class is not shown, the user can open additional candidates through the More classes option.
- **Review and validation before continuation:** Generated scripts and connector parts are not accepted silently. The user can validate scripts, execute operations such as search, inspect returned results, and either continue or return to correction.
- **Add documentation and regenerate where validation exposes gaps:** Validation screens provide a way to add documentation when the generated result is inaccurate, incomplete, or suspicious. The affected script can then be regenerated from improved input instead of forcing the user to restart the generator.
- **Summary accessible by the button above the progress sidebar and on generation process completion:** The connector summary shows readiness, basic information, connection status, documentation coverage, object classes, and relationships. It also provides follow-up actions, such as creating a resource, uploading the connector, adding an object class, or adding a relationship.



- **Collapsed process stages after completion:** After a connector generation stage is completed, the expanded branch is collapsed to show only its parent stage section. The user can reopen a configured stage to view and adjust the configuration, inspect object classes, see configured capabilities, or add more object classes later.



- **Relationships managed as a separate connector area:** Relationships are shown separately from object classes because they describe how configured objects relate to each other. The relationship view uses compact cards to show subject, relation attribute, and object so that links can be reviewed without reading raw scripts first.

- **Uncategorized scripts surfaced in navigation:** Scripts that cannot yet be clearly assigned to a specific connector area are exposed through the Uncategorized scripts entry with a count indicator. This prevents generated or imported technical output from being hidden.
- **Fallback for repeated AI repair loops:** If the workflow appears to be stuck in repeated failed repair attempts, the UI can ask whether the process is going in circles and offer recovery options, such as adding documentation, reporting the issue, or returning later after review.

Results and Delivered Outputs

The resulting interface combines a left-side progress area, a task-specific central workspace, and a connector summary used as the main checkpoint. Completed sections stay visible, active work is shown in context, and finished object class branches collapse back into a manageable Object classes area. A minimally usable connector can be produced by providing basic information, documentation review, connection configuration, connection testing, and at least one object class with validated search behavior. Additional object classes, relationships, and capabilities can then be configured from the summary as follow-up actions instead of being forced into the initial path.

The Web UI connector generator introduces an AI-assisted workflow in which a generated connector is assembled progressively rather than produced as a single opaque artifact. Users can start generation from clear product entry points, review AI-discovered documentation, adjust recommended options, edit generated content, validate scripts, inspect returned results, and continue only after the relevant outputs have been checked.

The design supports user control, reviewability, validation, and recovery throughout the connector generation process. It reduces manual connector creation effort while keeping AI output visible, editable, and testable before it becomes part of the final generated connector.

4.6.3 MidPoint Studio Plugin

MidPoint Studio plays the role of an expert-oriented companion to the web-based workflows. While the web user interface targets guided onboarding and review-based configuration, midPoint Studio addresses more advanced use cases in which users need structured editing, deeper control over configuration artifacts, and stronger support for validation and refinement.

In its broader role, midPoint Studio provides a stable environment for working with midPoint configurations in a structured way. This includes project-oriented editing, work with midPoint objects, validation support, browsing and uploading objects, diff-oriented workflows, and other developer-focused capabilities that make it suitable for expert configuration work. Because of this, Studio is a natural place for extending the user-facing solution with AI-assisted workflows aimed at more advanced users.

This direction is visible in a dedicated smart-suggestion implementation. The plugin contains specific actions for object type suggestions, correlation rule suggestions, mapping suggestions,

and association suggestions, together with shared infrastructure for launching these workflows through dialogs, wizards, and a dedicated smart-suggestion tool window. This shows that midPoint Studio is not treated merely as a passive XML editor, but as an active host environment for assisted configuration workflows.

An important aspect of the Studio-side design is that the suggestion workflows are review-oriented. The suggestions are presented in a structured form, such as tables and dialog-driven flows, rather than being injected automatically into the configuration. This mirrors the same design principle as in the web UI: AI assistance accelerates configuration work, but the user still reviews the result and remains in control of the final configuration state.

The implemented Studio actions are integrated into the midPoint action menu under the dedicated **Suggestion** group. There are also keyboard shortcuts mapped to resource object type, mapping, correlation rule, and association suggestions. When a suggestion is triggered from a local resource file, Studio offers to upload the resource with full processing so that the suggestion is based on the current resource definition. The user then selects the resource and either an object class or an existing object type, depending on the suggestion mode. For mapping suggestions, the wizard also exposes the inbound/outbound direction choice.

The generated suggestions are displayed as filterable tables with action controls. The action controls support inspecting the raw XML of a suggestion and applying it to the local resource configuration through a diff preview. Accepting the preview writes the suggested change back to the local resource file. This gives Studio users a review-and-apply workflow for suggestions rather than an automatic remote configuration update.

From the perspective of the delivered solution direction, it is reasonable to describe midPoint Studio as providing AI-assisted expert workflows for object type, correlation, mapping, and association suggestions. At the same time, these capabilities should still be understood in the broader context of Studio as a structured expert tool, where AI assistance complements existing validation and editing functionality rather than replacing it.

The **midpilot** branch also contains a registered **Connector Generator** action in the Studio toolbar and action menu. The active wizard path contains steps for initial entry, application identification, documentation discovery, and connector identification. It creates a necessary connector development object in midPoint, starts the documentation discovery operation, polls its status, and displays discovered documentation sources. The Studio client layer also contains support for connector development operations, such as creating or updating the connector development object, submitting create-connector and discovery operations, and reading their status and results.

4.6.4 Association Suggestions Interface

The following material about association suggestions is included because associations are a natural extension of the same reviewable suggestion pattern used for mappings and correlation. It should be interpreted primarily as UX and interaction design work for adjacent midPoint

configuration, not as a separately evaluated end-to-end pipeline in this report.

The association suggestions interface covers the configuration of relationships between subjects and objects in midPoint and the introduction of AI-generated suggestions for subject-object connections.

Problems Identified in the Existing Interface

- **Deeply nested wizard flows:** Adding a new association type required navigating two nested wizards to define the subject, object, and their relationship, making the process difficult to follow and inconsistent with the rest of the configuration interface.
- **No AI assistance for association setup:** Association configuration had no suggestion layer, leaving users to define subject-object relationships manually without guidance from the system.

Design Challenges

- **Consistency with existing suggestion patterns:** The association suggestion flow has to follow the same interaction model as mapping suggestions without introducing new mental models or interaction patterns for users already familiar with the mapping interface.
- **Option to review before apply:** As with mappings, accepted suggestions need to transition cleanly into the standard configuration state, making a suggested association indistinguishable from a manually created one once applied.

Key Design Decisions

- **Simplified wizard structure:** The nested wizard flows are replaced by a more navigable step-based structure consistent with the mapping and synchronization interfaces, reducing navigation depth without changing the underlying configuration model.
- **Suggestions surfaced at the top of the table:** AI suggestions appear above existing associations, visually separated by a purple left border and the Suggestion badge. Each row shows the proposed subject and object object classes with a directional connector, including Accept and Discard actions inline.
- **Table and tile view:** Users can switch between the table and tile views where suggestions are presented as collapsible cards with the same reviewable detail layer. Bulk Accept all and Discard all actions are available in the toolbar.
- **Clean transition on accept:** Once accepted, a suggestion transitions into a standard association row identical in appearance to any manually created association, with no visual residue of its AI origin.

4.6.5 Data Mapping Interface

The project includes an in-depth UX analysis and redesign of the data mapping interface in

midPoint—from initial analysis of the existing interface through iterative screen proposals to final designs prepared for developer handoff. The redesign focuses on three interconnected areas: data mappings, their hierarchical structure, and mapping simulations.

Problems Identified in the Existing Interface

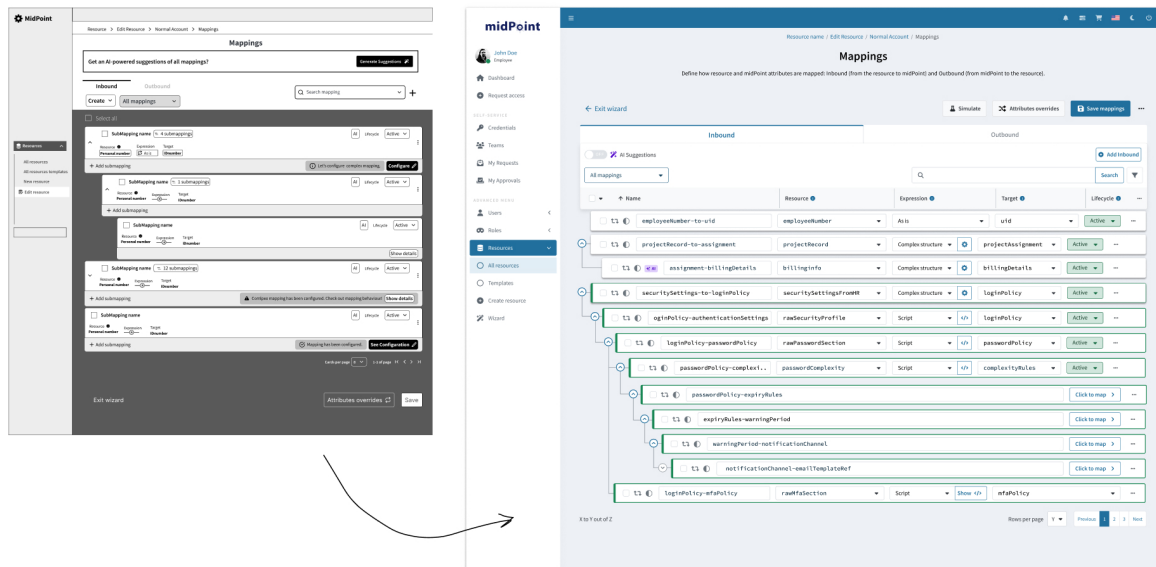
- **Hidden data logic:** Complex mappings and Groovy scripts were buried deep within wizard form fields, making the underlying data logic invisible to users and excluding them from understanding or managing the internal structure of the system.
- **Fragmented visual hierarchy:** The product had grown without a unified design system, resulting in inconsistent layouts and a confusing button hierarchy. Primary styling was applied to actions in every table row, creating visual competition and increasing cognitive load.
- **Poor handling of error and empty states:** The interface lacked designs for empty states and edge cases, leaving users with blank screens and no guidance. There were also no clear indicators distinguishing active buttons from disabled ones based on configuration state.

Design Challenges

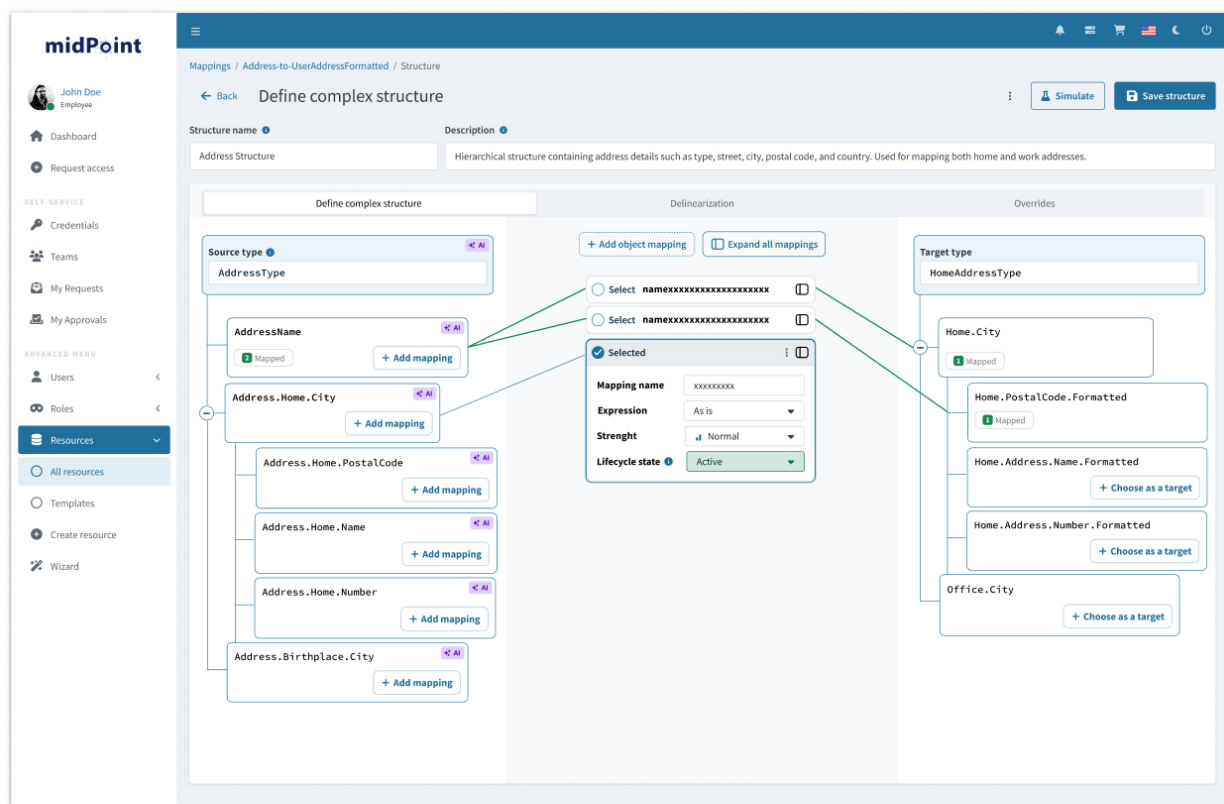
- **Native AI assistance:** The AI suggestion layer has to generate mapping proposals automatically without making the interface feel overloaded or autonomous.
- **New structural model:** The design has to support complex mapping hierarchies (mapping → sub-mapping → sub-sub-mapping) without losing context.
- **Design system:** No component library existed for mapping-specific patterns—one had to be built from scratch.
- **Accessibility for non-technical users:** An interface originally built for IT administrators comfortable with XML has to be made navigable for less technical audiences.

Design Decisions

- **Rows, not tiles:** The mid-fidelity wireframes used a tile-based layout that did not scale for deep hierarchies. A row format with indented sub-rows, colored left borders, and vertical guide lines enables display of up to seven levels of nesting without losing context or forcing users to navigate away from the current screen to access deeper levels of the hierarchy.

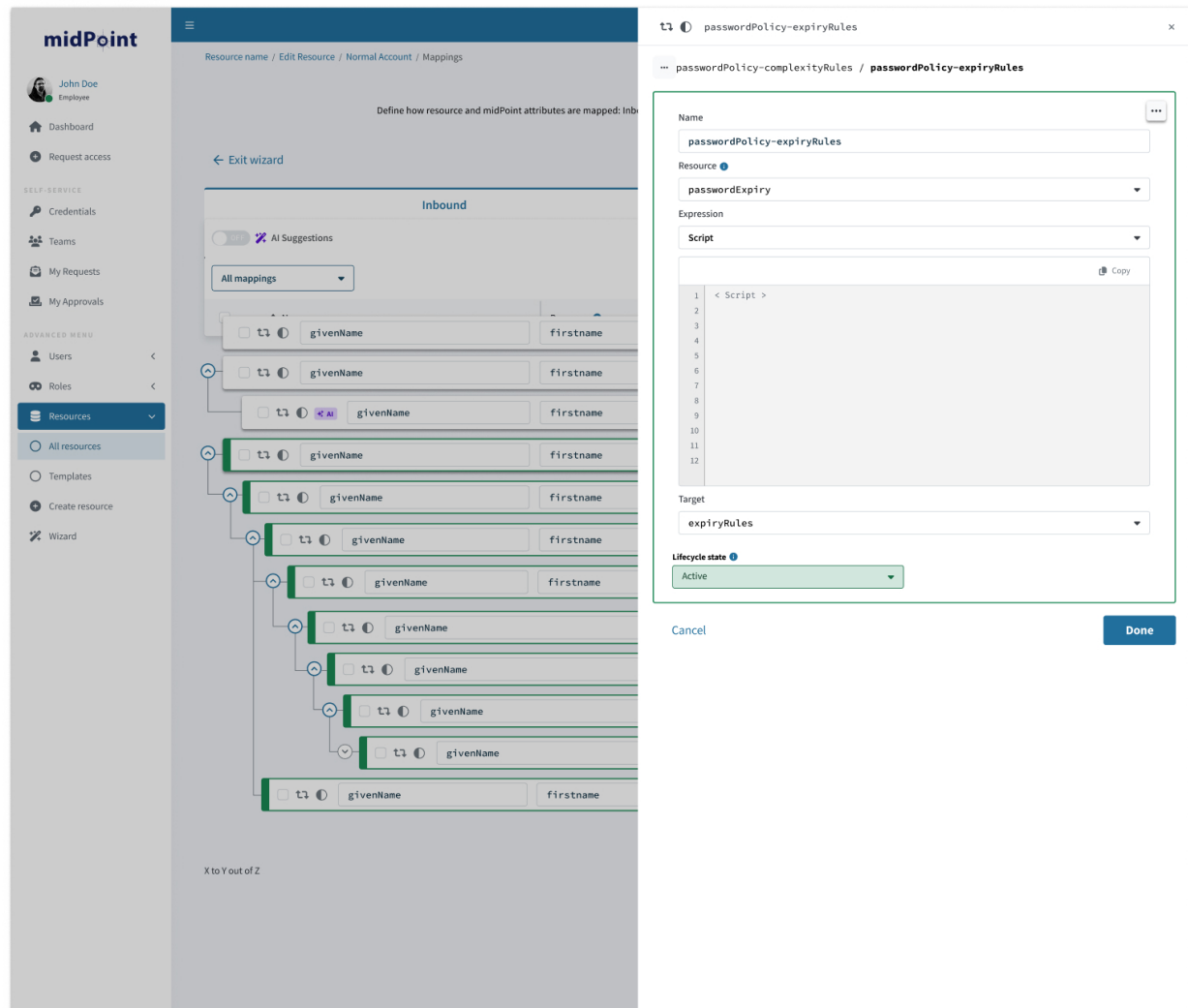


- **Fixed three-column canvas for complex structures:** For mappings with the expression set to **Complex Structure**, a dedicated canvas uses a fixed three-column layout — source attributes on the left, mapping nodes and expressions in the centre, target attributes on the right. The connecting lines update live.

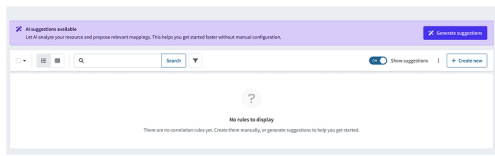
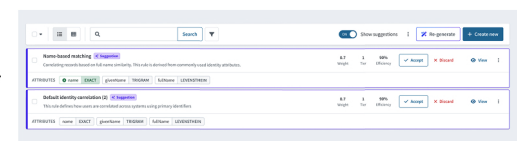
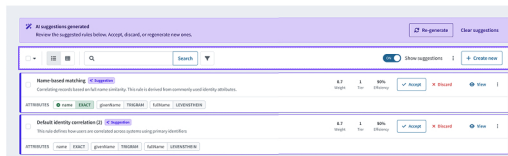
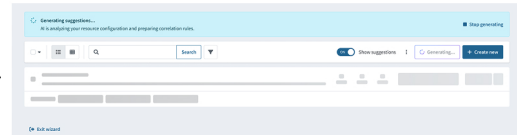
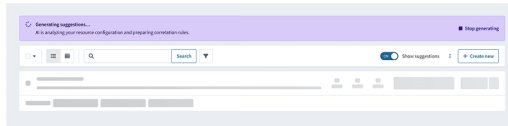


- **Side drawer with full ancestor breadcrumbs:** Inline display is limited to four levels.

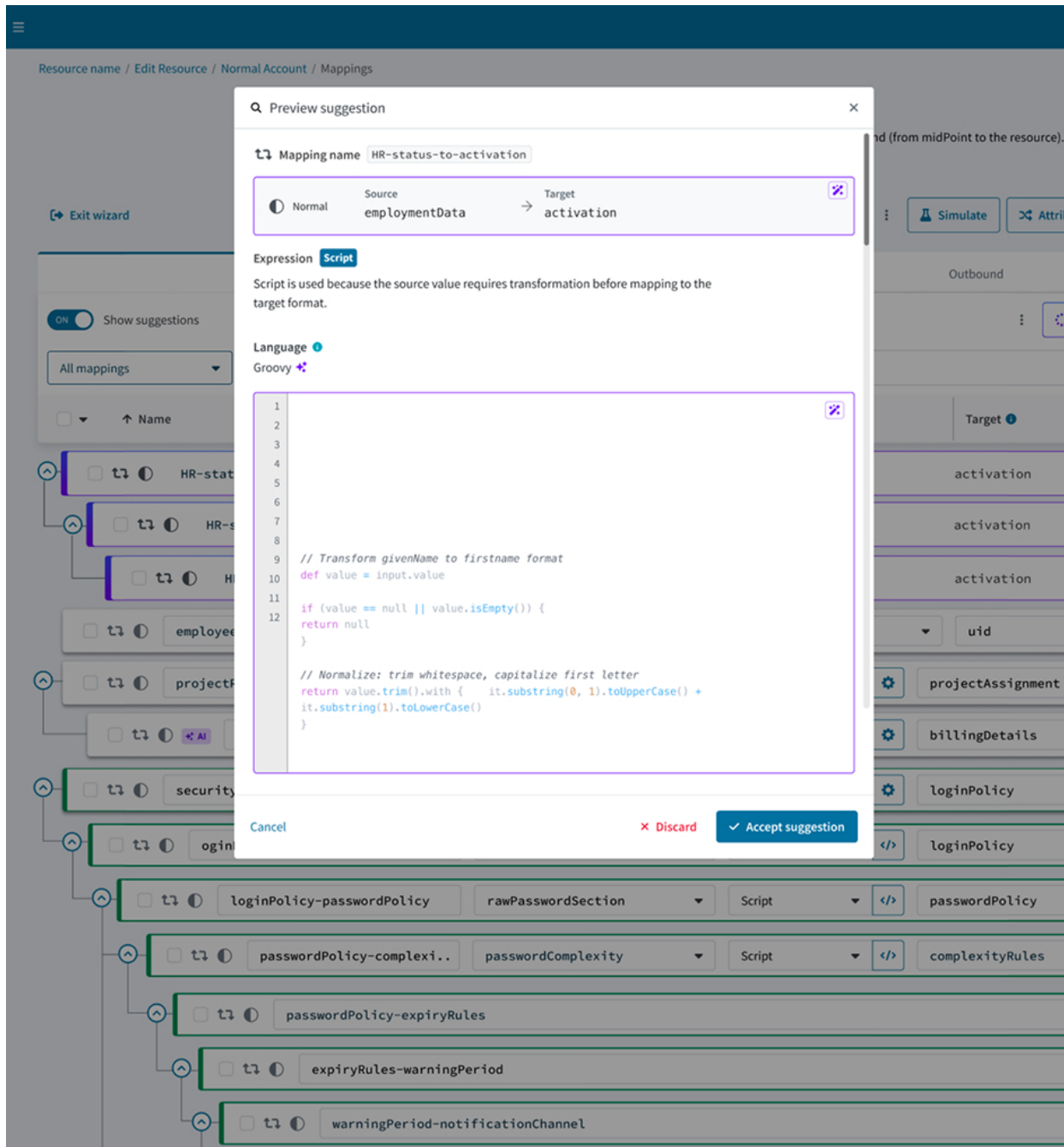
Deeper structures are accessible via the side drawer showing the complete ancestor breadcrumb path, keeping the main view readable.



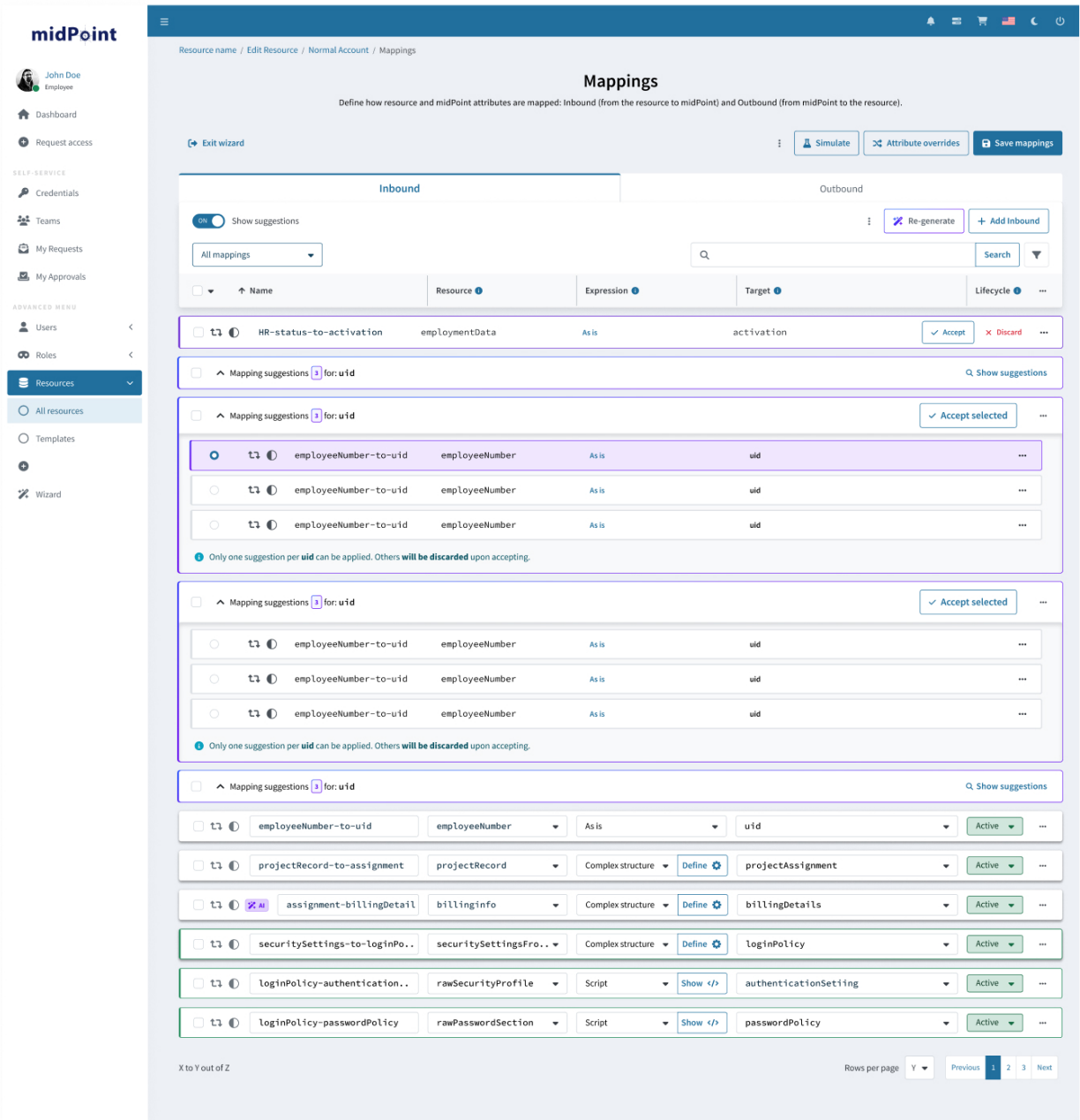
- **AI suggestions button bar replaces the banner:** The AI banner during the milestone 1 phase occupied space even when AI was not in use. To improve the user experience in the delivered product, all AI entry points have been consolidated into the toolbar under the "Generate" button next to the "Create New" button. The suggestions toggle remains inline.

BEFORE**AFTER**

- **AI suggestions as a reviewable layer:** A preview mode displays the full Groovy script of a suggestion before it is applied. Suggestions appear as purple-bordered rows at the top of the table—each can be individually accepted or discarded.



- **Mapping groups by target attribute:** When AI generates multiple suggestions for a single target attribute, they are visually grouped. The user can compare options and select one using a radio (single-option) button pattern. Accepting a suggestion discards the remaining ones in the group automatically.



Results and Delivered Outputs

The mapping interface displays mappings as rows with indented sub-rows, supporting complex hierarchies without losing context. AI-generated suggestions appear as a reviewable layer above the existing mapping table. They are grouped by target attributes so that users can compare alternatives and select one via the radio button pattern before committing. Each suggestion can be previewed in full before it is applied, and accepting one discards the remaining alternatives in the group automatically. A dedicated mapping review modal gives users a structured view of the full configuration before changes are saved. Mapping simulations help users validate the behavior before deployment so that edge cases are visible and addressable before the configuration goes live.

The design keeps AI output visible, reviewable, and reversible at every step. Configuration logic

previously hidden inside code is transformed into visual structures accessible to non-technical users with full user control and the option to opt-out throughout the workflow.

4.6.6 Guiding UX Principles

One of the main UX findings reflected in the final design is the need to prefer guided workflows over free-form AI interaction. In the context of identity integration and configuration, users benefit more from AI assistance when it is embedded into a clearly defined step of the workflow than when it is presented as a general-purpose assistant without strong constraints. This principle is followed both in the web UI and in midPoint Studio, where suggestions are tied to concrete configuration tasks, such as object type definition, mapping setup, or correlation design.

A second important finding was the need for explicit review points. The final solution consistently treats AI output as a proposal rather than as an automatically applied change. This is particularly important because configuration decisions in midPoint often have structural, architectural, or security-related consequences. As a result, the UX emphasizes human review, explicit confirmation, and the ability to modify or reject generated suggestions before they became part of the resulting configuration.

A third finding concerns the role of validation feedback. The final design does not rely only on prompt quality or on the internal plausibility of the generated output. Instead, validation and correction are built into the workflow itself. This makes the user-facing experience more robust, because suggestions are not only generated but also constrained by backend checks and, when needed, iteratively corrected. This significantly reduces the risk of presenting users with outputs that appear convincing but are not executable or semantically valid—a property enforced by the backend validation rather than prompt quality alone.

Another important lesson is that different user groups require different interaction styles. The web UI is better suited for guided onboarding and review-driven configuration of resources. MidPoint Studio, by contrast, is better suited for advanced users who need to combine AI suggestions with detailed editing, validation, comparison, and surrounding configuration work. The final design therefore moved toward a pragmatic separation of roles: web-based AI workflows for guided configuration tasks and Studio-based AI workflows for expert refinement and more advanced assistance.

Finally, the overall project direction reflects a shift from earlier, more ambitious expectations toward a constrained and practical model of AI assistance—one that benefits users primarily through greater reliability and predictability of the generated suggestions. Instead of aiming at a single autonomous AI interface for the entire integration lifecycle, the final design favors smaller, well-bounded suggestion workflows. This change improves predictability, supports validation, reduces unnecessary user friction, and better matches the real product environment in which AI assistance has to coexist with existing midPoint configuration, review, and security mechanisms.

4.7 Complete Pipeline Evaluation and Test Scenarios

4.7.1 Purpose

The component-level evaluations in the previous sections show how individual AI-assisted capabilities behave in isolation. The complete pipeline evaluation has a different purpose: it should assess whether the delivered smart integration workflow works as a coherent onboarding process when the individual capabilities are combined.

4.7.2 Evaluation Scope and Methodology

The complete pipeline is evaluated by identity engineers using end-to-end test scenarios.

For configuration assistants, the goal is to evaluate the whole flow instead of isolated LLM metrics in order to assess the ability of the whole solution to simplify the configuration of midPoint for administrators and engineers in realistic test environments.

The test scenarios for configuration assistants are executed in the following environments:

1. Environment based on MCM 2026 workshop
2. Environment based on Evolveum infrastructure
3. Customer-like PoC Environment
4. Customer-like Environment

For the connector generator, the assessment covers the full workflow from the generator input to the executable connector behavior in a test environment. Validation of the connector code generator, in case of an end-to-end pipeline, was done using the web-UI-based assistant. The selected scope evaluates applications exposing a plain REST API and a SCIM-based API.

The systems used in the test scope are the following applications exposing a REST API:

- OpenProject
- Codeberg-Forgejo
- Nextcloud
- Zendesk

The following application used in the test exposes an API conforming to the SCIM framework:

- Atlassian

4.7.3 Connector Code Generator Evaluation

Connector Generator Test Environments

A separate test environment based on official Docker images for each application, except "Zendesk" and "Atlassian", was set up. These were used as part of individual Docker Compose configuration stacks. In the case of Atlassian and Zendesk, sandbox environments were set up based on the guidelines present in their documentations. The midPoint application itself and the microservices representing the midPilot connector code generator are also set up as part of the Docker Compose stack. In the case of both the midPoint application and the midPilot connector code generator, images for the purpose of testing the current project state were set up in a private registry.

The tested applications were selected from the application set for which the ground truth and information-gathering materials had already been prepared during the project. These materials describe the relevant API documentation, object classes, operations, and expected connector behavior, and were used as structured input and reference material for the connector code generator evaluation.

The following table provides the basic parameters of the applications which were used in the testing stack.

Application	Version	API Type	Object classes
OpenProject	17	REST	User, Group
Codeberg-Forgejo	11.0.14	REST	User
Nextcloud	34.0.0	REST	User (ProvisioningApi UserDetails)
Zendesk	June-2026 (cloud-based SaaS)	REST	User, Group
Atlassian	June-2026 (cloud-based SaaS)	SCIM	User, Group

Connector Generator Test Scenarios Design

The evaluation follows the real engineering workflow of a connector engineer working with the generator in the midPoint user interface. The process starts with preparing information about the system that is to be integrated and providing structured input for the AI generator. Based on this input, the engineer selects the correct application documentation from the proposed candidates or provides alternative documentation manually when needed. The generator then produces connector artifacts and related configuration which the engineer reviews to ensure accuracy and completeness. If flaws are identified, the artifacts are modified or regenerated, tested in the UI, and re-evaluated until the result satisfies the expected business criteria. This approach reflects the intended human-in-the-loop workflow, where generation, validation, troubleshooting, and refinement are all inherent parts of the authoring process rather than

separate activities.

The test scenarios are based on several assumptions about the systems included in the evaluation scope. Each tested system contains at least one object class representing a user account and provides publicly accessible documentation for its API and object schema. The selected systems are assumed to support at least the basic CRUD capabilities for the object classes relevant to the evaluation. In some cases, additional object classes representing entitlements, such as groups, roles, or organizational units, are also available. Where supported by the target system, relationships between these entitlement objects and user objects may also be included in the evaluation.

Connector code is generated exclusively through the UI of the connector code generator integrated into midPoint. This UI provides the working environment in which the engineer performs generation, validation, testing, troubleshooting, and repeated refinement of the produced artifacts. As a result, the evaluation covers not only the generated connector artifacts themselves but also the practical usability of the full wizard-based workflow in which they are created and verified.

Pass/Fail Criteria

The criteria for this test are based on a composite weighted scoring model that measures a set of independent metrics. The following is a summary of the metrics together with the recommended weights:

Metric	Weight	Note
Information Discovery	10 %	How well the generator finds candidate docs and endpoint links
Endpoint accuracy	20 %	Correctness of candidate URLs, builder definitions, or attribute mappings for each operation
Generation quality per operation	25 %	Functional correctness of the generated declarative SCIMREST script for each supported operation, considering whether the operation works and how much manual editing is needed.
Manual repair effort	20 %	Time required per operation to reach deployable state.
Repeatability	15 %	Variance across repeated runs for same input
Operational readiness	10 %	Startup, auth, logging, error messages, and test run success in target runtime.

Scoring rubric (0–5 scale for each sub-metric):

- 5 - excellent, meets expectation with no edits and passes tests.
- 4 - good, minor edits (<5 minutes) or small config tweaks.

- 3 - acceptable, needs non-trivial manual edits (5–20 minutes) or minor code fixes.
- 2 - significant edits (>20 minutes) or missing major pieces.
- 1 - unusable, generator output not buildable or incorrect; requires complete rewrite.
- 0 - no output, crash.

Concrete sub-metrics and how to collect them

Information discovery:

Function of percentage of sources the author would accept automatically and percentage of sources provided by the author (0–5).

Endpoint and mapping quality:

- The proportion of required attributes correctly mapped.
- Correctness of HTTP method and resource path.
- Correctness of builder definitions present in script.

Scoring is done via the following formula:

$$S_i = 5 \cdot \max\left(0, \min\left(1, \frac{C_i - E_i - M_i}{T_i}\right)\right)$$

Where:

S_i = score for item i , on a 0–5 scale.

C_i = number of correct elements.

E_i = number of incorrect extra elements.

M_i = number of missing required elements.

T_i = total number of expected elements.

The overall score is then computed as:

$$S = \frac{1}{n} \sum_{i=1}^n S_i$$

Per-operation correctness:

This metric estimates how close the generated operation artifact is to a working implementation. For each generated operation, the evaluator checks whether the artifact can be executed successfully and records the manual changes needed to make it work, such as endpoint correction, attribute mapping correction, parameter handling changes, or script edits. Fewer and smaller edits indicate higher per-operation correctness.

Manual repair effort:

Time required to achieve operational usability (measured in minutes) per operation, normalized to a 0–5 scale as follows:

- no repair needed = 5
- less than 5 minutes = 4
- 5–20 minutes = 3
- 20–60 minutes = 2
- greater than 60 minutes = 1
- not applicable = 0

Repeatability and determinism:

- Execution of the same script generation for five times, output is a function of deviation in correctness and output proportion.
- We measure repeatability using the relative deviation from the sample mean. We compute this deviation for the total number of generated lines and for the number of lines requiring modification.

$$\text{Deviation} = \frac{\sum_{i=1}^n |x_i - \bar{x}|}{n \cdot \bar{x}} \times 100$$

- The overall repeatability metric is then defined as a weighted average of these two normalized deviations.

$$R = 100 - (w_1 D_{lines} + w_2 D_{repair})$$

D_{lines} = deviation of total generated lines

D_{repair} = deviation of lines that must be modified

Both values are weighted with equal weight

$$w_1 = w_2 = 0.5$$

The final number is bounded between the values 0 and 100, the reason is that a deviation can be greater than a calculated mean value.

Operational readiness

- Startup success, auth test success, meaningful logs and error messages, as yes/no answers mapped into 0–5 aggregate.

The evaluation metrics are reported per generated connector artifact using the concrete artifact names defined in the midPoint "connector development object". This allows each generated operation script or search operation to be scored individually before aggregation into the final

overall result.

Schema-related artifacts and operation-related artifacts are evaluated with the same metric categories, but the interpretation of “generation quality” differs slightly. For schema artifacts it means correctness of schema/model structure, while for operation artifacts it means correctness of executable behavior and endpoint mapping.

The evaluated artifacts are:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

These artifacts are grouped into two categories. The schema-related artifacts, nativeSchemaScript and connidSchemaScript, are evaluated primarily with respect to correctness of generated schema representation, object class handling, and compatibility with the connector and resource model. The operation-related artifacts, such as createScript, updateScript, and the search and delete operations, are evaluated primarily with respect to endpoint selection, mapping quality, executable correctness, and validation outcomes during testing.

In certain instances, applications do not support the full set of artifacts defined for a given object class. In such cases, unsupported artifacts are excluded from evaluation. This exclusion criterion is applied when neither the technical documentation nor the established ground truth explicitly specifies that the operation corresponding to the artifact is supported.

Aggregation and reporting

Core quality metrics, other than "Information discovery" and "Operation readiness" will be calculated per operation artifact for each object class.

The metrics, reported per artifact, are using the dimensions present in the following example:

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	85	80	75	80

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
connidSchemaScript	90	95	100	100
createScript	95	80	85	90
updateScript	80	75	75	80
searchFilterOperation	80	75	85	90
searchAllOperation	90	80	75	80
searchIdOperation	80	75	85	90
deleteScript	95	80	85	90
Average	86.88	80.00	83.13	87.50

The artifact-level scores are later aggregated across artifacts and aggregated across object classes to produce the final connector-level evaluation.

This approach makes it possible to identify specific weak points in the generator, such as accurate schema generation but unstable search operations or correct endpoint selection combined with high manual repair effort in the update logic. Because each operation in midPoint is executed through a defined connector capability and schema model, this granularity improves both defect analysis and result interpretation.

Together with the "Information discovery" and "Operation readiness", which are assumed to be global metrics for the overall connector generation, a final score is produced.

The final score is a computed weighted score defined as:

$$\text{weighted score} = \sum_{i=1}^n (w_i \cdot s_i)$$

where w_i is the weight and s_i is the normalized score for each evaluated criterion i .

The normalized score is the grade 0-5 normalized to a percentage (0 - 100) before weighting.

Example scoring table:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	4	80.00	8.00
Endpoint and mapping quality	20 %	n/a	86.88	17.38
Generation quality per operation	25 %	n/a	80.00	20.00

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Manual repair effort	20 %	n/a	83.13	16.63
Repeatability	15 %	n/a	87.50	13.13
Operational readiness	10 %	3	60.00	6.00
Total	100 %	—	—	81.14

Pass thresholds:

- ≥ 85 : Demonstrates a high level of quality with minimal author effort.
- 70–85: Reflects a sound quality baseline, targeted refinement needed.
- 50–70: Indicates an emerging quality level with clear potential for structured improvement.
- < 50 : Indicates limited operational readiness, substantial enhancement required.

The scoring model does not assume that high-quality connector generation is a one-shot process. It rewards correctness, efficiency, and stability across the full authoring workflow, including user-guided regeneration and refinement.

Test Data Strategy

The test data strategy is based on a small, curated, and representative set of objects for each tested object class. Its purpose is not to simulate production-scale application data, but to validate that the generated connector operations execute correctly in a realistic target environment using typical object shapes and selected relationship patterns.

For each object class, the dataset contains only a limited number of manually prepared objects, typically covering standard valid examples and a few relationship-oriented cases. Where the target system supports relations between objects, the dataset includes selected examples such as users assigned to groups, users assigned to organizational units, and groups containing other groups, so generated operation artifacts can be validated against non-trivial but still manageable scenarios.

The generator itself does not consume or analyze the actual application data. Instead, it generates connector operation artifacts from available documentation and user-guided refinement in the wizard, while the prepared test objects are used only afterward to validate the correct execution of the generated scripts for individual operations.

Because the main goal of the evaluation is connector correctness and authoring efficiency, the dataset is intentionally small but semantically representative.

The strategy focuses on examination of operation behavior, mappings, endpoint selection, and relationship handling with enough variation to expose errors, while avoiding unnecessary complexity of large test data volumes.

Each object class should include objects with and without relationship to evaluate the coverage of both types of objects, ones with and ones without relations.

Test Coverage

This chapter describes the extent and organization of testing performed for the connector code generator. Coverage is defined in terms of object classes, operations, relationship scenarios, AI-specific properties, and workflow/operational aspects.

The evaluation focuses on the standard object classes for which the generator is expected to produce connectors, and on any additional custom object classes relevant to the tested application. The primary object classes are:

- User (or any other abstraction representing a user object class, e.g., "Account", "Person", "Member" ...)
- Group (or any other abstraction representing an entitlement or a container grouping other object classes, e.g., "Team", "Organization", "Project" ...)
- Custom object classes relevant to the application (e.g., object classes representing a relation like "Membership", "Relation", "Grant" ...)

For object classes with relationships the test coverage should include a representative scenario that exercises those relationships.

For each object class, the test coverage includes the following operations, which correspond to the generated operation artifacts:

- Authentication
- Test
- Schema
- Create
- Read
- List/Search
- Update
- Delete

For each operation, the test validates whether the generated artifact uses the correct HTTP method and endpoint path, applies the expected attribute mappings and parameter handling, and correctly supports filters, pagination, and sorting where these capabilities are relevant. The evaluation also verifies that update behavior is implemented correctly and that relationships between objects are handled as expected when the target system supports them.

Because the tested system is an AI-based generator, the test coverage also includes properties specific to generative systems. The evaluation checks whether the generator suggests plausible

endpoint URLs and HTTP methods for each operation. It also checks whether attribute and parameter mappings are correct or need manual correction. Another focus is hallucination detection, meaning whether the generator invents endpoints, attributes, or configuration elements that are not present in the documented API. Finally, the evaluation measures repeatability, that is, whether repeated runs with the same input produce similar outputs and scores. These AI-specific dimensions are evaluated per operation and per object class, and they are included in the scoring model.

Test coverage also includes aspects of the authoring workflow:

- The complete end-to-end flow in the MidPoint wizard, starting with specifying the application name, through documentation selection, endpoint suggestions, operation generation, validation, regeneration, and testing.
- Startup success, authentication test success, meaningful logs and error messages.
- Whether incorrect or partial operation artifacts can be repaired by the author with reasonable effort and regenerated successfully.

Evaluation Criteria

The evaluation is designed to answer the following questions:

- Does the Connector Code Generator produce connectors that implement the required SCIM/REST capabilities correctly?
- Are the generated connectors deployable and usable without substantial manual rework?
- Are the outputs consistent enough across runs to support practical engineering use?
- Are the generated connectors operationally ready?
- How consistent are outputs across repeated runs for the same input, and does the generator minimize hallucinated endpoints or attributes?
- How well does the generator surface and use application documentation to drive correct endpoint and mapping proposals?
- What are the most common failure patterns, and what do they imply for operational readiness?
- Does the generator support efficient iterative refinement, allowing the connector author to correct, regenerate, and validate generated operation artifacts until a usable result is reached with acceptable effort?

The evaluation is performed per generated operation artifact and per object class, and the resulting sub-scores are aggregated into application-level metrics and the final overall score.

A satisfactory result is one in which the generator produces correct and testable operation artifacts with low manual repair effort, stable repeated outputs, and sufficient operational quality for pilot use.

The connector generation workflow is evaluated as an iterative, human-in-the-loop process rather than a one-shot generation task. A perfect result on the first attempt is not required and is considered unlikely for complex connector generation scenarios. The evaluation focuses on whether the generator helps the connector author converge to a correct and deployable result with reasonable time, effort, and number of refinement cycles.

Connector Generator Results

This section contains the summarized metrics for the tested systems representing the "quality view" as well as personal evaluation base on an empirical approach. Also, it is divided into subsections per tested application with the metrics, technical validity, operational usability and readiness classification.

The metrics and commentary should answer the questions put forward in the evaluation criteria.

OpenProject

OpenProject is an open-source project management software system that supports classical, agile, and hybrid project management methodologies throughout the entire project lifecycle. The application communicates with the IAM system via a REST api. The documentation and schema is publicly available. The test covers the generation of connector artifacts for the object classes "User" and "Group".

For the "User" object class, the following artifacts were generated:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	84.25	76.80	60.00	61.00
connidSchemaScript	84.00	98.34	96.00	53.65
createScript	94.00	74.65	92.00	23.85

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
updateScript	94.00	90.79	86.00	32.15
searchFilterOperation	98.00	91.41	80.00	59.83
searchAllOperation	100.00	98.58	96.00	99.30
searchIdOperation	95.00	88.30	64.00	81.85
deleteScript	100.00	100.00	100.00	95.90
Average	93.66	89.86	84.25	63.44

In case of the "Group" object class, this object class does not support native filtering, so the "searchFilterOperation" is omitted from the evaluation. The following artifacts were generated:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	24	17.80	60.00	92.90
connidSchemaScript	100	100.00	100.00	100.00
createScript	100	100.00	100.00	89.50
updateScript	100	100.00	97.29	100.00
searchAllOperation	90	88.57	92.90	31.72
searchIdOperation	90	81.69	80.00	60.84
deleteScript	100	100.00	100.00	89.30
Average	86.29	84.01	90.03	80.61

Overall metric:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	5	100.00	10.00
Endpoint and mapping quality	20 %	n/a	89.98	18.00
Generation quality per operation	25 %	n/a	86.94	21.74
Manual repair effort	20 %	n/a	87.14	17.43
Repeatability	15 %	n/a	72.03	10.80
Operational readiness	10 %	3	60.00	6.00
Total	100 %	—	—	83.97

The testing focused on both the read and write capabilities provided by the OpenProject application through its REST API. The application offers readily accessible documentation, including examples of operation execution and sample data structures based on the schemas of specific object classes. However, it was observed that portions of the documentation contain inaccuracies, such as incorrect attribute names, misassigned access flags, and, in some cases, flawed operation examples. These inconsistencies had to be accounted for during the evaluation process, as well as in the construction of the “ground truth” datasets used for this use case.

The general use of the connector generator in this context did not require a substantial amount of input from the operator. The generated schema scripts were largely accurate, with only occasional instances of irrelevant or incorrectly derived properties based on the documentation. Greater challenges arose in relation to attribute properties, particularly with respect to incorrect access flags and, at times, mutually exclusive flag combinations. Nevertheless, most of these issues were resolved through minimal modifications, typically involving one or two lines of code, with the primary effort concentrated on verifying and reconciling discrepancies in the documentation.

Regarding operation scripts, the most significant issues stemmed from AI-generated inconsistencies, including the introduction of script segments that were either invalid for the given operation or entirely non-existent. Additionally, there were recurring instances in which the connector generator substituted framework-specific properties with Groovy-native constructs, leading to semantically incorrect representations. Despite these limitations, the overall impact remained minor, the required manual intervention was limited, and the operator was able to complete tasks more efficiently compared to a fully manual approach.

A specific aspect worth highlighting is the effectiveness with which the connector generator identified endpoints and captured particular nuances in update scripts, a task that would typically require considerable time and effort from an engineer.

Codeberg-Forgejo

Forgejo is a self-hosted, lightweight Git repository management system that provides distributed version control, issue tracking, pull request management, and project coordination capabilities as a free and open-source platform. The application communicates with the IAM system via a REST API, for which both documentation and schema definitions are publicly available.

The testing focused on the artifacts generated for the “User” object type:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	16.0	39.71	82.76	40
connidSchemaScript	100.0	89.70	88.92	80
createScript	100.0	89.09	26.00	96
updateScript	0.0	33.98	72.14	44
searchFilterOperation	60.0	55.00	95.00	60
searchAllOperation	76.0	71.83	52.00	80
searchIdOperation	80.0	75.40	52.00	76
deleteScript	100.0	100.00	95.27	100
Average	66.5	69.34	70.51	72

Overall metric:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	4	80.00	8.00
Endpoint and mapping quality	20 %	n/a	67.75	13.30

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Generation quality per operation	25 %	n/a	69.34	17.33
Manual repair effort	20 %	n/a	70.51	14.10
Repeatability	15 %	n/a	72.00	10.80
Operational readiness	10 %	3	60.00	6.00
Total	100 %	—	—	69.53

The evaluation revealed several structural limitations related to mapping of ConnId schema attributes. In particular, the `login` attribute is meant to consist of `name` and `uid` attributes, but since mapping two attributes into one is not yet supported by the current connector framework, a workaround is required—using the `id` attribute as UID even if it is not described as such in the "ground truth".

In the context of native script generation, a high degree of structural consistency was observed, with the same attribute blocks being repeatedly generated across iterations. Variability was primarily confined to access flags which were frequently incorrect or inconsistently assigned. Despite this, the effort required to correct such issues remained minimal, typically involving straightforward adjustments.

Search operation scripts frequently paired the selected endpoint with an additional, unintended endpoint that was not chosen during configuration. While this secondary endpoint did not correspond to the intended use case, it was not entirely irrelevant and could be applicable in other search-related scenarios. A recurring issue was also identified in the improper use of Groovy-native constructs, instead of adhering to framework-defined property references, resulting in semantically invalid scripts.

Greater inconsistencies were observed in the create and update operations. The create scripts demonstrated low repeatability, largely due to occasional hallucinated outputs that significantly diverged from otherwise consistent generations. The update scripts frequently contained an excessive number of irrelevant or non-existent attributes, necessitating careful validation against the Forgejo REST API documentation. The primary effort in these cases shifted from code correction to documentation verification and reconciliation.

Further limitations were identified at the framework level. The absence of mechanisms to dynamically substitute placeholder path parameters with values not present in the schema restricted the ability to fully validate delete and update operations.

Overall, the connector generator demonstrated a clear benefit in reducing development effort. The generated scripts, despite their imperfections, provided a strong baseline that required only incremental refinement. Documentation links produced by the system were generally accurate, though occasional supplementation was necessary. The extracted documentation data proved

useful, particularly in identifying authorization requirements and endpoint structures. However, the generation of inter-object relationships remained unreliable and frequently incorrect, indicating an area for further improvement.

Nextcloud

Nextcloud is an open-source content collaboration platform that provides self-hosted file storage, synchronization, and sharing capabilities with desktop, mobile, and web interfaces. The platform enables real-time document collaboration, video communication, and granular access control while emphasizing data security and privacy through end-to-end encryption capabilities. Similarly to the previously evaluated applications, Nextcloud provides a REST API for managing selected object types. While the documentation is publicly available, it proved difficult to navigate and, in several cases, presented ambiguous or insufficiently clear information.

The testing focused on the artifacts generated for the object class "ProvisioningApiUserDetails" representing the "User" object type.

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	0.00	28.79	40.00	81.04
connidSchemaScript	60.00	97.50	92.00	39.00
createScript	54.00	47.53	76.00	72.15
updateScript	0.00	34.81	44.00	32.67
searchFilterOperation	50.00	36.18	80.00	83.30
searchAllOperation	64.00	71.44	80.00	38.64
searchIdOperation	68.00	75.56	76.50	80.00
deleteScript	46.00	66.39	80.00	74.52
Average	42.75	57.28	71.06	62.66

Overall metric:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	3	60.00	6.00
Endpoint and mapping quality	20 %	n/a	42.75	8.55
Generation quality per operation	25 %	n/a	57.28	14.32
Manual repair effort	20 %	n/a	71.06	14.21
Repeatability	15 %	n/a	62.66	9.40
Operational readiness	10 %	3	60.00	6.00
Total	100 %	—	—	58.48

The evaluation of this application was influenced by certain limitations related to documentation accessibility and its interpretation by the generator. All relevant documentation had to be supplied manually, as the automatically suggested sources were either incomplete or insufficiently detailed. Furthermore, the base API URL was not inferred from the documentation, and authentication mechanisms could not be consistently extracted, necessitating minor manual adjustments.

The filtering mechanism of the application differs from conventional approaches, relying on a generic search parameter that performs broader matching across multiple attributes. In combination with occasional inaccuracies in endpoint and header generation, this introduced additional complexity into the implementation of search functionality.

The generation of operation scripts showed some variability across iterations. Create operation scripts occasionally included redundant or non-applicable operation builders, and while required headers were sometimes correctly identified, their implementation was not always fully aligned with the expected syntax or semantics.

Update operations reflected similar patterns, including the use of imprecise or non-existent endpoints, the presence of unsupported PATCH operations, and the inclusion of attributes that were not always relevant or modifiable. In addition, the inability to execute POST and PUT operations due to the lack of support for the application/x-www-form-urlencoded content type within the framework limited the scope of full write-operation validation.

Overall, the generator remained usable in this context, although its effectiveness was influenced by a combination of documentation-related constraints, framework limitations, and minor inconsistencies in script generation. Functional results could be achieved with the use of workarounds, though the required level of manual intervention was higher compared to other evaluated systems.

Zendesk

Zendesk is a cloud-based customer service and support platform that provides ticketing, user management, and workflow automation capabilities. The application exposes a comprehensive REST API centered around core object types such as users, tickets, organizations, and groups. The API supports standard CRUD operations, pagination, and filtering.

For the "User" object class, the following artifacts were generated:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	58.00	59.96	68.00	89.05
connidSchemaScript	100.00	100.00	100.00	89.26
createScript	8.00	31.85	80.00	59.16
updateScript	0.00	28.96	40.00	92.58
searchFilterOperation	10.00	14.52	40.00	89.98
searchAllOperation	40.00	54.28	80.00	20.40
searchIdOperation	80.00	90.77	80.00	86.67
deleteScript	48.00	62.02	80.00	76.02
Average	43.00	55.30	71.00	75.39

For the "Group" object class, the same artifacts were generated as for the "User" object class, excluding the artifact "searchFilterOperation" as there is no endpoint provided for this operation by the target system:

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	0.00	0	40.00	94.29
connidSchemaScript	50.00	84	80.00	85.71
createScript	20.00	0	32.35	80.00
updateScript	0.00	0	60.00	79.30
searchAllOperation	0.00	0	60.00	96.19
searchIdOperation	0.00	0	60.00	81.89
deleteScript	60.00	0	80.00	43.86
Average	18.57	12	58.90	80.14

Overall metric:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	4	80.00	8.00
Endpoint and mapping quality	20 %	n/a	30.79	6.16
Generation quality per operation	25 %	n/a	33.65	8.41
Manual repair effort	20 %	n/a	64.95	12.99
Repeatability	15 %	n/a	77.77	11.67
Operational readiness	10 %	3	60.00	6.00
Total	100 %	—	—	53.23

The documentation links suggested by the code generator services were accurate and helpful in regard to the "User" object class. Documentation for the "Group" object class was not reliably identified and relevant excerpts for most operations could not be obtained even after multiple attempts to refine or supplement the provided documentation links. Moreover, certain inconsistencies were observed, particularly regarding the suggested base API URL and the clarity of the descriptions for each suggested authorization option. Similarly, as in the previous case, some workarounds had to be done due to the impossibility to define authorization headers in read-related operations.

List operations were, in most cases, generated correctly with only occasional inclusion of redundant or interchangeable endpoints. By contrast, filter operation scripts exhibited lower precision, frequently containing syntactic inaccuracies, irrelevant attributes, or suboptimal endpoint selections. Addressing these issues required a moderate level of effort, primarily

involving consultation of the documentation and alignment with framework-specific constructs.

Create and update operations demonstrated similar patterns. Generated scripts often included redundant endpoints or unsupported constructs, such as inappropriate use of some framework builder methods.

In the case of update operations, some endpoints required multiple path parameters or more complex handling of attribute changes, which exceeds the current capabilities of the framework. Notably, in some instances, the generator indicated these limitations through approximate or pseudo-code representations which were considered a positive aspect in the evaluation. Further inconsistencies in update scripts included occasional mismatches in HTTP method usage and the presence of irrelevant endpoints. The effort required to refine these scripts remained moderate.

Write operations, however, could not be fully executed due to the requirement for an additional wrapper structure in request payloads, which is not currently supported by the framework.

Delete operations required minor manual adjustments, particularly in selecting the correct endpoint as they were not consistently identified in the generated scripts. Overall, most observed issues were associated with imprecise endpoint selection or redundancy.

Despite these limitations, the generated outputs provided usable foundation.

Atlassian

Atlassian Cloud provides a SCIM-based REST API for identity provisioning centered around core entities, such as users and groups within an organization. The API supports standardized CRUD operations, enabling automated user lifecycle management and integration with external identity providers.

For the "User" object class, the following artifacts were generated:

- nativeSchemaScript
- connidSchemaScript
- createScript
- updateScript
- searchFilterOperation
- searchAllOperation
- searchIdOperation
- deleteScript

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	60.00	71.37	60.00	76.25
connidSchemaScript	100.00	96.00	96.0	4.40
createScript	66.00	71.36	60.0	73.64
updateScript	84.00	31.88	52.0	65.71
searchFilterOperation	60.00	60.16	80.0	51.75
searchAllOperation	28.00	34.35	80.0	78.57
searchIdOperation	40.00	20.00	80.0	22.22
deleteScript	0.00	22.36	80.0	65.86
Average	54.75	50.93	73.5	66.20

The "Group" object class has the following metrics, evaluated based on the same artifacts as the previous object class:

Artifact	Endpoint and mapping quality	Generation quality	Manual repair effort	Repeatability
nativeSchemaScript	94.00	84.78	80.0	90.89
connidSchemaScript	80.00	97.14	96.0	74.12
createScript	86.00	83.92	80.0	52.69
updateScript	30.00	60.12	80.0	92.15
searchFilterOperation	46.00	62.13	76.0	79.50
searchAllOperation	40.00	61.23	80.0	62.58
searchIdOperation	50.00	50.00	80.0	100.00
deleteScript	0.00	31.30	80.0	67.75
Average	53.25	54.19	81.5	77.46

Overall metric:

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Information Discovery	10 %	4	80.00	8.00
Endpoint and mapping quality	20 %	n/a	54.00	10.80

Metric	Weight	Score (0–5)	Normalized (0–100)	Contribution
Generation quality per operation	25 %	n/a	52.56	13.14
Manual repair effort	20 %	n/a	77.50	15.50
Repeatability	15 %	n/a	71.83	10.77
Operational readiness	10 %	2	40.00	4.00
Total	100 %	—	—	62.21

The evaluation of this system demonstrated generally good alignment between the generated outputs and the target environment, despite certain limitations related to documentation completeness and framework capabilities. While some documentation links had to be supplemented manually, the available materials were sufficient to support the overall process. Although the current implementation primarily supports scalar attributes, the generator showed promising capability in partially incorporating more complex structures, including those associated with schema extensions.

Minor inconsistencies were observed in schema and script generation. In some cases, native schema definitions included elements of the ConnId schema that would ideally be separated into a dedicated script, and certain scripts combined REST-oriented constructs with SCIM-specific builders. These issues, however, were straightforward to identify and resolve. Similarly, filter and create scripts required moderate refinement, mainly due to the inclusion of irrelevant attributes or the explicit definition of properties that could otherwise be handled implicitly.

Update scripts occasionally contained incorrect operation types or included redundant attribute constraints, while delete scripts tended to be more verbose than required. Notably, the generator successfully identified relationships between entities, such as user-to-group associations.

Overall, the generated scripts provided a reliable starting point for implementation, with required adjustments remaining within a manageable scope. Although full execution could not be validated due to undocumented limitations within the Atlassian environment, which were not present in the documentation and were identified only through external support channels, the generator proved effective in accelerating development and supporting the engineering workflow nonetheless.

4.7.4 Configuration Assistant Test Environments

This chapter describes the environments used for the end-to-end testing.

MCM 2026 Workshop Environment

The testing environment is based on the environment originally prepared for the MidPoint

Community Meetup 2026 (MCM) workshops. The environment contains data in source and target systems. The environment corresponds to an "enterprise" environment with a single data source.

Source and Target Systems

The environment contains two systems:

- 1. HRIS (source system)
 - A simple HR system with two categories of persons capable of exporting data to a CSV file.
- 2. LDAP (target system)
 - Contains several types of users
 - Account data contain deliberate mistakes.

Evolveum Infrastructure Environment

The testing environment is based on data from Evolveum production infrastructure. The environment contains data in a single source system and several target systems. The source system contains multiple types of records (e.g., employees, partners, subscribers, community). The data do not overlap. The environment corresponds to an "enterprise" environment with a single data source.

	During the test, the data were protected according to the Evolveum security guidelines. The test was supervised by the Evolveum Chief Information Security Officer.
NOTE	Most of the data were used in a form of CSV files to contain the testing in a local controlled environment on engineers' laptops. No credentials (e.g., passwords) were used in this scenario. No public services were used to process the testing data. Only internal AI services running on Evolveum hardware in Evolveum infrastructure accessible via a VPN were used to process the data.

Source and Target Systems

The environment comprises the following systems:

- 1. CSV file (source system)
 - CSV file is exported from the production system and contains multiple categories of users:
 - a. Evolveum employees
 - b. Partner users
 - c. Subscriber users

- d. Community users
 - e. System users
 - f. Administrator users
 - g. External users
2. LDAP (target system)
 3. Partner portal (target system)
 - Data exported to a CSV file
 4. Learning portal (target system)
 - Data exported to a CSV file

Customer-like PoC Environment

The testing environment is based on an existing PoC environment with synthetic data. The environment data was originally used for testing delineation, correlation and mapping suggestions during the project development. The environment contains data in source and a target system. The environment corresponds to an "enterprise" environment with a single data source.

Source and Target Systems

The environment contains two systems:

1. CSV file (source system)
 - CSV file contains synthetic data, but its structure is based on real PoC data. It contains active and inactive users.
2. LDAP with Active Directory Data Structure (target system)
 - Contains user accounts

Customer-like Environment

The testing environment is based on an existing environment with synthetic data. The environment contains data in source and a target system. The environment corresponds to an "enterprise" environment with a single data source. This environment contains tens of thousands of users/accounts.

Source and Target Systems

The environment contains two systems:

1. CSV file (source system)
 - CSV file contains synthetic data, but its structure is based on real PoC data. Multiple user

types are used.

2. LDAP (target system)
 - Contains user accounts (multiple types).

4.7.5 Configuration Assistant End-to-End Test Scenarios

In this chapter, we describe the generic test scenarios followed by the descriptions of these tests for our testing environments.

Resource-based Configuration With Existing Connector

Before testing any configuration assistants, a resource must be created.

1. As a midPoint administrator or engineer, create a new resource in midPoint based on an existing connector.
 - a. Specify the basic configuration.
 - b. Specify the connection configuration.
 - c. Validate and update discovered configuration.
 - d. Validate object classes.
 - e. Save the resource.
2. As a midPoint administrator or engineer, preview objects on the resource to verify the connection.

The resource is kept in the **proposed** lifecycle state.

Resource - Delineation Configuration

Delineation suggestions (object type candidates) are tested first.

1. As a midPoint administrator or engineer, **generate suggestions for object types using the configuration assistant (AI)**.
 - a. Select one of the object types and review the suggested configuration (name, display name, kind, intent, focus type, etc.).
 - b. Confirm and save the object type suggestion.
2. Validate the configuration.
 - If needed, repeat the process—regenerate the object type suggestions if they are incorrect.

Resource - Object type configuration

Object types are configured (correlation, inbound and outbound mappings). For the first

authoritative source system, no correlation is required.

1. As a midPoint administrator or engineer, **generate correlation suggestions using the configuration assistant (AI)**:
 - a. Use schema matching (attribute name or data type matching, schema-driven correlation).
 - b. Validate the configuration.
 - If needed, repeat the process—regenerate the correlation suggestions if they are incorrect.
 - c. Run a simulated correlation to understand the impact of the correlation configuration.
 - d. Mark uncorrelated or impossible to correlate accounts for later inspection.
 - e. Accept and save the suggested correlation configuration.
 - f. Verify that a new inbound mapping is created for correlation.
2. As a midPoint administrator or engineer, **generate synchronization configuration using the configuration assistant (system suggestion)**:
 - a. Specify the information data flow direction (source→target) for this object type.
 - b. Suggest synchronization configuration based on the data flow direction.
 - c. Accept and save the suggested synchronization configuration.
3. As a midPoint administrator or engineer, **generate inbound mapping suggestions for the source system using the configuration assistant (AI and heuristics, if available)**:
 - a. Suggest inbound mappings (resource object attributes → midPoint).
 - i. If there are multiple suggested mappings for the same target midPoint property, allow the administrator or engineer to choose.
 - b. Validate the scripts (if generated).
 - c. Remove unnecessary mappings.
 - d. Identify missing mappings.
 - e. Validate the configuration.
 - If needed, repeat the process—regenerate the mapping suggestions if they are incorrect.
4. As a midPoint administrator or engineer, **generate outbound mapping suggestions for the target system using the configuration assistant (AI and heuristics, if available)**:
 - a. Suggest outbound mappings (midPoint → resource object attributes) using AI and system suggestions.
 - i. If there are multiple suggested mappings for the same target resource attribute, allow the administrator or engineer to choose.

- b. Validate the scripts (if generated).
- c. Remove unnecessary mappings.
- d. Identify missing mappings.
- e. Validate the configuration.
 - If needed, repeat the process—regenerate the mapping suggestions if they are incorrect.

Resource - Object Type Configuration Simulation and Activation

Configuration is simulated to evaluate its correctness and impact on the data. After successful evaluation, the configuration is activated.

1. As a midPoint administrator or engineer, **run a simulated reconciliation for the source and target systems (for the configured object type)**.
 - a. Review the simulation results and adjust object type configuration if needed.
2. As a midPoint administrator or engineer, activate the resource configuration.
 - a. Set lifecycle state of the resource to active.

Resource - Object Type Data Synchronization

Data between midPoint and source and target systems can be synchronized (for the configured object type).

1. As a midPoint administrator or engineer, run the reconciliation task defined for the resource in midPoint.
 - a. Review the reconciliation results:
 - i. Source system: the resource accounts linked to midPoint users; these create midPoint focal objects (e.g., users) if the system is configured as an authoritative source of data.
 - ii. Target system - linking phase: the resource objects (e.g., accounts) linked to midPoint focal objects (e.g., users).
 - iii. Target system - outbound phase: the resource objects (e.g., accounts) are updated based on midPoint policies (e.g., outbound mappings); midPoint is authoritative for the resource data.

MCM 2026 Workshop Environment: Test Description

1. We used the HRIS CSV data export as a source system for midPoint user creation using a standard CSV connector. We used delineation and focus type suggestions during the resource configuration, we worked only with employees, disregarding contractors. During the configuration, we used inbound mapping suggestions and simulated the configuration before activating it.

2. We used LDAP as a target system using a standard LDAP connector. We used delineation and focus type suggestions, we worked only with standard user accounts, disregarding service and administrative accounts.
3. The standard user accounts in LDAP were correlated with midPoint users using correlation suggestions by personal numbers. Any uncorrelatable accounts were marked for later processing.
4. We used LDAP system, while being a target system, also as a source of usernames for correlated users, allowing using LDAP usernames for midPoint users.
5. We used outbound mapping suggestions for LDAP standard accounts, and simulated them before activating the configuration.
6. After activating the LDAP outbound mappings, we configured midPoint for a regular synchronization with HRIS to allow creation of LDAP account for each employee.

In this environment, we focused on users and their accounts. Testing scenarios were repeated with Groovy and MEL scripting languages.

The aforementioned approach is based on Evolveum's [First Steps with midPoint](#) methodology.

Evolveum Infrastructure Environment: Test Description

1. We used the CSV source data export as a source system for midPoint user creation using a standard CSV connector. We used a suggested delineation by **archetype** column and focus type suggestions during the resource configuration. We worked with Evolveum employees, Partners, Subscribers and Community users, disregarding External users, Administrator accounts and System users (only several such users exist in the CSV file). During the configuration, we used inbound mapping suggestions and simulated the configuration before activating it.
2. We connected LDAP as a target system using a standard LDAP connector. We used delineation by e-mail address suffix for LDAP to have two sets of accounts: Evolveum and non-Evolveum users, which corresponds to the actual usage in Evolveum infrastructure.
3. We correlated LDAP accounts using username matching for both sets of accounts. Any uncorrelatable accounts were marked for later processing.
4. We used outbound mapping suggestions for LDAP accounts for both sets of accounts, and simulated them before activating the configuration.
5. We connected both portals as target systems using a standard CSV connector. We used delineation and focus type suggestions, but ultimately used a single object type for each portal (which corresponds to the actual usage in Evolveum infrastructure).
6. Accounts in both portals were correlated with midPoint users using correlation suggestions by usernames matching. Any uncorrelatable accounts were marked for later processing.
7. We used outbound mapping suggestions for both portals and simulated them before activating the configuration.

In this environment, we focused on users and their accounts. Testing scenarios were repeated with Groovy and MEL scripting languages.

The aforementioned approach is based on Evolveum's [First Steps with midPoint](#) methodology.

The principal difference between the first testing scenario and this one is that it is not based on fabricated data. There were several types of users, better representing a more complex use case.

Customer-like PoC Environment: Test Description

1. We used the CSV source file as a source system for midPoint user creation using a standard CSV connector. Delineation generator suggested delineation by user state (active/former employee), which was not accepted, and ultimately a single catch-all object type was created by manual adjusting the remaining delineation suggestion. During the configuration, we used inbound mapping suggestions and simulated the configuration before activating it.
2. We connected LDAP as a target system using a standard LDAP connector. Delineation generator suggested delineation by user state (active/former employee), which was not accepted, and ultimately a single catch-all object type was created by manual adjusting the remaining delineation suggestion.
3. We correlated LDAP accounts using personal number matching employeeNumber LDAP attribute.
4. We used outbound mapping suggestions for LDAP accounts and simulated them before activating the configuration.

In this environment, we focused on users and their accounts. Testing scenarios were repeated with Groovy and MEL scripting languages.

The aforementioned approach is based on Evolveum's [First Steps with midPoint](#) methodology.

This scenario contained data with more account attributes, better representing a more complex use case.

Customer-like Environment: Test Description

1. We used the CSV source file as a source system for midPoint user creation using a standard CSV connector. Delineation generator suggested delineation by user type (three types). We used all three suggested object types in further tests. During the configuration, we used inbound mapping suggestions and simulated the configuration before activating it.
2. We connected LDAP as a target system using a standard LDAP connector. Delineation generator suggested delineation by subtrees. We used all three suggested object types in further tests.
3. We correlated LDAP accounts using personal number matching employeeNumber and businessCategory LDAP attributes.
4. We used outbound mapping suggestions for LDAP accounts and simulated them before

activating the configuration.

In this environment, we focused on users and their accounts. Testing scenarios used scripts with MEL scripting language.

The aforementioned approach is based on Evolveum's [First Steps with midPoint](#) methodology.

This scenario contained multiple types and larger numbers of users/accounts, better representing a more complex use case.

4.7.6 Configuration Assistant Quantitative Results

The configuration-assistant pipeline was evaluated quantitatively at the scenario level. The scoring intentionally stays close to the practical review workflow used by identity engineers: it measures whether suggestions were usable, whether they covered the expected configuration, and how much review or correction was needed.

Object type and delineation suggestions were scored per scenario as 0, 0.5, or 1. A score of 0.5 means that at least part of the suggestion was usable, even if the final object type configuration required manual adjustment. Correlation was scored per scenario as 0 or 1, which corresponds to a simplified recall-style measure of whether the suggested correlation could be used.

Mappings were evaluated per scenario using precision, recall, and F1 over all suggested mappings, including both as-is mappings and script mappings. At this level, the metric evaluates whether the required mapping was suggested; it does not yet judge whether the generated mapping script itself was fully correct. Script mappings were therefore evaluated separately in two ways: first as precision, recall, and F1 for script-mapping presence, and second as a per-script score of 0, 0.5, or 1, where 0.5 means that the recommended script was directionally useful but required manual correction.

Area	Unit	Metric	Evaluated items	Aggregate result	Notes
Object type / delineation suggestions	Scenario	Score 0, 0.5, 1	14	Average 0.714	Partial usable suggestions were counted as 0.5.
Correlation suggestions	Scenario	Score 0, 1	11	Average 1.000	This is a simplified recall-style measure.
Mapping suggestions	Scenario	Precision, recall, F1	24 evaluable scenarios	Precision 0.530, recall 0.946, F1 0.680	Micro average over TP=193, FN=11, FP=171. Mean per-scenario values were precision 0.540, recall 0.950, F1 0.663.

Area	Unit	Metric	Evaluate d items	Aggregate result	Notes
Script mapping suggestions	Scenario	Precision, recall, F1	19 scenarios with script mappings	Precision 1.000 , recall 0.852 , F1 0.920	Micro average over TP=23, FN=4, FP=0. Mean per-scenario values were precision 1.000 , recall 0.895 , F1 0.930 .
Recommended script quality	Recommended script	Score 0, 0.5, 1	25 scored scripts	Average 0.840	A score of 0.5 means that the script was useful but needed manual correction.

The results confirm the same tendency as the qualitative evaluation. The configuration assistants were strong at recall: expected correlations were found, and most required mappings were suggested. Precision was lower for mapping suggestions because extra suggested mappings were counted as false positives, even though the delivered workflow expects engineers to review and remove unnecessary suggestions before activation. Script mapping quality was generally usable, but several generated scripts still required manual correction before being accepted.

4.7.7 Configuration Assistant Results By Environment

MCM 2026 Workshop Results

During the testing, we used the suggestions for delineation, focus type, correlation and mappings. All of them significantly simplified the configuration of midPoint - and made it much faster. The delineation suggestions for HRIS were correct in most of the cases during the first iteration; rarely we needed to regenerate the suggestions (when the recommender used delineation using different/additional attributes than expected, e.g., using employee's status). The inbound mapping suggestions for HRIS employees contained all required mappings and few extra ones. For one of the mappings, a (correct) script was suggested to convert employee's status from HRIS to midPoint "lifecycleStatus" property. For other mapping, the script needs to be added manually by administrators/engineers, as it was not possible to recommend the script automatically for given data.

The delineation suggestions for LDAP standard accounts were correct in most of the cases during the first iteration; rarely we needed to regenerate the suggestions (when the recommender used delineation using different/additional attributes than expected, e.g., using username suffix). The correlation suggestion for LDAP was repeatedly correct, basing the suggestion on employee numbers. The inbound mapping suggestion for LDAP repeatedly contained the mapping used to import LDAP username to midPoint. The outbound mapping suggestions repeatedly contained all required mappings including scripts (suggested by AI and

built-in heuristics for LDAP-based systems). The scripts were repeatedly suggested correctly, yet with a slightly variable output for the AI-based script(s), which is typical for LLM.

The attribute/property statistics provided extremely helpful insights during mapping suggestion validation.

We also tested the scenarios without AI (by using suggestions, but rejecting access to data and schema), just with using built-in algorithms and/or heuristic functionality: testing was of course possible, but there were limitations. For example, while the mapping suggestions work without AI, the scripts were suggested only for selected target systems/connectors (e.g., LDAP). Also, the delineation suggestions require AI; without it, only a manual configuration is possible.

Short Summary

Using recommendations and suggestions combined with simulations allowed to significantly increase the speed of midPoint configuration, while keeping the solution stable, and while administrators/engineers have a complete control over the configuration.

For administrators/engineers using midPoint GUI wizards, the suggestions were extremely helpful.

The suggestions were consistent during repeated test executions.

Evolveum Infrastructure Results

During the testing, we used the suggestions for delineation, focus type, correlation and mappings. All of them significantly simplified the configuration of midPoint - and made it much faster.

The delineation suggestions for Source CSV system were correct in most of the cases during the first iteration.

The inbound mapping suggestions for Source data (Employees, Partners, Subscribers, Community) were mostly correct. They worked for standard midPoint properties as well as extension ones. At least one mapping could be improved to parse for multivalue data in CSV format; this is an obvious limitation of CSV-based system, which does not have any schema information about data cardinality and engineer must configure the CSV resource explicitly to support such configuration. (This should not happen with systems, where the connector can parse the resource schema.)

One disadvantage of suggesting categorical mappings is that there are inherently based on the *existing values* of attributes, and that they can not predict additional values as they may occur. Such mappings for state property (e.g., `lifecycleStatus`) can suggest mappings between some, but not all values, because they simply do not exist in data. All mappings are subject to engineer's validation, and they can be updated when necessary.

It was also interesting to see the "statistics thresholds" in action: for example, a mapping for

Community users attribute "job title" was not suggested, because only a small set of users were actually having values filled, below the default threshold of 5%. In such cases, the engineer configuring midPoint, who understand the data, can manually add such mapping.

With LDAP, the delineation results were a bit over-suggested (considering also e.g., test accounts), but they could be slightly manually updated to create two object types, and were usable even though the data structure in LDAP was not ideal.

Correlation of LDAP accounts was originally tried with by e-mail addresses, but this caused problems later as there are multiple LDAP accounts using the same e-mail addresses, and, by default, midPoint does not allow multiple accounts linked to the same correlated owner in midPoint. This is obviously a drawback of the currently implemented solution that will be addressed in the following weeks. So, even though the correlation suggestion provided a little bit better estimations for e-mails, another correlation suggestion using usernames was ultimately used.

Outbound mapping suggestions for both LDAP object types were repeatedly suggesting correct mappings (suggested by AI and built-in heuristics for LDAP-based systems).

With both portals, delineation results were consistently over-suggesting object types based on patterns that are not used in Evolveum (e.g., by information about empty/non-empty course list). As Evolveum is using a single object type here, these suggestions were not used and a single object type configuration was created. We will consider implementing a GUI-initiated action to allow administrators/engineers to request a single object type delineation configuration.

Outbound mapping suggestions for both portals were consistent with the following limitations: the suggested mapping scripts for full name-related attributes would not work correctly for users with multiple family names and required a manual modification to cover such cases (this can be explained by the fact that there are not many such users, the mapping script quality depends on the obtained random samples).

Short Summary

Using recommendations and suggestions combined with simulations allowed to significantly increase the speed of midPoint configuration, while keeping the solution stable, and while administrators/engineers have a complete control over the configuration.

For administrators/engineers using midPoint GUI wizards, the suggestions were extremely helpful.

Most of the suggestions were consistent during repeated test executions.

While testing this scenario (which contained more realistic data in comparison with the previous one), it became clear, that even in complex environments, the midPoint configuration assistants help administrators/engineers in many ways: they hint (but do not push) the next configuration steps; show patterns in data (while allowing administrators/engineers to ignore

them); suggest mapping scripts (while allowing administrators/engineers to validate, correct or replace them).

With more complex environments, the number of suggestions to review increases. Also, the number of needed adjustments in the suggested configurations increases. With a proper usage of configuration lifecycles and simulations, such adjustment requirements are safely reported in simulation reports as unexpected changes, and administrators/engineers do not jeopardize data of the systems being connected.

The configuration assistants help administrators/engineers to make their own decisions; the final responsibility is theirs. It is always the human in the loop who is responsible for the resulting configuration.

We also need to note that configuration assistants do not replace environment analysis done by engineers, which is required during IAM/IGA projects.

Customer-like PoC Environment results

During the testing, we used the suggestions for delineation, focus type, correlation and mappings. Some of them were insufficient, and required manual adjusting of the results, e.g. delineation and scripted mappings.

The majority of inbound mapping suggestions for Source data were correct for standard midPoint properties as well as extension ones. MidPoint username mapping script required a manual adjustment, and one mapping was consistently missing from the suggestions and required a manual input. This is partially a consequence of how the synthetic data was generated in the environment. But all suggested configuration was subject to engineer's validation.

With LDAP containing basically "Active Directory data", the delineation suggested two object types (for active and inactive accounts). Such delineation was ignored and a single object type was used. We will consider implementing a GUI-initiated action to allow administrators/engineers to request a single object type delineation configuration. Correlation by personal numbers was a 100% match.

Outbound mapping suggestions for LDAP were repeatedly correct except two scripted mappings, which required a manual adjustment. Again, this was partially a consequence of how the synthetic data was generated in the environment.

Short Summary

Using recommendations and suggestions combined with simulations allowed to significantly increase the speed of midPoint configuration even in the environment which required more manual adjustments and inputs from administrators/engineers. With more supported account attributes, the number of mapping suggestion increases. The administrator/engineer has a complete control over the suggestions and is responsible for removing the unnecessary ones.

Customer-like Environment

During the testing, we used the suggestions for delineation, focus type, correlation and mappings. All of them significantly simplified the configuration of midPoint - and made it much faster. As this environment contains several thousands of users/accounts, the ability to run tasks with multiple threads was a must. The delineation suggestions for source system were correct. The inbound mapping suggestions for source system were correct for all object types with few additional suggestions.

The delineation suggestions for LDAP accounts were correct for all object types. The correlation suggestions for all LDAP object types were repeatedly correct, based on unique identifiers (employeeNumber, businessCategory attributes). The outbound mapping suggestions repeatedly contained almost all required mappings including scripts (suggested by AI and built-in heuristics for LDAP-based systems). Few outbound mapping suggestions were always missing in all object type configurations and required engineer's intervention. This was mainly caused by the synthetic data limitations. The mapping scripts were repeatedly suggested correctly.

Short Summary

Using recommendations and suggestions combined with simulations allowed to significantly increase the speed of midPoint configuration even in the environment which required few manual adjustments and inputs from administrators/engineers. The performance of suggestions, statistics computation and reconciliations required increasing the number of threads for task executions, otherwise the testing scenario would take too much time. While the tasks can be created with a specific number of threads, for some scenarios the configuration option is quite hidden - there is a potential room for improvement of midPoint UX in future releases.

4.7.8 Key Findings and Limitations

Based on the end-to-end testing of the whole solution, we can summarize the following key findings and limitations:

1. The quality of the generated connector scripts is strongly influenced by the completeness and accuracy of the provided documentation.
2. Suggested documentation sources require verification to ensure that they contain sufficient and relevant information for correct artifact generation.
3. A solid level of domain knowledge is necessary, both for understanding the integrated systems and for validating generated outputs against the documentation.
4. Fully correct script generation on the first iteration is unlikely, iterative validation and refinement should be considered an integral part of the process.
5. The effectiveness of the generated scripts is also dependent on the quality and extent of available framework documentation and examples.
6. Systems with more complex data models, custom sub-resources, or APIs that partially follow

RPC-style patterns tend to yield less precise outputs, which is consistent with their inherent complexity.

7. While relationship suggestions are generally accurate, the corresponding scripts typically require further refinement to reach a production-ready state.
8. Suggestions generated by the configuration assistants significantly increase the speed of the configuration for delineations, correlations and mappings.
9. Configuration assistants can reveal patterns in data, but they do not necessarily represent how data is managed (e.g., delineation suggestions based on patterns, which are not used in connected systems in such ways).
10. System suggestions can be more correct than AI-based suggestions for some mappings in supported connectors.
11. Generated mapping scripts always require a special attention, some need corrections.
12. With more complex environments, the number of suggestions to review/correct increases.
13. With larger number of accounts, multi-thread configuration of tasks is a must.
14. Simulations of resulting configurations and human review help avoiding unexpected data modification or deletion.
15. Configuration assistants do not replace the environment analysis which is typically done by identity engineers in IDM/IGA deployments.
16. Keeping human in the loop is the key, as it is the human who is responsible for the configuration and its impact. Human has always a complete control over the configuration.

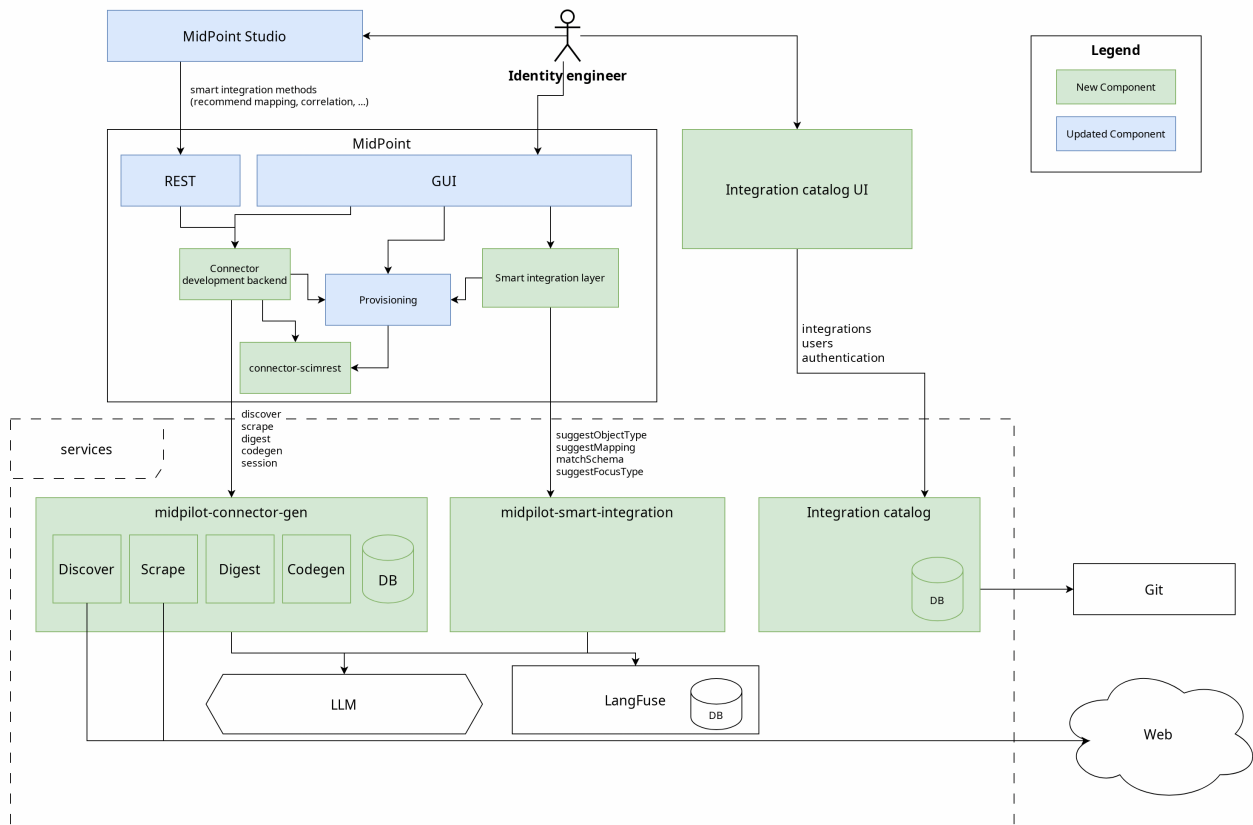
5. Architecture and Design

This chapter describes the final implemented architecture of the delivered solution. It focuses on the technical structure of the system, the boundaries between MidPoint, AI-assisted microservices, connector generation services, the integration catalog, and supporting operational components. While chapter 4 explains what the AI-assisted capabilities do, this chapter explains how the complete solution is structured and integrated.

5.1 Component Structure

The final delivered solution is organized around midPoint as the central orchestration system. The diagram below shows the main component groups and their runtime relationships:

- MidPoint Studio, GUI, and REST API provide the user-facing and tool-facing entry points.
- Inside midPoint, the smart integration layer and connector development backend orchestrate AI-assisted workflows and connect them with provisioning, validation, scoring, persistence, and user review.
- The external AI-assisted services, `midpilot-smart-integration` and `midpilot-connector-gen`, provide bounded recommendation and connector generation functions that midPoint invokes.
- The integration catalog, catalog UI, Git, web sources, LLM provider, and tracing/persistence components support publication, source material retrieval, model execution, and operational observability.



The delivered solution consists of the following main components:

- **MidPoint**: Provides the primary user workflow and technical orchestration layer.

Its AI-specific implementation is concentrated in the smart integration layer and in the connector development backend. Both of them are accessed via the GUI and the REST interface.

- The smart integration layer is the Java-side orchestration layer inside MidPoint: it calls the **midpilot-smart-integration** AI microservice where needed, combines its results with MidPoint data and rules, gathers statistics, validates and scores suggestions, handles retries and asynchronous activities, and prepares reviewable configuration changes for the user workflow.
- Connector development backend is Java-side orchestration layer inside MidPoint: it calls **midpilot-connector-gen** AI microservice where needed, stores AI-generated artifacts, orchestrates creation and edits of a connector, and validates and tests the connector using midPoint built-in provisioning services (the Provisioning component).
- **midpilot-smart-integration** service: A Python FastAPI-based microservice that exposes focused APIs for schema matching, mapping suggestion, correlation-related matching support, object type suggestion, and focus type suggestion. Its responsibility is not full onboarding orchestration. Instead, it provides bounded AI-assisted computations that midPoint can invoke and then post-process.

- **midpilot-connector-gen** service: A separate session-oriented Python microservice pipeline for connector generation. It handles discovery, scraping, structured extraction of documentation-derived knowledge, and generation of connector-specific artifacts. Its outputs are later consumed and persisted by the midPoint-side connector development logic.

NOTE

External microservices are used for narrowly-scoped AI-assisted tasks such as schema matching, mapping suggestion, object type suggestion, correlation-oriented matching support, and connector artifact generation.

This is an important difference from the milestone-1 target design. The final implementation is not a loose set of disconnected AI experiments. It is a coordinated architecture in which midPoint remains the controlling system and the AI services provide bounded recommendation or generation functions.

NOTE

Naming follows the implemented code and configuration:

- **midpilot-smart-integration** is the Python FastAPI microservice repository and deployed service name.
- **SmartIntegrationService** is the Java API inside midPoint; **SmartIntegrationServiceImpl** is its implementation in the **smart-impl** module, and **smartIntegrationService** is the Spring bean name used in code.
- **smartIntegration** is the midPoint system configuration section. Its fields include **serviceUrl**, **connectorGenerationUrl**, and **connectorFrameworkUrl**.
- In prose, "smart integration service" and "smart integration layer" are used as lowercase architectural terms unless a concrete code identifier is referenced.

- The integration catalog: A separate publication and distribution component. It stores application, implementation, and implementation version metadata, manages upload and publication workflows, and exposes downloadable connector artifacts together with related metadata and statistics.

In the diagram, supporting operational components are represented by the database-backed services, **Langfuse** tracing, Git, web documentation sources, and the LLM provider. Additional deployment and experiment tooling is described later in this chapter, but is not part of this high-level runtime component diagram.

5.2 Interface and API Specifications

The interfaces between the main components are explicit and service-oriented.

On the smart integration side, the **midpilot-smart-integration** microservice exposes dedicated endpoints for:

- `matchSchema`
- `suggestMapping`
- `suggestExtensionCorrelators`
- `suggestObjectType`
- `suggestFocusType`

These endpoints are intentionally granular. MidPoint uses them as building blocks and supplements them with local preprocessing, statistics collection, validation, scoring, deduplication, and result selection.

On the connector generation side, the `midpilot-connector-gen` service architecture is split into the `session`, `discovery`, `scrape`, `digester`, and `codegen` areas. The midPoint-side connector development logic communicates with these stages through session-scoped backend calls and background-job-oriented execution.

Within midPoint itself, the main internal interface is the `SmartIntegrationService` Java API, implemented by `SmartIntegrationServiceImpl` in the `smart-impl` module, together with operation classes and activity handlers for mapping, correlation, object type suggestion, and connector development. This gives the final solution a clear split: external services provide bounded AI functions, while midPoint provides stateful orchestration and end-user integration.

5.3 Connector Framework and Code Generation Architecture

Connector generation is implemented as a two-sided architecture. The external side is the `midpilot-connector-gen` microservice, and the internal side is the midPoint connector development backend.

The microservice pipeline is session-oriented. It stores uploaded and discovered documentation, processes it asynchronously, extracts structured information, and then generates connector artifacts per operation or schema area. This design allows discovery, documentation processing, and artifact generation to be run as separate persisted jobs instead of one blocking request.

Inside midPoint, connector development is represented as a first-class persisted object. The `ConnectorDevelopmentType` model stores application data, documentation sources, detected schema information, authorization information, object classes, relations, and generated artifacts. The midPoint `ConnectorDevelopmentBackend` contains the logic of communication with microservice, state synchronization between `ConnectorDevelopmentType`, and materializes generated outputs into the connector project structure and regenerates the connector manifest. The midPoint connector generation wizards drive the process based on the described flow and user input by invoking `ConnectorDevelopmentBackend`, creating a test resource using the connector under development, and invoking integration tests using midPoint provisioning APIs.

This means the final architecture does not end with generated text. It supports a stateful development workflow in which generated artifacts become part of an evolving connector project structure aligned with the connector framework.

5.4 Mapping Recommendation Architecture

The mapping recommendation architecture is layered. The external `midpilot-smart-integration` service provides schema matching and mapping suggestion, while midPoint provides selection and control logic over the final result.

The first layer proposes candidate attribute correspondences between the application schema and the midPoint schema. The second layer proposes transformation logic for selected mapping pairs. The third layer, implemented in midPoint, evaluates `as-is`, heuristic, and AI-generated mapping variants using representative data, validation checks, and quality scoring.

This architecture is important because the delivered result depends on more than the raw LLM response. The final quality is shaped by heuristic rules, duplicate filtering, hallucination filtering, validation of generated expressions, and retries with validation feedback when needed.

5.5 Correlation Recommendation Architecture

The correlation recommendation architecture builds on schema matching and the midPoint-side correlator reasoning. The `midpilot-smart-integration` service contributes schema-level matching optimized toward recall of plausible correlation candidates. MidPoint then combines these matches with focus-type knowledge, existing mappings, and sampled focus and shadow data.

The final recommendation logic therefore has two distinct parts. The service layer tries to avoid missing useful candidates. The midPoint layer transforms these candidates into draft correlator proposals, scores them according to coverage and ambiguity, and prepares them for user review in the correlation workflow.

This is a practical final architecture for onboarding because it keeps the high-recall search in the microservice while preserving the final correlator selection and refinement in midPoint.

5.6 Delineation Architecture

The delineation capability is implemented as an object type suggestion architecture driven by schema information and computed statistics. MidPoint computes statistics for object classes and object types, serializes the relevant schema context, and sends this information to the `midpilot-smart-integration` object-type suggestion endpoint.

The service returns draft suggestions for `kind`, `intent`, delineation filter, and optional base-

context information. MidPoint then validates the returned filters, aggregates validation feedback, and can retry the generation with structured error context.

This architecture again reflects the final project pattern: the microservice generates bounded suggestions, while midPoint owns the validation, persistence, and user-facing refinement process.

5.7 User Interface Architecture

The final user-interface architecture is centered on the midPoint web UI. AI-assisted functions are integrated into existing or closely related onboarding workflows instead of being exposed as a separate autonomous front-end product.

From an architectural point of view, the web UI communicates with midPoint services, not directly with the external AI microservices. This preserves a single orchestration point for permissions, workflow state, validation, persistence, and final configuration changes.

The UI therefore reflects the layered backend design. Users initiate onboarding and review operations in midPoint, while midPoint itself invokes smart integration or connector generation services as needed and then returns validated suggestions or generated artifacts back into the user workflow.

5.8 Integration Catalog

The integration catalog is implemented as a separate application with a Spring Boot backend, Angular frontend, and PostgreSQL persistence layer. It is delivered as a standalone catalog application, not as a midPoint-internal runtime component.

The implemented domain model is centered on the **Application type** (presented in the UX as "Application" to simplify the interface), the **Integration method**, and the **Connector**. This structure allows one application type to have multiple integration methods. An integration method acts as a tutorial—detailing what to connect, how to connect it (including tutorial and sample configuration files), and the available integration options - it specifies the exact connectors to use. Integration methods can have multiple manually published versions. The model also tracks lifecycle states (Published, In Review, etc.), **Integration method types** (e.g., REST, SCIM, DB, which act as filtering tags), compatibility/capabilities, application tags, countries of origin, requests, votes, logos, catalog users, and organizations. The technical distribution format, the **Connector bundle** is managed by the system but purposefully hidden from the end user.

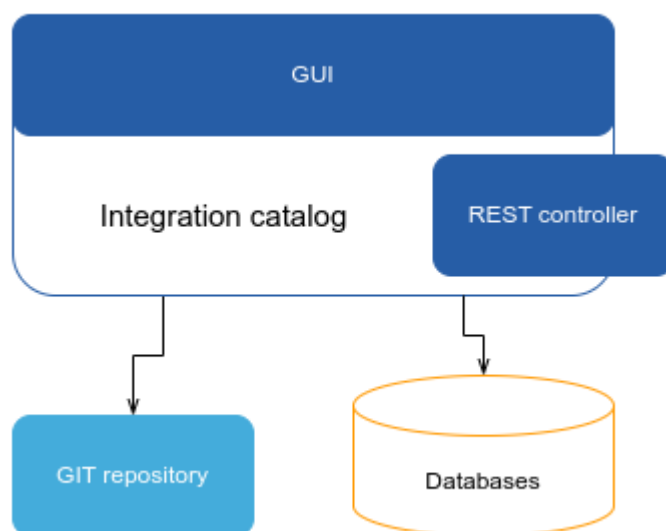
The backend exposes REST APIs for application listing and detail retrieval, active connector listing, application search, integration method listing, integration method upload, build verification, build success and failure callbacks, artifact download, download statistics, request creation and voting, tag and country metadata, capability metadata, logo upload and retrieval, recently used applications, and simple catalog authentication. The Angular frontend uses these APIs for the main catalog views: application list, application detail, request form, upload/publish

form, login modal, filter modal, and connector download actions.

The upload workflow creates or updates the relevant application, integration method, connector, connector bundle, version records, and stores metadata. After submitting this information, the newly created integration method is in the **IN_REVIEW** state, waiting for the Evolveum team to review the method and its connectors - this was implemented from the security and compliance reasons. After the review is done and the integration method is approved, integration catalog triggers the Jenkins job **integration-catalog-upload-connid-connector**. The Jenkins callback then promotes a successfully built connector version to **ACTIVE**, records build output such as connector version, download link, capabilities, connector class, and publish date, and sets the integration method lifecycle to **ACTIVE**. Failed builds mark the connector version, and in some cases the integration method, as **WITH_ERROR**.

The primary user experience revolves around discovering and downloading an integration method to connect an application to midPoint. Users search for an application and choose an integration method based on its method type, pre-selected connectors, and specified capabilities. When a user downloads an integration method, the package contains the connector builds along with metadata about the connectors, the integration method, and the application type. Application logos are stored as files with database metadata, and they are served through dedicated logo endpoints with content type and caching headers.

Architecturally, the integration catalog complements the onboarding pipeline. It is not the runtime execution environment for smart integration. Its role is publication, discovery, metadata management, request collection, artifact download, and reuse of produced integration artifacts.



5.9 Validation Services

Validation in the delivered solution is distributed across midPoint and the surrounding workflow. MidPoint validates generated mapping scripts, evaluates filter correctness for object

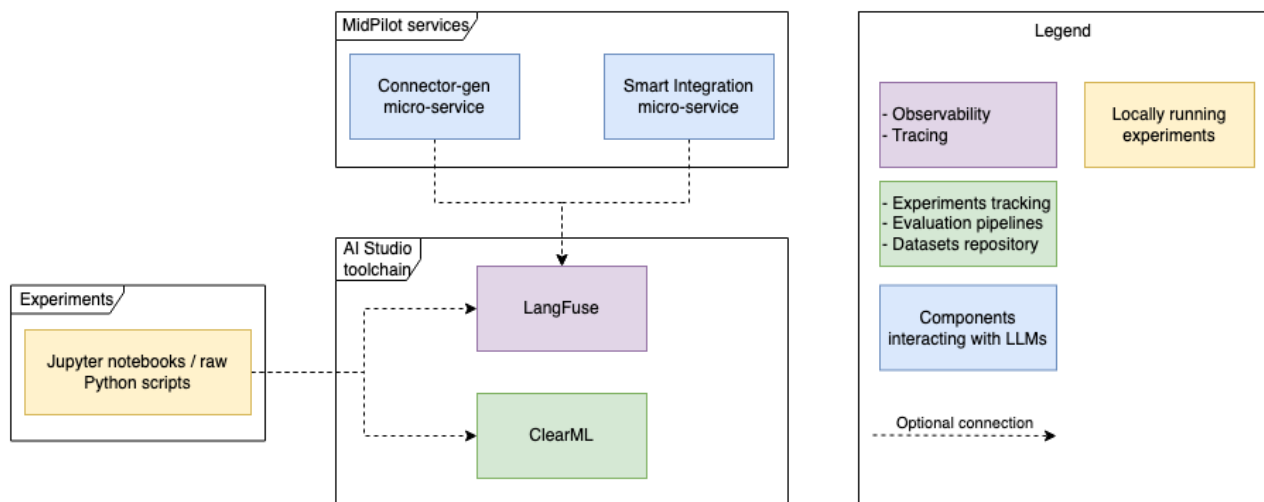
type suggestions, and uses validation results as feedback for retry when generation initially fails.

For mapping suggestions, script validation is part of the quality-selection process between **as-is**, heuristic, and AI-generated variants. For object type suggestions, filter and base-context validation can produce structured feedback that is sent back into a corrective retry cycle.

This validation architecture is one of the key final design decisions of the project. It ensures that AI output is not treated as final configurations by default, but as material that must pass platform-aware validation before being promoted into the onboarding flow.

5.10 Supporting Internal Tooling

Supporting internal tooling includes tracing, experiment tracking, and internal build/publication support.



Langfuse was used for tracing and observability of AI-assisted service execution and also for experiments aimed at better understanding the results. This provided visibility into prompt flows, retries, and runtime behavior of the smart integration and connector generation services.

ClearML was used for experiment management and development-time evaluation workflows. In practice, this gave the project a usable AI engineering toolchain even though the final result was not packaged as one separate **AI Studio** product.

Both **LangFuse** and **ClearML** are optional tools, which developers/ops may or may not use. However, they proved to be very useful, if not essential during the development. It is expected, that at least the observability part provided by LangFuse will be even more important in production deployment.

5.11 Security Considerations

The most important architectural security decision is that the delivered solution keeps midPoint in control of configuration state and treats AI outputs as draft suggestions or generated artifacts subject to review and validation. The architecture therefore avoids direct autonomous application of unvalidated AI-generated changes.

Secrets and runtime endpoints are externalized through environment-based configuration, secret files, and system configuration rather than being embedded in source code. The integration catalog deployment uses Docker secrets for external credentials, and the AI services use runtime configuration for LLM and observability access.

The final architecture also reduces operational risk through bounded service responsibilities. The external microservices do not own the full onboarding transaction. They provide focused outputs that are later validated, stored, and integrated inside midPoint.

5.12 Changes to MidPoint

Project-related changes in midPoint are substantial and represent one of the strongest delivered outcomes.

The MidPoint `smart-impl` module provides:

- smart integration orchestration and service-client integration
- schema matching support
- mapping suggestion orchestration
- correlation suggestion orchestration
- object type and focus type suggestion workflows
- statistics computation services
- mapping heuristics and quality assessment
- validation and retry mechanisms
- activity handlers for asynchronous execution of suggestion workflows

In parallel, the connector development backend adds:

- a persisted `ConnectorDevelopmentType` model
- backends for session-driven connector development workflows
- activity handlers for documentation discovery, processing, object-class analysis, artifact generation, and connector download
- Manifest generation and connector-project materialization of generated artifacts.

In order to enable connector development flows inside midPoint, i.e. to enable live editing of connector scripts, several changes have been made to connector loading and management.

In parallel with the midPilot work, related midPoint platform work introduced a new, safer, and more constrained alternative to the Groovy language which can be used across various areas of midPoint. In the future, we expect it to be adopted in areas where Groovy is currently used.

These changes have turned midPoint into the main integration and control layer for the delivered VAIA solution.

5.13 Changes to ConnId Framework

The delivered solution has produced framework-facing outputs for the connector ecosystem; especially through generated ConnId-related schema and operation artifacts, and through connector-framework-oriented project materialization in the connector development workflow.

At the architectural level, the project has therefore extended the practical use of the connector framework by automating preparation of framework-compatible connector pieces and by integrating those outputs into a structured development flow.

The outputs can be classified into three distinct areas:

- Extending core functionality of the ConnId framework - more detailed schema operations, support for complex attribute deltas, small changes necessary to enable low-code connector development.
- Methodology for developing connectors - guided workflow, and how to approach connector development with a focus on providing added value in the first steps.
- New development framework to ease the implementation of connectors by providing declarative reusable blocks, which can be used by developers and AI systems to write connector functionality without the need to reimplement common functionality on a per-connector basis.

5.14 Changes to midPoint Studio

The **midpilot** branch of midPoint Studio adds an expert-oriented user interface layer for selected smart integration workflows. The plugin registers dedicated actions for resource object type, mapping, correlation rule, and association suggestions. These actions call the midPoint smart integration endpoints through the midPoint Studio client, rather than calling the AI microservices directly.

The midPoint Studio workflow is centered on local resource configuration files. When a suggestion action is started from a resource file, midPoint Studio can first upload the resource with full processing, then lets the user select the target resource and either an object class or object type. Mapping suggestions additionally expose the inbound/outbound direction. The

generated suggestions are shown in a dedicated smart-suggestion tool window as filterable tables with action controls.

Applying a suggestion is implemented as a local review step. MidPoint Studio serializes the suggested resource change, opens a diff preview against the current local file, and lets the user accept the suggested change into the file. This keeps midPoint Studio aligned with the overall project architecture: AI-assisted output is presented for expert review and editing, not silently applied as final configuration.

The same branch also contains connector generator integration work. A registered **Connector Generator** action opens a wizard for application identification, documentation discovery, and connector identification. The client layer contains Remote Procedure Call (RPC) methods for creating connector development objects, and for submitting and polling connector generation and discovery operations through midPoint.

5.15 MEL - MidPoint Expression Language

As a related midPoint platform activity conducted in parallel with midPilot, Evolveum invested a lot of effort into the development of a new, safe, and intentionally constrained expression language. The main incentive was safety: the security risks introduced by using the Groovy language, which is very hard to constrain enough not to pose a security threat. The need became especially visible in the midPilot context, where AI-assisted mapping suggestions can generate executable expressions that must be reviewable and safe.

Our main goal was to introduce a language that can be safely used in most areas of midPoint where Groovy is currently used.

Based on our evaluation of viable existing solutions, we decided to build our new expression language on CEL (Common Expression Language), originally developed by Google Inc. The main benefit of CEL is its safety, which is achieved, among other factors, by the fact that it is not Turing-complete. This naturally introduces certain limitations. However, we think that the capabilities of CEL, combined with our custom extensions, are sufficient for most, if not all, of the use cases we have identified.

The main changes in midPoint that add support for MEL are in the following areas:

- Introduction of a new expression evaluator based on the **cel-java** library.
- Introduction of custom extensions that provide a lot of new functionality frequently needed in midPoint.
- Migration of selected areas (namely mapping suggestions) to generate MEL instead of Groovy where appropriate.

We also created a reference documentation that describes how MEL differs from CEL, how to use it, and how to migrate to MEL, e.g., from Groovy.

6. DevOps and Operational Setup

This chapter describes infrastructure and deployment-oriented aspects of the solution, including DevOps and CI/CD practices. The deliverables for this chapter are summarized in Section 8.4.

6.1 Hardware and Infrastructure

As mentioned in the Milestone 1 Report (D1), this project demanded procurement of new hardware, including a server with GPUs, and its placement in a professional data center to meet the compliance requirements. This required formulating requests for proposal (RFP), waiting for proposals, evaluating them, finalizing the order, and finally waiting for delivery. In an attempt to formulate an RFP allowing for fair competition, we repeated the main RFP for hardware after the first round with just one relevant option.

This process inevitably introduced some delay. However, in August 2025, the orders for both the servers and network devices were finalized. In the end, we ordered three Supermicro servers (one with GPUs) from Anafra s.r.o., and networking devices and cabling from Inalogy a.s. Network devices, including the firewall Netgate 8300 MAX, two switches - MikroTik CRS326-24S+2Q+RM and the management switch MikroTik CRS354-48G-4S+2Q+RM - were delivered shortly after. In September 2025, we also finalized the selection of a data center provider, choosing VNET a.s., with both primary and secondary data center sites located in various parts of Bratislava. On October 8, 2025, we installed the networking infrastructure into the primary data center rack, along with a temporary test server that served as a placeholder for the ordered Supermicro servers.

In the following months, we considered ourselves fortunate to have procured high-end hardware with a lot of computing power, plenty of RAM and enterprise NVMe disks at a very reasonable price. We closed the deal before the AI-driven demand caused a rapid increase of RAM (4-5x) and SSD pricing (2x or more). However, delivery was significantly later than expected; the first two non-GPU servers were delivered in November 2025, and the last GPU server was delivered on March 10, 2026.

The GPU server delay was the main reason the team had to rely on external services such as OpenRouter for a much longer period. At the same time, this reduced the available time for testing the solution on the new hardware. The infrastructure team did its best to be ready for the last GPU server delivery; most automation for operating systems, virtualization, and Kubernetes deployment was already in place. A GPU-enabled VM was provided to developers within two weeks of server delivery, including time required for physical installation, networking, software installation and some initial burn-in tests.

To improve automation, we adopted Ansible as the infrastructure automation and provisioning tool. We briefly experimented with Terraform for VM provisioning; although the tool worked as expected, in our context, Ansible achieved the same results with minimal overhead, allowing us to consolidate on a single tool. Ansible automation artifacts were mostly out of scope for the

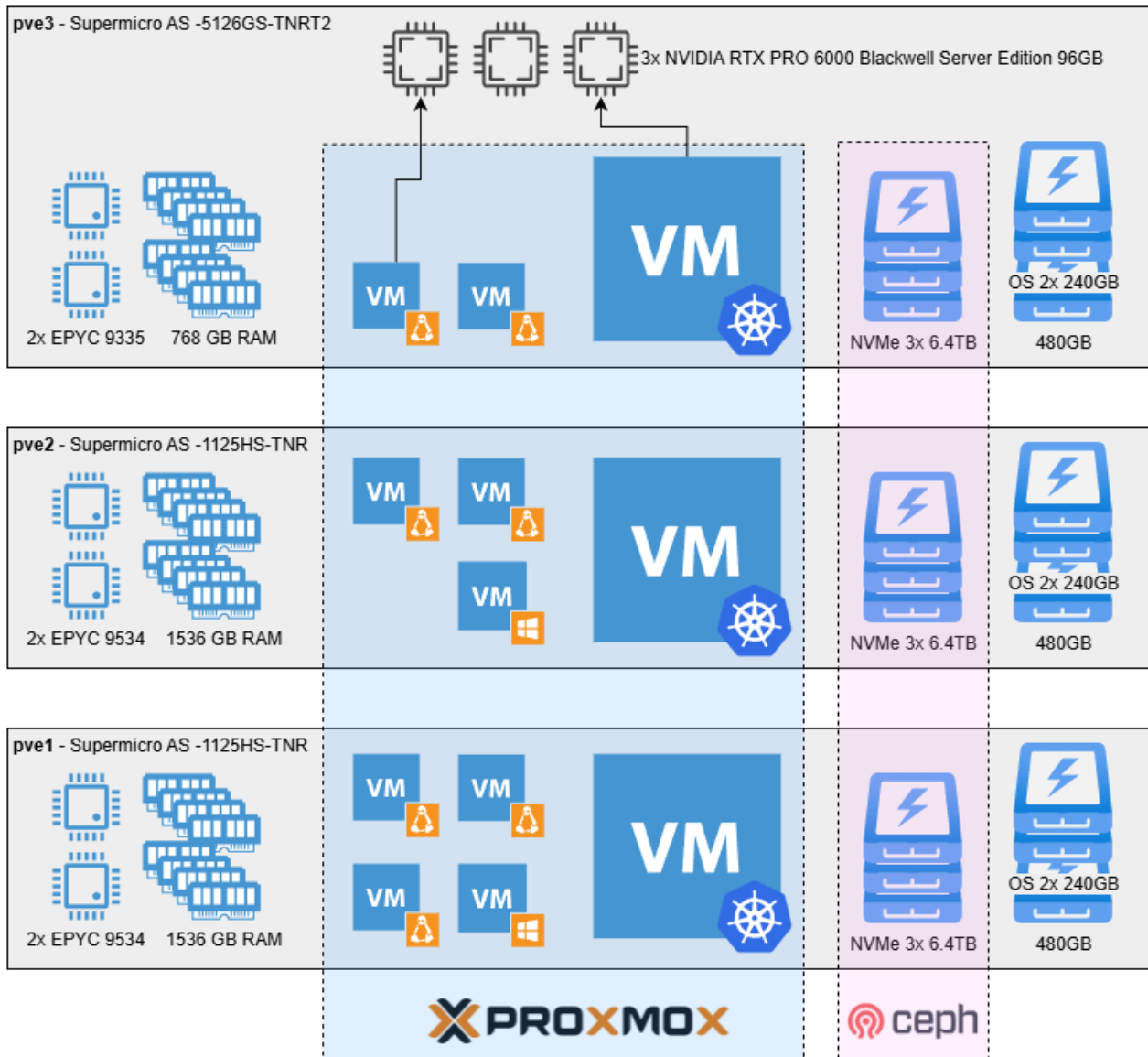
VAIA project; due to their infrastructure sensitivity, they are treated as proprietary internal assets and are not delivered publicly.

6.1.1 Current Hardware Setup

Current state of the new infrastructure generally matches our initial designs:

- Three servers, two general-purpose servers (Supermicro AS -1125HS-TNR, each with dual AMD EPYC Genoa 9534 and 1536 GB RAM) and one server with GPU for AI workloads (Supermicro AS -5126GS-TNRT2 with dual AMD EPYC Turin 9335 and 768 GB RAM).
- Networking with two MikroTiks CRS326 with 10Gbps SFP+ ports working in a MLAG configuration where two switches act as one. This allows the use of LACP bonds on machines connected through these switches, with each bond connected into both switches (each link going to a different switch), while the connected servers see register the two switches as a single "virtual" switch.
- A dedicated management switch provides a separate data plane for low-level administration; the details are not important for this project.
- The firewall interconnects the internal network and the internet, acting as both a router and a security device. The Netgate 8300 MAX uses an open-source firewall solution, pfSense, based on FreeBSD, which provides security features in a package offering a reasonable degree of flexibility. The hardware itself is strong enough to handle significantly more than firewall duties for our 1 Gbps uplink. It also functions as our internal DNS, reverse proxy (HAProxy), and can also provide WAF functionality, e.g., OWASP Coraza.

The following diagram is an overview of our PVE servers, their main HW features, and disk configuration:



This diagram shows that:

- Our PVE servers handle workloads using PVE and storage duties using Ceph.
- All workloads run inside VMs; the Kubernetes cluster is also built on top of VMs (with direct access to Ceph via a CSI driver).
- Server **pve3** provides GPUs for either VMs or Kubernetes pods; distribution and concrete configuration depend on the current needs. Section 6.7 summarizes GPU setup for PVE HW, OS, virtual machines and Kubernetes.

The diagram omits any networking and physical connections, as these are too low-level for the needs of this document.

Even in the short time available to finalize our PVE cluster in the new data center environment, we can confirm that the original design was sufficient and provides flexibility for future expansion.

Only some minor assumptions were miscalculated, e.g., we expected that local SSD drives may be beneficial for CI/CD pipeline workloads, but this assumption was based on our legacy hardware where even NVMe SSDs were generally bottlenecked by the legacy Ceph OSD nodes that were mostly suitable for HDD storage. The new infrastructure quickly showed that network storage based on Ceph built on top of PCIe Gen5 NVMe drives was so fast (both in sustained and IOPS heavy workload) that the local SATA3 SSDs were repurposed for other duties instead. This actually showed that investment in the recent NVMe technology paid off significantly.

6.1.2 GPU Performance Testing

We evaluated the performance of the internal GPU infrastructure to determine its operational limits and establish realistic performance expectations. The testing consisted of both general-purpose benchmarks and application-specific performance tests. Specifically, we measured the inference latency and throughput of the gpt-oss-120b large language model. The server is equipped with two NVIDIA RTX PRO 6000 Blackwell Server Edition graphics cards, each with 97,887 MiB of VRAM.

General Tests

We conducted general-purpose benchmarks on the described LLM infrastructure. The benchmark scenario used prompts with 4,096 input tokens and 1,024 output tokens, with a maximum concurrency of 500 requests.

The key results are summarized below:

Successful requests:	1,998
Failed requests:	2
Maximum request concurrency:	500
Benchmark duration (s):	574.83
Total input tokens:	8,183,808
Total generated tokens:	2,045,952
Request throughput (req/s):	3.48
Output token throughput (tok/s):	3,559.21
Peak output token throughput (tok/s):	451.00
Peak concurrent requests:	508.00
Total token throughput (tok/s):	17,796.07

Although the benchmark used relatively small, general-purpose prompts, the results indicate that the infrastructure is capable of handling a high volume of concurrent requests while maintaining stable operation.

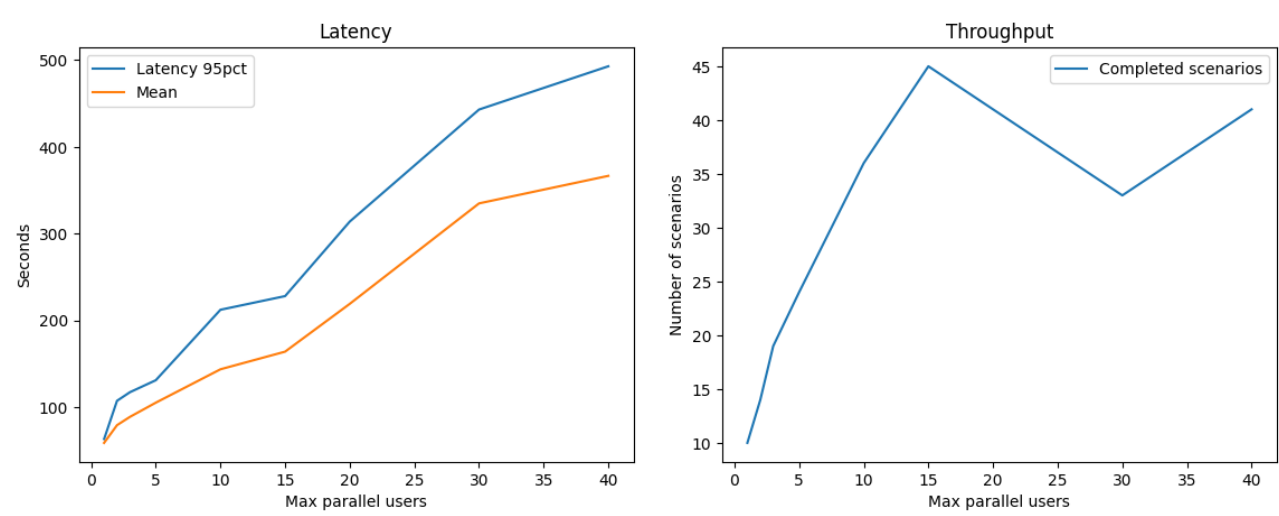
Application Specific Tests

To better understand the system behavior under realistic workloads, we performed an application-specific load testing that simulated real user interactions as if executed from midPoint.

The test scenario simulated a non-trivial workflow for object type suggestion, consisting of 5 parallel LLM requests performing the following tasks:

- Schema matching (reasoning-intensive).
- Object type suggestion (medium complexity).
- Three focus type suggestion requests (low complexity).

The tests used the gpt-oss-120b model configured with high reasoning effort, which represents the recommended production configuration for this application. The workload was executed with varying levels of parallelism ranging from 1 to 40 concurrent users. The figure below shows how the number of concurrent users affects latency and throughput:



The results show that for a single user, the complete processing pipeline finished within approximately 60 seconds. However, latency increased as the number of concurrent users grew. We observed that at approximately 15 concurrent users, average latency reached around 170 seconds. Beyond this point, latency increased significantly with additional load, for example, with 25 or more concurrent users, request completion times frequently exceeded five minutes. Additionally, we observed that beyond this threshold, the number of successfully completed scenarios no longer increased, and in some cases, it even decreased within a 10-minute time window. This indicates that the optimal operating point for maximizing throughput is around 15 concurrent users. These findings provide us with realistic expectations for the GPU infrastructure and offer guidance for future system scaling and performance optimization.

6.2 Kubernetes and Network Infrastructures

The current state is in line with the original design plans, with a few implementation-level changes.

As for the Kubernetes cluster and the supporting services around it:

1. Virtual machine creation

There are 3 control plane VMs and 3 worker VMs for the Kubernetes cluster distributed across all three physical PVE nodes.

2. Operating system

Originally, we wanted to use Ubuntu; however, eventually we settled on Rocky Linux 10. One of the reasons was a better security model, but also a more stable package ecosystem (Ubuntu has started delivering some server packages as snaps instead of apt/deb).

3. Kubernetes Distribution and Installation

We used RKE2 as planned, using a Ceph cluster for storage and Cilium with eBPF networking (no kube-proxy). Our installation uses native networking without encryption between nodes, as we have the underlying network completely under our control.

4. Argo CD Deployment and GitOps Setup

Argo CD was deployed as soon as possible and the rest of Kubernetes services are deployed using GitOps practices, just like any other deployment on Kubernetes, e.g., applications or midPilot services.

When it comes to network segmentation, the original idea of using separate VLANs was utilized, but the numbering evolved.

- Instead of a `10.10.<VLAN-ID>.x` schema, we adopted the less commonly used range `10.200.x.x`.
- The third octet still matches VLAN-ID wherever applicable. In some cases, e.g., the management network `10.200.10.0/24`, VLAN is not used, as this network is handled by a separate switch.
- As planned, there are separate VLANs for various purposes, such as Hypervisor management (20), Proxmox Sync (21), Ceph public network (80), Ceph internal network (81), monitoring (128), etc. The structure was revised as it was created, as there were also some omissions in the original design. For VMs, we dedicated a complete range of networks starting from 128, i.e., `10.200.128.0/24`. The Kubernetes cluster uses VID 130, legacy VMs migrated from the old infrastructure are separated in VLAN 166, and potentially more sensitive Windows VMs are in VLAN 167, etc.
- The original plan to build monitoring at the secondary site was postponed due to the late delivery of the new servers. This caused delays in decommissioning of our old infrastructure, which in turn blocked preparation of the secondary site using the most capable old servers. Currently, the monitoring infrastructure is run on the primary site and will be migrated to the secondary site once it is ready. The backup server was moved to the secondary site, so backups are already off-site, utilizing Proxmox Backup Server.

6.3 CI/CD Pipeline

During the project, we validated various parts of the CI/CD pipeline, although some work is still required for further productization.

When it comes to artifact registry (repository):

- We stayed with Nexus for Maven and npm packages, more out of continuity and familiarity than satisfaction. We did not prioritize migrating our Nexus repository to a new solution.
- We introduced Harbor as a container image registry, as it provides a convenient UI with access control, which was useful during development. Harbor also helped with image vulnerability scanning, although this can also be part of an image build pipeline; we used both approaches.

Building and deployment pipelines were kept separate, as this better aligned with the responsibilities of various teams. However, where developers were responsible for testing deployments, they were allowed to do so.

To summarize the results related to the original **5.3.3 CI/CD Solution** section:

- **Source repositories**

Status unchanged, we use GitLab internally and GitHub for our public work. Use of GitHub Actions or GitLab CI/CD pipelines is rare and not essential. These are primarily treated as Git repositories; any Git repository would work for us.

- **CI/CD product**

We decided to keep using Jenkins. We successfully evaluated the use of "Declarative Pipeline" syntax instead of the previously used "Scripted Pipeline". We also introduced a common library of pipeline functions, which helped us adhere to the Don't Repeat Yourself (DRY) principle.

- **Artifact repository**

The existing Nexus was complemented with a Harbor registry for container images.

- **Kubernetes deployment**

Kubernetes deployments use a GitOps philosophy, using Argo CD as the main delivery tool. We set up various Argo CD projects and roles, all automated by Ansible, and allowed "shell access" into pods for administrators of the respective projects. This eliminated the need to provide low-level Kubernetes access to individual namespaces. At the same time, developers could trigger restart of deployments or individual pods to pull a new container image.

To summarize GPU workloads (5.3.4 GPU Workloads in the M1 report), we successfully verified that we can fully utilize our new GPUs both in VMs (via PCIe pass-through) and Kubernetes. In Kubernetes, the node itself is a VM that requires GPUs to be passed through using the pass-through mechanism. Then, within Kubernetes, GPU usage required container support and the Kubernetes `nvidia-device-plugin`.

We also learned that for RAM-VRAM throughput scenarios, it is important to pin the relevant VM to the CPU that is physically connected to the GPU. This may be just a minor issue in most cases when inference-oriented workloads fit into VRAM, but it can still significantly speed up the loading of the model from disk/RAM and any future training activities. The CPU pinning for VMs accessing GPUs is handled automatically in our Ansible playbooks.

6.4 Operational Architecture

The original design of the operation architecture was quite ambitious. During the course of the project, we successfully evaluated most of the planned components, but their full productization will require further effort - not only beyond the scope of the project but also involving other people within the organization.

6.4.1 Observability Architecture

Let's review the aspects of the original design and the results achieved:

- **Shared company-wide monitoring and logging stack:**

A centralized observability platform was built using the Kube Prometheus stack, which includes Grafana dashboards, Prometheus for metrics collection, and Alertmanager for alerting; alerts are also sent to our Zabbix monitoring.

When it comes to log collection, centralized log collection from heterogeneous sources (OS/application logs from VMs and bare metal servers, networking logs) is still in a development phase. However, importantly for the midPilot project, container logs are already collected with Grafana Alloy and stored in Loki. These logs are then available in Grafana dashboards where it is very easy to search them or use them for visualization.

- **Namespace-aware data collection and logical isolation and Developer empowerment and collaboration** aspects are naturally provided by the monitoring and logging suite mentioned in the previous bullet.

- **Integration with CI/CD and security tooling:**

Here, the capabilities are available, but we use a more hands-on approach with Dependency-Track and scans in Harbor. CI/CD pipelines are responsible for generating SBOMs; these are then regularly reviewed. Vulnerability scanning and patching is currently a very hot topic in the company, so the process is currently in flux and the "automation tail" will adapt to the process that will be implemented in the future.

- **Alerting and incident escalation:**

Alert notification and escalation is provided primarily by the Zabbix monitoring server. We switched from mail notifications to a dedicated channel in our internal Nextcloud Talk. This is significantly less intrusive and noisy, and at the same time, it provides a much cleaner view of alert history.

6.4.2 Workload Organization and Namespace Strategy

From the start, we used one namespace per deployment unit, e.g., a hosted service (see the component diagram in 5.1). Various services were developed by different teams and had a different update cycles during development. Each deployment is easy to control from Argo CD, and it is also easier to completely recreate the whole deployment after deleting the namespace. In terms of permissions, we did not restrict access to individual namespaces. Similarly, in Argo

CD, there was just one project for the whole midPilot team. Naturally, the infrastructure team will take responsibility for deployments when midPilot goes to production.

6.4.3 API Gateway and Ingress Management

Results in this section lag behind the original expectation for a variety of reasons, including the delayed HW delivery and an unexpected discontinuation of NGINX Ingress Controller as we knew it. This was partially offset by the adoption of [LiteLLM AI Gateway](#) that acted as an OpenAI proxy in front of the actual OpenAI-compatible LLM service (vLLM in our case). LiteLLM provided better control over LLM API usage, with API key management, throttling, etc. However, this component covers only API access to LLM services, not to all other hosted services.

Right now, access to all services goes through a central Kubernetes Ingress controller. We realize that for a true production use, we must seriously consider other Ingress controllers, e.g., Kong, Traefic, or perhaps even Cilium Ingress, to achieve better synergy, as we already use Cilium for networking.

6.4.4 Workload Standards and Best Practices

We made serious progress in this area.

- **Container Images:** We established a process and tooling for scanning container images. This was arguably the most important aspect for midPoint itself, as it is the product delivered to customers and other users. Establishing the process was much more difficult than we had expected, especially beyond the tooling itself. It required some organizational refocusing, and we plan to resolve any remaining friction as part of our continuous improvement effort.
- **Observability Instrumentation:** Essential instrumentation and health checks were implemented, as these are not just best practices, but a necessary requirement for setting up reliable Kubernetes deployments. At the same time, there is room for improvement in collecting not only technical, but also business-oriented metrics in the future.
- **Logging and Log Forwarding:** In Kubernetes deployments, the service logs are processed by Grafana Alloy and stored in Loki. This provides many options for further analysis in Grafana dashboards.
- **Resource Management:** All deployments are now set up according to the internal best practices, which have evolved significantly. In particular, the role of "requests" is now much better understood (especially their role in scheduling). We are currently discussing how to handle CPU limits, as there are various recommendations, including opinionated guides advising setting no limits at all for highly burstable workloads. Even for burstable workloads, we currently use high CPU limits as a safety precaution in case of bugs and runaway processes.

We continually review our best deployment practices when it comes to Kubernetes where we

opted for three main styles:

- Primarily used for 3rd-party Helm charts, we adopted strategy of "wrapper charts", where we create our own minimalistic chart and consume the intended 3rd-party Helm chart as its dependency.

This may seem counter-intuitive at first, but it is actually much simpler and more reliable than any other way of tracking 3rd-party charts, such as copying or forking. Finally, this allows us to add additional Helm templates (such as custom network policies) into our wrapper chart very easily. In some cases, the wrapper chart can evolve into a full-blown chart where the dependency is not the only main point for the deployment. This happened with LiteLLM chart (as a dependency) that was combined with our own vLLM deployment, creating a completely new deployment as a result.

- Our custom Helm charts - we use this for more sophisticated deployments where complex templating is necessary.
- Kustomize deployments - this is what we use by default for its simplicity.

6.5 Security Operations

During the project, our company was certified as ISO 27001 compliant. We knew that security is not a state but a process and this process is often shaped by people, especially in organizations of our size. Security of our core infrastructure was boosted significantly due to its complete overhaul and also thanks to significantly stronger security expertise in both the infrastructure and compliance teams.

Let us quickly review the points from the original design document:

- **Access control** is enforced using Kubernetes RBAC with further restrictions applied through GitOps workflows (e.g., Argo CD) to ensure auditable and declarative changes. Authentication is integrated using our Keycloak instance, which enables centralized identity and access management. This is fully implemented as designed.
- **Secrets management** is implemented using Kubernetes Secrets. Instead of HashiCorp Vault, we started using its fork **OpenBao**, which is fully backed by Linux Foundation's OpenSSF. As our first step, we use OpenBao as our internal certificate authority; in the future, we want to adopt it for application and other secrets.
- **Container security** is ensured as planned: we use minimal base images and combine vulnerability scanning of images using Trivy in Harbor with both SBOM generation and additional security analysis in Dependency-Track.
- **Infrastructure hardening**
 - We introduced restrictions on administrative accounts for the infrastructure.
 - For VMs, we use hardened Rocky Linux 10 base image. Similarly, our bare metal infrastructure with Proxmox VE (Debian-based OS) was reviewed and hardened.

- We enforce Secure Boot for our bare metal and virtual machines, as well as the installed operating systems. Where applicable (Rocky Linux VMs), we use SELinux.
- **Auditability** was experimentally verified, but the full implementation in the organization requires further steps, including the introduction of a SIEM solution.

Our improvements in this area are not directly transferable to another organization, but at the same time, they are not specific to the midPilot project. See chapter 7 for further discussion of the security and privacy concerns related to this project and its use of AI.

6.6 Backup and Disaster Recovery

As planned, we adopted a multi-layered backup and disaster recovery strategy. Redundancy is provided by Ceph, snapshot capabilities of which may prove useful as a "lightweight backup" in case of an unexpected situation. The true backup is handled by Proxmox Backup Server (PBS) for all the VMs. For Kubernetes workloads, we use Velero, which stores its backups in a Minio VM that is then backed up by PBS.

The design was tested repeatedly during the new infrastructure test installations where VMs and Kubernetes applications were restored from PBS or Velero.

Even with reliable and redundant hardware and two sites at different locations, we plan to improve our processes further with practices such as 3-2-1 backups, staggered backups, and tiered retention policies (the GFS scheme).

6.7 GPU Setup Checklist

The list below is a checklist for successful GPU use, from low level HW/UEFI settings to Kubernetes specifics. The purpose of the checklist here is to consolidate the points in one place rather than having them scattered throughout.

- In UEFI, ensure Resizeable BAR (ReBAR) is enabled for optimal performance.
- Do not install NVIDIA drivers on the PVE host, it will only provide raw PCIe device to VMs.
- Disable any other conflicting drivers; e.g., the `nouveau` driver must be blacklisted.
- VFIO must be enabled for the GPU cards (the `vfio-pci` OS module and `vfio.conf`).
- IOMMU is enabled in kernel parameters.
- When creating VM in PVE:
 - In Hardware, Add PCI Device, then choose GPU or GPUs for that VM.
 - For optimal performance in multi-socket servers, pin the VM to the CPU that is directly connected to the GPU. Use `lspci -v -d ...` to see the "NUMA node" for the GPU, then `lscpu | grep node` to see which logical CPUs are on that socket. Use this in PVE, Hardware, Processors to set CPU Affinity, e.g., `32-63,96-127`.

- Install NVIDIA drives as recommended for the operating system used on the VM.
 - It is recommended to install proprietary drivers which may be part of the confusingly named `nvidia-open`. You generally do not want the `cuda-drivers` package; `nvidia-open` will install the right CUDA drivers as well.
 - Be sure to crosscheck the OS documentation for NVIDIA GPU drivers with the NVIDIA installation documentation. If there is a driver procedure recommended by the OS distribution, it is likely preferable.
 - After successful installation, you should see packages `nvidia-driver-cuda` and `kmod-nvidia-open-dkms` on your system.
- Install containerization support for Docker and Kubernetes, the package is typically named `nvidia-container-toolkit`.
- Deploy the [NVIDIA device plugin for Kubernetes](#) to your Kubernetes. We deployed it as a Helm chart with Argo CD.
- Nodes with GPU are labeled `nvidia.com/gpu.present=true`, device plugin exposes the `nvidia.com/gpu` resource.
- Set up your monitoring, e.g., GPU/NVIDIA specific templates for Zabbix or similar for your monitoring solution.

The steps around the driver installation get more complicated if you want to use Secure Boot on the VM, but this is beyond the scope of this checklist.

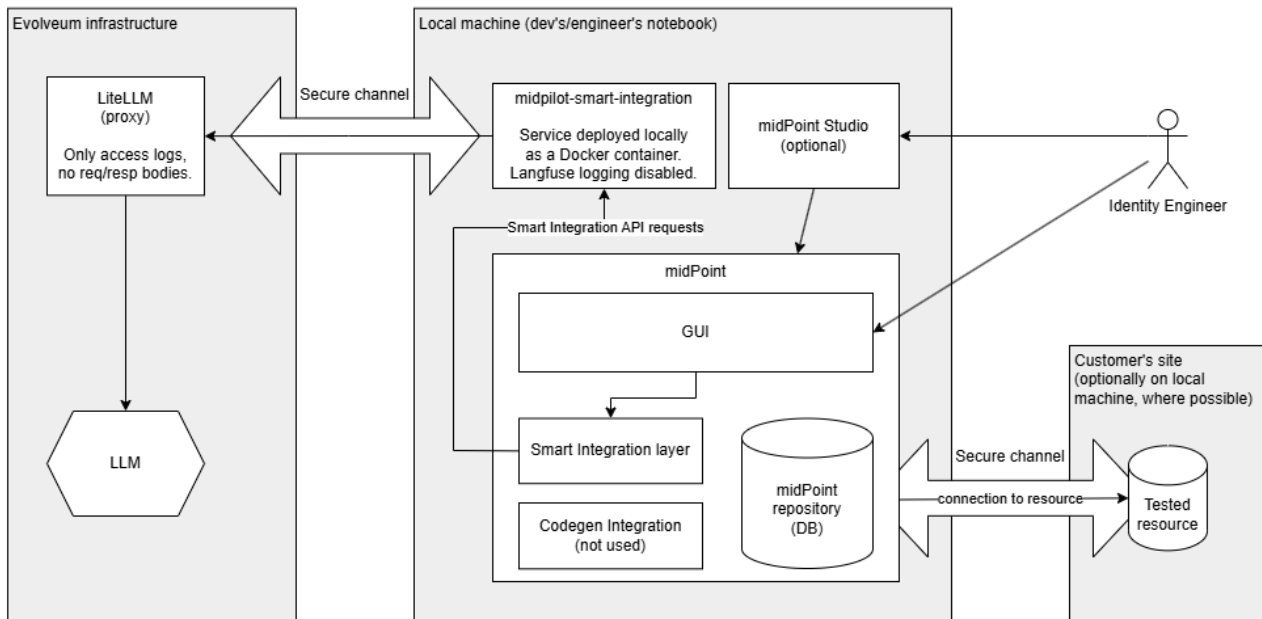
6.8 Deployment Flexibility

Beyond the complete deployment as described in Section 5.1, the components can be used more creatively. Let us consider a scenario where an identity engineer works on a proof of concept (PoC) for a customer who forbids using the publicly available midPilot instance. The customer may have stricter requirements for the use of AI/LLM solutions, request/response logging, etc.

In this scenario, the integrator is allowed to use the smart integration microservices for mapping and delineation suggestions—no other service is necessary. To meet the customer's additional requirements—such as disabling Langfuse tracing and configuring a custom LLM endpoint—the smart integration service is run locally in the same environment as midPoint used for the resource configuration.

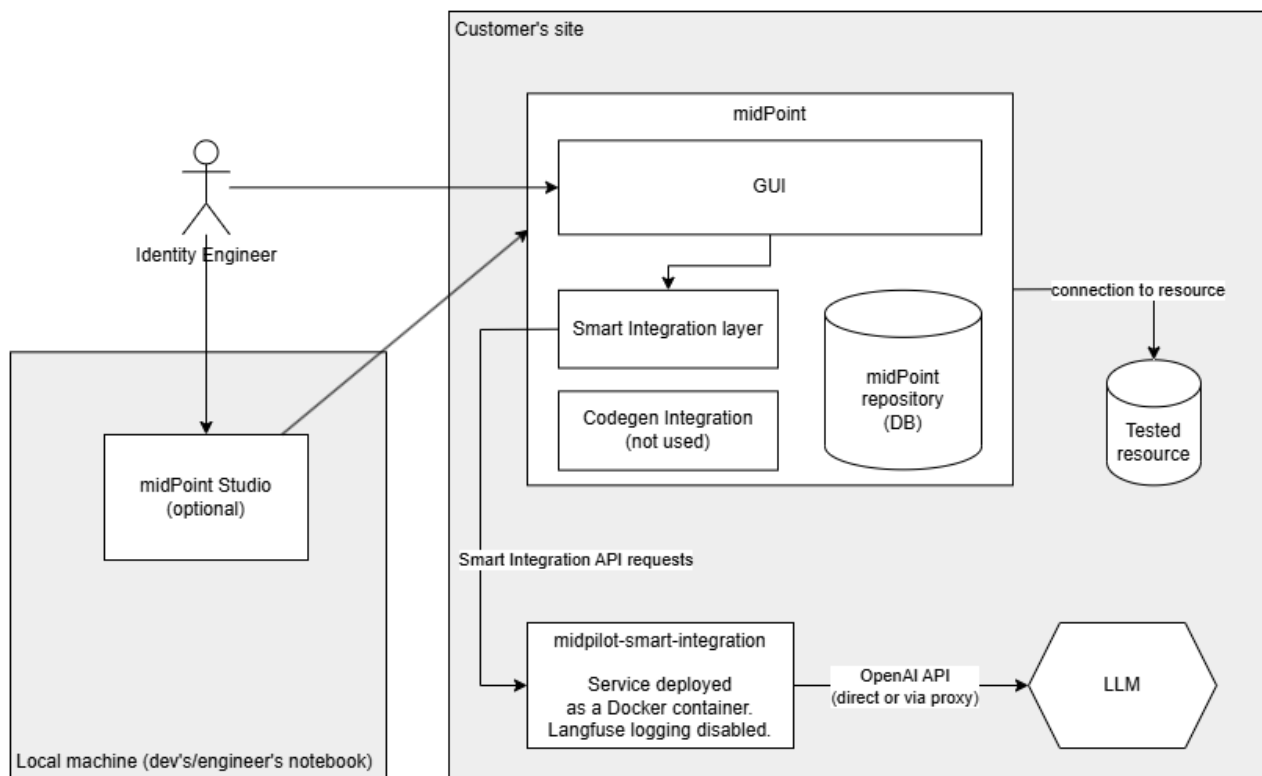
There are two options to meet the customer's requirement.

The first option is that the identity engineer runs midPoint on their computer; the tested target resource is accessed over a secure channel on the customer's site.



In this case, it is allowed to use an LLM service on the integrator's site (Evolveum in this case); however, a separate LLM on a dedicated GPU is required. LiteLLM can be shared or dedicated for this PoC as needed if it meets the minimal logging requirements.

The second option adhering to the customer's requirements shows the same set of components, this time all placed in the customer's environment. This means that the customer has complete control over the data processed by the LLM.



In this case, LiteLLM is not shown in the picture explicitly. For the **midpilot-smart-integration**

service, it does not matter if it accesses the LLM directly or via a proxy, such as LiteLLM. An OpenAI-compatible API is used in both cases.

In both options, the key enabler is that the service can be deployed in Kubernetes or using Docker Compose. For quick reconfiguration of services run locally, Docker Compose is more approachable and avoids any inter-team friction.

That said, if the customer insisted, the smart integration environment (the second scenario) could be deployed in Kubernetes as well.

7. Compliance, Security, and Governance

This chapter covers the applied governance approach, regulatory considerations, and implemented security and privacy measures. The measures considered and applied during the project are described in this chapter. Also, potential measures that are beyond the scope of this project are described, as a form of a follow-up plan.

The results of the midPilot project are a part of the midPoint ecosystem. MidPoint, being an identity and access management (IAM) platform, is classified as *important product* under the Cyber Resilience Act (CRA, Regulation (EU) 2024/2847). This classification implies elevated cybersecurity requirements, given both by the legislative frameworks and market expectations. Considering the ever-evolving requirements, the midPilot project has adopted a responsible approach to compliance and cybersecurity, going above and beyond mandatory requirements and current expectations.

7.1 Cybersecurity Considerations

Development of midPilot was an integral part of the usual Evolveum software development and quality assurance processes, which are covered by ISO/EIC 27001 certification. Moreover, midPilot is built on top of the midPoint platform, so the cybersecurity requirements of midPoint development were consistently applied to midPilot development as well. Therefore, midPilot components that are integrated into midPoint re-used existing cybersecurity mechanisms of the midPoint platform (authentication, authorization, access control).

The *Secure by design* principle was applied during the project. Application of this principle led to re-considerations in early stages of some project components, leading to a final design which is aligned with the cybersecurity principles and requirements.

Secure by default principle was also considered. Application of this principle led to realization that the default midPoint scripting language (Groovy) poses a significant risk, especially when combined with AI features. Therefore, development of a new secure scripting language (MidPoint Expression Language, MEL) was conducted in parallel with the midPilot project. MEL was not a standalone midPilot component, but a related midPoint platform hardening activity. It was motivated in part by the safety requirements exposed by midPilot and provides significant advantages for future midPilot development.

Cybersecurity aspects of the external AI-assisted midPilot services (*midpilot-smart-integration* and *midpilot-connector-gen*), as well as the integration catalog services, were considered as part of the midPilot project. The use of remote network services provided across organizational boundaries, as well as the use of AI techniques, posed new challenges for the team. The team focused primarily on the functional elements of the solution in the first half of the project. The focus shifted toward functional, practical, and secure design of the services in the second half of the project. However, the production-ready implementation of these network services is beyond the scope of the midPilot project. Proper cybersecurity mechanisms for these services are a

challenging matter on their own, especially considering the ever-increasing regulatory requirements on network services. While functional aspects of the services were addressed as part of the midPilot project, proper production-ready implementation of network service cybersecurity will be covered in a follow-up project (see [Section 7.6](#)).

7.2 AI Governance

The project applied a pragmatic AI-governance approach centered on bounded use of AI assistance, human review, and platform-side validation. The delivered solution does not position the language model as an autonomous decision-maker. Instead, it uses AI to generate suggestions and draft mappings, delineation filters, schema matches, and connector artifacts that are then reviewed and validated in midPoint-managed workflows.

This architecture aligns with the core AI governance principles later formalized in the ISO/IEC 42001-style AI management thinking:

- **Clearly defined purpose and bounded scope of AI use:** AI is used exclusively for clearly defined and bounded tasks (mapping suggestions, correlation suggestions, parts of connector configuration, etc.) AI mechanisms are never used outside specific scope. AI-processed data are never shared or stored persistently (except for observability reasons). The data are not used for training of AI models. Moreover, the use of AI mechanisms is completely optional.
- **Human oversight:** Every AI output is considered to be a suggestion, not a decision. The system never makes any decisions or autonomous actions based on AI outputs. Every suggestion needs to be reviewed by a human operator before it is used. AI suggestions are scoped to reduce their volume to a manageable size and avoid information overload for the human operator.
- **Risk reduction through validation and staged adoption of generated outputs:** The AI models work with structured data. Structured outputs of the AI models are validated before they are presented to the user, eliminating any obvious hallucinations. The outputs are presented to the user for review (clearly marked as AI outputs). The AI suggestions, once reviewed, are ready for immediate testing and feedback loops before they are applied to the production environment.

Overall, midPilot has adopted a *low code / no code* approach for code generation. The AI models generate configurations rather than code (*no code*), if possible. If code is necessary, only short snippets suitable for easy human review are generated (low code). Moreover, secure MidPoint Expression Language (MEL) was tested during the project as a replacement for potentially dangerous Groovy code.

- **Traceability and transparency of technical behavior:** AI transparency is implemented on several levels. Internally, appropriate observability tools are used to monitor the behavior of the AI models. From the user perspective, AI outputs are always clearly marked and presented for review. Technical aspects of AI use are fully documented and the

documentation is publicly available. However, the ultimate transparency mechanism is the open source nature of the project. The code of all the services, tools, and relevant mechanisms is publicly available under an open source license for any interested party to review.

The project therefore implemented governance primarily through the system design and workflow control, not through a separate heavyweight governance subsystem.

The governance approach is also consistent with the trustworthy AI criteria used in the Slovak AI Awards 2026 framing, which emphasize human oversight autonomy, accountability, transparency and traceability, reliability and safety, fairness, societal and environmental impact, and privacy and data sensitivity. These criteria are useful for interpreting the delivered design because midPilot is not a conversational assistant aimed at end users; it is an AI-assisted engineering and administration tool for identity integration. The main governance risk is therefore not direct manipulation of end-user decisions, but incorrect or over-trusted technical configuration output. The project mitigates this by splitting the work into small reviewable steps, keeping midPoint as the controlling system and requiring human confirmation before generated artifacts or recommendations are accepted into the configuration.

7.3 EU AI Act Alignment and Obligations

Regulation (EU) 2024/1689 (the "AI Act") establishes a harmonized, risk-based framework governing AI systems placed on the market or put into service in the EU. The obligations arising under the AI Act are determined by the risk classification of the AI system in question. The classification is based on a self-assessment carried out by the provider. Evolveum has conducted a self-assessment of midPilot using the official EU AI Act Compliance Checker tool. The assessment was supported by a legal analysis of the impact of the AI Act on the provided system with particular regard to Article 6 and Annex III categories and the relevant definitions under the AI Act. MidPilot qualifies as an AI system and is classified as a limited-risk system, because it does not fall within any of the high-risk categories under Article 6(1) or Annex III of the AI Act. Moreover, midPilot does not constitute a prohibited AI practice under Article 5.

MidPilot is poised to satisfy the following obligations of AI Act:

- **Intended Purpose** (Article 3(12)): The purpose of midPilot functionality is stated in several documents, presentations, and marketing materials. The purpose is properly scoped to integration capabilities.

As midPilot matures from an R&D project to a production-ready system, the intended purpose needs to be refined and formally defined.

- **AI Literacy** (Article 4): The midPilot development team is highly skilled in AI technologies, which was a prerequisite for the development of midPilot functionality in the first place.

As midPilot matures from an R&D project to a native part of midPoint product, Evolveum

plans to roll out a company-wide AI literacy training program.

- **Transparency** (Article 50(1,2)): All AI interactions are clearly identified and all AI results are properly marked.

The final review of midPilot functionality by the compliance team uncovered some minor issues in the human review user experience (see [Section 7.6](#)). These issues will be resolved before midPilot is deployed for production use.

Even though midPilot is classified as a limited-risk system, it goes above and beyond regarding the fair use of AI. MidPilot voluntarily fulfils (at least partially) the following obligations on top of the mandatory minimum set by the AI Act:

- **Risk Management System** (Article 9): Development, deployment, and extension of midPilot functionality is under the oversight of Evolveum's dedicated compliance team. Members of the compliance team are not part of the midPilot day-to-day development to ensure independent review.
- **Data Governance** (Article 10): Validation and testing datasets were constructed to be relevant and representative. The presence of biases that could affect fundamental rights was considered as well.
- **Technical Documentation** (Article 11): Complete technical documentation of the system and all the components is publicly available, as is all the source code.
- **Automatic Logging** (Article 12): The performance of AI models is automatically monitored by an observability platform.
- **Human Oversight** (Article 14): MidPilot never makes any autonomous decisions. Outputs of the AI mechanisms are presented as suggestions. Human oversight is required at every step where AI-generated outputs are used.
- **Cybersecurity** (Article 15): Cybersecurity concerns were considered during the midPilot design, implementation, and testing. Several cybersecurity concerns were identified during the development. These findings initiated associated development activities to mitigate the risks (such as development of the secure MidPoint Expression Language).

Overall, midPilot is designed, implemented, and tested with the AI Act compliance in mind. It is well prepared for full AI Act compliance once it matures from an R&D project into a full deployment readiness.

7.4 Personal Data Protection

MidPoint is an identity and access management (IAM) platform, therefore it works with personal data by its very nature. Historically, midPoint was designed for on-premise deployments, keeping all personal data in control of the original data controller. However, the introduction of external AI-assisted midPilot services changes that concept for selected workflows. Resource-configuration assistance can use representative data samples and schema

details, while connector generation workflows can use application documentation and connector-related metadata. Such data are passed from midPoint to the relevant external AI-assisted service for processing. This situation was addressed by employing both technological and organizational measures.

There are several technological measures addressing the additional risks introduced by the use of remote services and AI:

- **Data use minimization:** MidPilot is designed to keep the data on-premise (in the midPoint repository) whenever possible, sending data to the external AI-assisted service components only when necessary. It also keeps the amount of sent data to a necessary minimum. For smart integration tasks such as mapping and correlation assistance, this means local processing of schema metadata, documentation, statistics, and heuristics whenever possible, and treating any use of representative personal data as a controlled, explicit, and minimized step. Only selected statistics and data samples are processed by the services. Complete data sets are never shared; they never leave the midPoint repository.
- **Stateless processing:** The midPilot design treats external service calls as bounded ephemeral computations. This means that each external AI-assisted service processes the data only for a single particular purpose, not retaining the data beyond the scope necessitated by the particular operation. The services do not use any long-term storage for the data; the data are processed only within a single interaction and then discarded. Relevant parts of the data may be retained for a limited period for observability and cybersecurity reasons (e.g., attack detection and response).
- **Failsafe constraints:** MidPilot has design elements in place that completely prohibit the processing of certain data items by the smart integration service and AI models. Special schema definitions can be used to mark sensitive data items, such as special categories of personal data. Such marking prohibits any use of such data by the AI mechanisms.
Note: this functionality is designed, but not yet implemented in the R&D stage of the project.

From the organizational perspective, there are (at least) two deployment options that affect personal data protection:

- **Use of Evolveum-provided AI-assisted midPilot services:** Evolveum provides the AI-assisted midPilot services for midPoint users, including the smart integration and connector generation services and the AI models. The services run on hardware owned by Evolveum, located in the EU, governed solely by the EU legislation. Evolveum becomes a personal data processor under the GDPR regime.
- **Use locally-installed AI-assisted midPilot services:** The code of AI-assisted midPilot services is released under the terms of an open source license (EUPL). Therefore, any interested party can install the services on their own hardware under their own control. Such locally-hosted AI-assisted midPilot services provide the greatest degree of control over personal data. No personal data are shared with any 3rd party in this case.

7.5 Implemented Security, Privacy, and Fairness Measures

This section lists concrete measures implemented in the delivered solution.

The AI-related security and governance measures include:

- Human review before generated outputs become accepted configuration.
- Clear marking of AI interactions and AI-generated output.
- Bounded service interfaces that keep midPoint in control of the overall workflow.
- Traceability through versioned source code, logged service execution, and observable AI-service traces.
- Small-step workflows that let users inspect, reject, modify, or regenerate suggestions before accepting them.
- Retry with structured validation feedback instead of blind acceptance of failed generation.
- Script and filter validation inside midPoint before wider use.
- A low code / no code connector framework and AI-assisted generator relying on configuration and a small amount of code which can be easily and reliably inspected by a human reviewer.

These measures do not eliminate all security and privacy concerns, but they significantly reduce practical risk compared with direct application of raw AI outputs.

From the perspective of a trustworthy AI, the strongest implemented safeguards are the human oversight, traceability, and technical validation. Users are informed throughout the workflow that the system provides AI-assisted suggestions rather than final autonomous decisions. The generated code and configuration remain subject to review, validation, testing, and usual midPoint configuration control conducted by the human operator. This reduces the risk of an automation bias, hallucinated configuration, and unsafe over-reliance on the model output.

The general cybersecurity measures include:

- Externalization of runtime secrets and service endpoints through environment variables and secret files.
- Docker-secret-based handling of selected external credentials in the integration catalog.

The governance and sovereignty measures include:

- Lifecycle state tracking for uploaded and published artifacts in the integration catalog.
- Support for local or deployment-specific model and service configuration instead of one mandatory 3rd-party-hosted AI provider.

- The system supports fairness and accessibility indirectly by reducing the specialist effort required for identity integration.

The environmental impact of the AI features was considered primarily through pragmatic model and prompt selection. The project evaluated different model sizes and trade-offs; the final architecture keeps AI calls bounded to specific tasks instead of using unconstrained long-running autonomous agents. Future work should continue to prefer smaller or locally deployable models for tasks where they provide sufficient quality, i.e., especially for repeatable engineering workflows.

Overall, the implemented cybersecurity, data protection, privacy, and fairness measures are adequate for the purposes of the midPilot R&D project, as was attested by the review of Evolveum's compliance team. Further improvements and extensions need to be implemented before the midPilot functionality is used for production purposes (see [Section 7.6](#)). However, the results of the midPilot project R&D phase provide an excellent potential for rapid midPilot functionality development and its further incorporation into midPoint.

7.6 Risks, Limitations, and Remaining Open Issues

The cybersecurity and compliance landscape is evolving at an unprecedented rate. Cybersecurity requirements and regulatory obligations have significantly evolved and intensified since the time the midPilot project plan was initially drafted. Even though the project team made an extraordinary effort to catch up with the evolving landscape, some of the cybersecurity, compliance, and governance issues remain open even after the closure of the midPilot R&D project. These issues need to be fully addressed to make the midPilot functionality production-ready.

The following issues were identified:

- **Production-level governance:** The project did not produce a complete threat model or a fully-featured governance framework for production deployments. The operational controls—such as production retention policies, final audit policy, deployment-specific DPIA-style assessment, and broader organizational governance—still depend on the target deployment environment.
- **Residual AI risks:** There is a residual risk typical for AI-assisted engineering systems: incorrect, incomplete, or misleading suggestions can still be produced. The delivered design reduces this risk through validation and review, but it does not remove the need for expert oversight. Future development work should therefore continue to strengthen the evidence evaluation, auditability, deployment guidance, and organization-level governance controls.

The project also identified a data-related limitation: broader access to diverse real-world configuration examples would improve evaluation and tuning of recommendations, but such data are often sensitive and difficult to obtain and use in a safe manner. This creates a practical tension between improving AI quality and respecting privacy, confidentiality, and

customer control. Future work should address this through better anonymization, synthetic test data, opt-in data collection, and independent review of the AI governance and regulatory alignment.

- **Risk of general-purpose scripting:** MidPoint traditionally supports Groovy as its default scripting language. The use of a powerful general-purpose scripting language poses a significant security risk when AI-generated outputs are validated, reviewed, or later applied in configuration. A safe expression language (MidPoint Expression Language, MEL) was developed in parallel with the midPilot project as a related platform activity. The delivered mapping-generation path now generates MEL rather than Groovy, which removes the specific risk of AI-generated Groovy mapping scripts in that path. However, production use still needs to keep generated expressions constrained to MEL or another safe expression mechanism, and avoid fallback paths that would reintroduce general-purpose scripting into AI-generated outputs.
- **Security of network services** (external AI-assisted services and integration catalog): Network services are not properly secured for production use of the system. Proper authentication and authorization mechanisms are missing. These mechanisms must be implemented before the services can be put into production use. An API gateway is recommended for the external AI-assisted services, use of an OAuth-based mechanism is recommended for integration catalog. The deployment of these components is planned.
- **Integration catalog processes and policies:** The work on lifecycle management, review procedures, and policies for the integration catalog is not finished. As the integration catalog is a content provider platform, compliance with the Digital services act (DSA, Regulation (EU) 2022/2065) needs to be considered.
- **AI Act compliance:** Compliance with the AI Act (Regulation (EU) 2024/1689) needs to be formally evaluated, considering all the production-level improvements and extensions of the midPilot project. The preparatory work for the evaluation includes a formal definition of the *intended purpose* of the AI system (AI Act, Article 3(12)).

Further compliance gaps were identified in the delivered midPilot code:

- The midPoint graphical user interface (GUI) does not show the generated mapping expression when presenting suggested mappings for human review. Even though the GUI correctly identifies a mapping as AI-generated, the generated mapping logic is not shown. This could potentially lead to the reviewer overlooking problematic generated logic.
- The smart integration service may evaluate (i.e., execute) a generated MEL mapping expression when validating the LLM output. This is no longer a Groovy-specific risk in the delivered mapping-generation path. However, validation still happens before human review, so the production implementation must keep this path constrained to MEL and covered by appropriate safety controls.
- The smart integration service may evaluate (i.e., execute) a correlation filter when validating the LLM output. If such a filter contains or references an expression language with broader execution capability, validation can be dangerous. Production handling should therefore

keep this path constrained to MEL or another safe expression mechanism.

- Functionality that prohibits use of sensitive data (such as special categories of personal data) by the AI services is planned, but it is not implemented yet.

Moreover, the following suggestion could provide benefits for further development of midPilot functionality:

- The solution would benefit from ISO/IEC 42001 compliance evaluation, once it gets closer to the production-ready stage.

8. Open-Source Outputs and Project Deliverables

This chapter summarizes the concrete outputs delivered by the project and their publication status. It focuses on the produced source code, documentation, deployment artifacts, licensing approach, and contributions integrated back into the MidPoint ecosystem. The goal is not to duplicate full user or developer documentation, but to identify what was delivered, where it belongs, and how it supports continued reuse and development after the project ends.

8.1 Delivered Artifacts Overview

The project delivered a complete open-source solution set for AI-assisted application onboarding into the midPoint ecosystem. The main outputs are:

- The `midpilot-smart-integration` microservice exposing AI-assisted operations for resource configuration tasks
- The `midpilot-connector-gen` microservice for connector generation
- The integration catalog application
- Project-specific implementation work integrated into midPoint
- Architecture, design, setup, testing, and project documentation

Taken together, these outputs satisfy the project objective of moving beyond isolated research prototypes and publishing reusable technical artifacts together with the materials needed to understand, build, and operate them.

8.2 Source Code and Repositories

The relevant source-code outputs are distributed across multiple public repositories.

- The `midpilot-smart-integration` repository contains the FastAPI-based microservice for schema matching, mapping suggestion, correlation-related matching support, object type suggestion, and focus type suggestion.
- The `midpilot-connector-gen` repository contains the session-oriented connector generation pipeline for target system documentation discovery, scraping, and digestion, as well as connector artifact generation logic.
- The `integration-catalog` repository contains the Spring Boot backend, Angular frontend, PostgreSQL-backed domain model, and upload, download, and publication workflows for reusable integration artifacts.
- The `midpoint` repository contains the main orchestration and integration work delivered by the project, especially in the `smart-impl` and connector development areas. This includes

service orchestration, validation, heuristics, statistics handling, activity handlers, and persistence of connector development state.

- The `connector-scimrest` repository contains framework to simplify building customized connectors using the declarative approach, a set of pre-built components and strategies, with option to customize behavior using Java or Groovy code.

Together, these repositories represent the principal open-source software outputs of the project.

8.3 Architecture, Developer, and User Documentation

Delivering working code is only half the objective; ensuring that the community can understand, deploy, and extend that code is equally important. To support this, the project produced documentation across multiple layers:

- **Project-level documentation:** Includes milestone reports and the final architecture and design documentation you are just reading.
- **Repository-Level Documentation:** Provides developer-facing materials such as README files, interactive API endpoint documentation, local setup guides, environment configuration, Docker-based startup instructions, migration handling, and internal project design and testing notes.
- **Public documentation portal:** Public-facing architecture, developer, operations, and configuration documentation is published through the [official midPoint documentation portal](#).
- **"Try me" guides:** Includes guides and links to public demo projects suitable for live demonstration of project results:
 - [Trying Out the MidPilot Connector Generator](#)
 - [Try Me \(Workshop\): AI Assisted Integration of First Systems with MidPoint](#).

The developer-facing documentation is delivered especially for local setup, environment configuration, Docker-based startup, migration handling, and API usage of the microservices. The user- and workflow-facing documentation was developed in parallel through testing scenarios, design notes, and onboarding flow descriptions used to validate the delivered solution. This also includes the comprehensive UI/UX documentation in the form of [high-fidelity Figma prototypes](#). These designs map out the complete user journeys for the new application onboarding workflows, object dashboards, and the AI-assisted connector generation, serving as the visual blueprint for the developer handoff and future frontend implementation.

This means the project did not publish the code alone. It also produced the surrounding materials required for understanding the intended workflows and continuation of the development after the project closes.

8.4 DevOps and Deployment Artifacts

With the solution split into relatively loosely coupled services and applications, it seemed logical to keep some deployment supporting artifacts together with these components. This is why code repositories mentioned in the section 8.2 typically contains not only code, but also:

- `Dockerfile` for building the Docker container image
- `docker-compose.yaml` describing the service deployment including other container dependencies, such as database, etc.
- SQL files for DB schema and initialization data (where applicable)
- Environment and configuration examples, such as `.env-example` files

Deployment-related outputs delivered by the project include:

- `Dockerfile` for building docker container images and `docker-compose.yaml` for Docker Compose deployments. These are available for all main three service components, `midpilot-smart-integration` (project root), `midpilot-connector-gen` (project root), and `integration-catalog` (in subdirectory `docker`).
- JSON with OpenAPI specification is available under `docs/openapi.json` for `midpilot-smart-integration` and `midpilot-connector-gen`. API in the integration catalog application is intended for Angular frontend only, not for public use.
- For Kubernetes deployment, we provide out-of-the-box runnable examples in the public repository `midpilot-deployment-examples`. Examples include deployment of the smart integration service, the connector generator service, the integration catalog application, and the LLM service with LiteLLM AI Gateway.

These examples are tested with `minikube` for maximum portability and convenience. Their adoption for production naturally requires further steps:

- Setting proper `storageClassName`, `ingressClassName`, and other specifics, depending on the actual storage solution and used Ingress.
- Network policies are not provided, as these are also environment-specific. E.g., if you use Cilium for networking, you may prefer `CiliumNetworkPolicies` instead of traditional `NetworkPolicies`. Network policies need to correspond with your database setup and backup solution. Perhaps you use a central database rather than separate ones for each service, as with Integration Catalog and LiteLLM in our example. Or you use Postgres Operator for database backups and the network policy has to include an egress to a storage, such as MinIO. There are many possibilities, and it is not possible to capture them all in the examples.
- Naturally, vLLM expects available GPU, which is not the default with minikube. If you use a CPU instead of a GPU, the performance will be unusable. That's why the examples are not meant as a cohesive production setup.

Deploying midPilot services and backend applications in Kubernetes requires some level of proficiency. The service documentation provided is then sufficient to configure each service to use the correct LLM deployment.

- For midPoint itself, we already have [midpoint-kubernetes](#) project on GitHub.

These artifacts provide a practical baseline for deployment, developer onboarding, and continued hardening of the solution.

8.5 Licensing and Publication

The delivered outputs are published through publicly accessible source code repositories. To ensure the software remains open, secure, and freely available to the community, the project repositories are licensed under the European Union Public Licence (EUPL 1.2).

We specifically chose the EUPL 1.2 because it aligns perfectly with the core principles of "Public Money, Public Code." Because this project was supported by the public funding, it is crucial to adopt a legal framework that guarantees the people's right to use, modify, and distribute the results. The EUPL 1.2 provides "copyleft" protection ensuring that any future modifications remain open while offering high legal certainty within the European Union and broad compatibility with other major open-source licenses. This licensing choice is entirely consistent with the wider midPoint ecosystem that has also adopted the EUPL 1.2.

Consequently, the AI microservices and the integration catalog explicitly present themselves as open-source project outputs, and the work integrated directly into midPoint is published through the main upstream open-source codebase.

Publication therefore covers not just the source code, but also the surrounding technical materials, documentation, and setup guidance needed for immediate reuse. Please note that while the source code is governed by the EUPL 1.2, our documentation is published under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) license. This comprehensive transparency is a significant part of the final deliverable set, as our project commitment was never just to develop a private solution, but to actively document, publish, and share our results with the global identity governance community.

8.6 Upstream Contributions

The most significant upstream-facing contribution of this project is the direct integration of our work into the core midPoint product itself. Rather than isolating orchestration, validation, heuristics, and connector development handling into a private, temporary wrapper, the project implemented these capabilities natively within the midPoint codebase.

Equally important are the upstream contributions of the project to the broader identity connector ecosystem, particularly the ConnId framework. To support AI-assisted and low-code

connector generation, the project extended the core functionality of the ConnId framework by introducing more detailed schema operations, support for complex attribute deltas, and the foundational architectural changes necessary for automated connector materialization.

Alongside these core framework updates, the project delivered a new declarative development framework (utilized in connector-scimrest) that provides reusable building blocks. This allows both human developers and AI systems to construct connector functionality without reimplementing common logic on a per-connector basis. These technical advancements are tied together by the newly established guided methodology for connector development, which focuses on providing immediate reviewable value during the initial stages of integration.

Furthermore, the project intentionally structured its AI microservices and the integration catalog as openly published reusable components rather than closed and internal-only tooling. This strategic decision establishes a sustainable reusable AI ecosystem around midPoint, ensuring that the features developed under this grant can be seamlessly upstreamed, packaged with midPoint, and maintained by the community long after the formal closure of the project.

9. Project Evaluation, Lessons Learned, and Future Work

9.1 Evaluation Against Project Goals

The project met its main technical goal of making application onboarding into the midPoint ecosystem faster, more structured, and less dependent on fully manual work. This was achieved not by one isolated prototype, but by a delivered combination of midPoint integration work, dedicated AI microservices, connector generation workflow support, and publication infrastructure.

During the development, the project converged toward pragmatic, review-oriented onboarding support instead of a broader, fully autonomous AI automation vision. The reasoning was straightforward: in identity governance and administration (IGA), AI hallucinations or incorrect configurations can lead to catastrophic security risks. Because user trust is paramount, we recognized that fully uncontrolled AI generation is unacceptable for identity systems. As a result, "human-in-the-loop" became the core product principle. The system accelerates the process by suggesting options, mappings, and schemas, but the expert administrator always remains in control.

The project outputs are fully aligned with our commitment to open-source publication. All deliverables were implemented in public Evolveum GitHub repositories. This includes upstream contributions to the main midPoint repository, the newly introduced integration catalog, the underlying SCIM/REST connector framework, validation tools, and the two dedicated AI microservice repositories. Accompanied by technical documentation, setup materials, and operational guidance, these public releases ensure that the broader community can immediately access, verify, and benefit from the outcomes of the project.

9.2 Main Technical Outcomes and Their Practical Value

The true measure of this project is not just the underlying code, but the practical value it delivers to identity engineers and administrators. The core achievement is drastically reducing the manual effort required for application onboarding, while keeping experts firmly in control of the final configuration.

To achieve this, the project successfully delivered four major technical outcomes:

- A dedicated smart integration microservice: This service handles schema matching, data mapping, and correlation support alongside suggesting object and focus types.
- A connector generation microservice: Features session-based discovery, documentation scraping, digestion, and artifact generation.

- Native midPoint workflow integration: We built substantial orchestration, validation, persistence, and UI integration directly into midPoint for the smart-impl and connector development areas.
- The integration catalog: We delivered a dedicated infrastructure for the publication, discovery, and reuse of application integration artifacts. This shifts the ecosystem away from isolated, one-off connector builds toward a collaborative model where configurations can be easily shared, versioned, and reused by the community.

The practical value of the solution lies in reducing manual onboarding effort while keeping experts in control. Instead of replacing the human judgment, the delivered system accelerates documentation analysis, schema preparation, mapping creation, correlator drafting, and object type setup inside structured midPoint workflows.

9.3 Community Presentation and Validation

9.3.1 MidPoint Community Meetup

The delivered solution was demonstrated and explored in practice with the midPoint community at the [2nd Annual MidPoint Community Meetup](#), MCM 2026, held in Prague on May 12-15, 2026. Nearly 200 experts from around the world attended, engaging with midPilot through hands-on workshops and technical sessions.

The public agenda included both presentation-oriented sessions and dedicated hands-on workshops directly related to the midPilot solution scope. The most important validation value came from the workshops because participants could work with the ideas and workflows in a practical setting instead of only seeing slide-show overviews.

The [AI Assisted Integration of First Systems with MidPoint](#) workshop focused on practical onboarding activities: integrating first systems with midPoint and working with data, mappings, correlations, and AI-assisted setup. The [Connector Development Methodology](#) workshop focused on the new AI-assisted framework for faster REST-like connector development with reduced boilerplate. In both cases, the workshop format gave users, partners, and practitioners a chance to try the concepts, ask implementation-level questions, and provide feedback based on their hands-on interaction.

The solution was also visible in the broader MCM program through presentation and roadmap content, including the preview of midPoint 4.11 and its AI application onboarding assistant. This helped position the project in the future midPoint product direction, while the workshops provided the more concrete validation of usability and practical relevance.

These workshops provided an important external validation point for the project. They moved the solution beyond internal implementation and testing by letting real users, partners, and practitioners from the midPoint ecosystem interact with the main workflow ideas directly.

Participants Polls

During the MCM 2026, we asked participants about their opinion on midPoint features being developed. The survey was done via Slido. We collected 19 responses. The most interesting results are summarized in the tables below.

Select two planned new 4.11 features that excite you the most.

Feature	Responses
Smart connector wizard	42%
Easy manual resources integration	37%
Integration catalog	32%
Better scripting APIs	26%
Safe expression language (MEL)	26%
Smart resource wizard	26%
Other features	11%

For what kinds of resources do you expect to use the smart connector wizard the most?

Kind of resource	Responses
REST	83%
Database	33%
SCIM	22%
Other	17%

What would be the preferred way of using AI services for you?

Usage	Responses
Self-hosted solution (own hardware)	63%
Public AI services like OpenRouter	16%
Evolveum AI services (when available)	11%
Existing cloud services, if the midPoint is already deployed in the cloud	11%

How do you like the idea of new application/resource wizards?

NOTE | These are described in [Section 9.5.2](#).

Answer	Responses
I like it very much, looking forward to it!	58%
I am not persuaded. Maybe it will be good, maybe not.	26%
I'm not a fan of that idea; I'd rather you focus your effort elsewhere.	16%

Although the sample is relatively small, the responses suggest strong interest in features that simplify and accelerate application onboarding. Interest was particularly high in the Smart connector wizard and related onboarding capabilities; this is consistent with the project's focus on faster integration, AI-assisted configuration, and improved wizard-driven workflows. Preferences for self-hosted AI also indicate that deployment control and openness are important for this audience, which fits well with midPoint's open-source character.

9.3.2 Identity at the Center

The core value proposition of the solution (solving the notoriously slow application onboarding process) was highlighted to the global IAM community on the popular Identity at the Center (IDAC) podcast. In the [podcast episode](#), we discussed why identity projects often fail due to onboarding bottlenecks and how Evolveum's new AI-assisted capabilities actively address this challenge. Presenting our approach on an industry-leading platform helped position the outcomes of the project directly within the broader conversation about the future of identity governance and administration (IGA). Within less than 30 days, the episode achieved more than 11 000 audio downloads, indicating strong interest in this topic within the identity management and governance community.

9.3.3 AI Awards

Our commitment to building secure, accountable, and privacy-respecting AI workflows has been recognized at a national level. Evolveum was selected as one of only four finalists for the [AI Awards 2026](#) in the category of "Trustworthy AI". This nomination is a massive external validation of our "human-in-the-loop" design philosophy and our strict approach to data privacy. Even though the midPilot project has not been selected as an [AI Awards 2026 winner](#), simply being shortlisted proves that our pragmatic, security-first implementation of AI in identity governance sets a high standard for the industry.

9.3.4 Gartner IAM Summit 2026

With the goal of expanding awareness of midPoint and midPilot beyond our standard reach, we attended the [Gartner IAM Summit 2026](#) in London, UK, on March 9–10, 2026. Our presence at this prestigious conference was supported by a PR Newswire campaign launched beforehand, which generated 73,200 views primarily across Europe and the USA, with additional reach in other regions worldwide. At the conference, the Evolveum team promoted midPilot and midPoint both at the booth and during the theater session The Hidden Cost of Slow Application Onboarding in IGA, presented by Evolveum's Chief Product Officer Pavol Mederly. The team

generated 65 leads at the booth and another 60 during the session, successfully meeting the event's objectives.

9.4 Lessons Learned

The most important lesson is that AI assistance is most useful when combined with strong platform-side control. The final project outcome is stronger because midPoint performs validation, retry, heuristics, scoring, and persistence around the raw AI output.

Another lesson is that practical onboarding workflows benefit more from bounded recommendation services than from overly broad one-shot generation. Breaking the solution into focused modules and reviewable steps proved more robust and more explainable than trying to automate everything in a single pass.

The project also showed that publication and reuse infrastructure matter. A connector or configuration suggestion becomes much more valuable when it can be published, versioned, downloaded, and reused in a catalog instead of remaining a one-off local result.

During development, integration, and validation of the delivered components, the project confirmed that proofs of concept with LLMs are relatively easy, but reliable production workflows are much harder. In identity systems, a confident but wrong answer can create security and operational risk, so the central engineering question is not whether an LLM can produce fluent output, but whether that output can be trusted, validated, reviewed, and operated safely.

The resulting practical lessons are:

- AI should be used for drafts, not final decisions. Generated connector code, schema matches, mappings, and correlation suggestions are valuable first drafts. However, engineers and administrators must remain accountable for the final configuration.
- Fluency is not accuracy. Polished explanations can hide uncertainty, so generated attributes, endpoints, operations, mappings, and filters must be checked against real schemas, runtime behavior, validation, linting, tests, preview modes, or through dry runs.
- Narrow prompts work better than a single broad prompt. The implementation became more reliable when large tasks were decomposed into small operation-specific prompts with explicit input structure and output formats.
- Deterministic work should remain deterministic. Normalization, fixed mappings, naming patterns, simple transformations, timeouts, defaults, and stable fallbacks are safer in ordinary code or heuristics than in an LLM call.
- Context quality matters more than context size. Large documentation or schema input had to be filtered, digested, and structured before reaching the model; more raw context did not automatically lead to better answers.
- Domain expertise has to be encoded before the AI call. The best object-type and mapping

results came from translating how experienced integration engineers reason about a resource into measurable signals: value distributions, missing-value counts, uniqueness, naming patterns, and schema metadata.

- Aggregated statistics are often better than raw data. Pre-computed and anonymized statistics gave the model enough structure to reason about resources without relying on raw personal data or large raw samples. Where representative samples are needed for validation or simulation, they are treated as controlled, minimized inputs rather than as unrestricted model context.
- Layered pipelines reduce cost and risk. System-provided mappings, known-schema heuristics, as-is evaluation, and deterministic transformation rules should run before an LLM is used; the model is most useful for tasks where cheaper and more predictable methods do not produce a sufficient result.
- AI workflows need observability. Latency, retries, prompt variants, model choices, token use, and evaluation metrics can become real architecture concerns, so tools such as Langfuse and ClearML became part of the engineering process rather than optional extras.
- Provenance and scoring have to be visible. Users need to see whether a suggestion was system-provided or AI-generated, how it was scored, and whether its validation failed. Otherwise, even good suggestions feel difficult to trust.
- Model size is an architectural decision for structured tasks. Schema matching, filter generation, and transformation scripting require strict correctness, so model choice cannot be treated only as a cost optimization question.
- Human review is a design feature, not a temporary limitation. The final review step is often the last security layer, especially for business logic, security-sensitive mappings, exception handling, and user-facing configuration changes.

The same lessons shaped the smart integration work. Prepared inputs and domain expert reasoning were more important than model choice alone, prompt engineering became a first-class engineering activity, and the user experience had to expose scores, preview steps, and reviewable suggestions. The project therefore confirmed that production-ready AI support for IGA must be hybrid, observable, intentionally narrow, and explicitly accountable.

9.5 Future Roadmap and Follow-Up Opportunities

From the product perspective, the midPilot project has successfully established a strong functional baseline for continued AI-assisted onboarding in midPoint. Because the core infrastructure is now in place, the next stage is not about reinventing the architecture. Instead, the focus shifts toward consolidation, hardening, evaluation, and the selective completion of areas that remained in experimental state when the project finished.

9.5.1 Productizing the Onboarding Pipeline

The immediate follow-up work is the continued hardening of the delivered onboarding pipeline

to ensure strict production-readiness. While the current pipeline handles standard integrations beautifully, we plan to expand scenario coverage to account for more complex and non-standard target systems. This requires broader evaluation across a wider variety of real-world APIs and schemas. Additionally, we will continue refining the user-facing workflows to ensure that error handling, AI fallbacks, and human-in-the-loop validation steps are as intuitive and frictionless as possible for human engineers. The final step toward production-readiness is resolving the remaining cybersecurity and compliance issues described in Section 7.6.

9.5.2 Application Onboarding

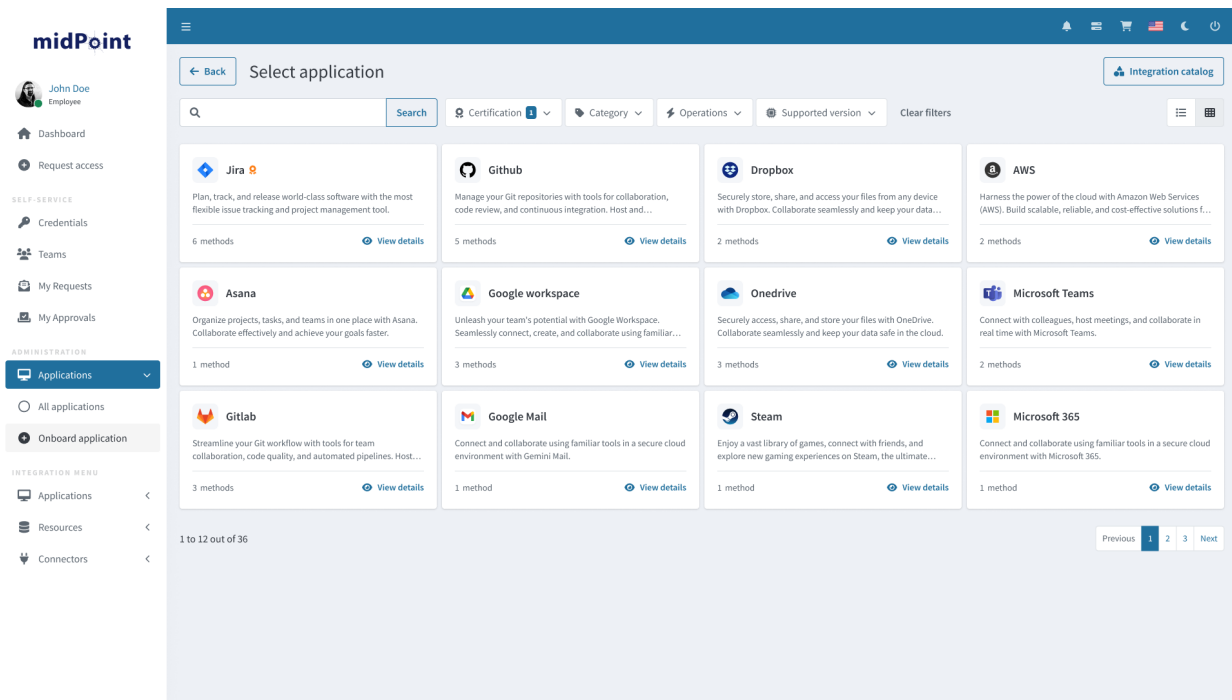
While the project successfully delivered the core AI infrastructure, we learned that injecting these tools into traditional, low-level workflows limits their impact. To fully unlock the value of AI-assisted onboarding, the user journey must be redesigned to prioritize the business context first.

Historically, identity integration is a bottom-up process: administrators start by installing a connector, configuring a resource, mapping schemas, and configuring technical aspects before ever thinking about the business context. The major strategic shift for the future of midPoint is moving toward a top-down approach, where the business purpose drives the technical configuration.

The planned application onboarding direction introduces a clearer separation between deployed applications and integration work in progress. Applications in the Administration area represent applications that are active, connected to a resource, and ready for operational use. The Integrations area provides a dedicated space for application instances, resources, and connectors that are being prepared, imported, configured, or reused as part of the integration process.

The new onboarding journey is designed around a guided, visually intuitive wizard starting directly from the main Applications view. Rather than diving straight into low-level configuration, the process utilizes progressive disclosure:

- **Application Selection:** Users begin by selecting a prepared application from a card-based UI enriched with vendor logos, descriptions, and metadata (supported versions, protocols). If the required application is not available locally, the system offers a seamless recovery path, allowing the user to browse the integration catalog, import the definition, and return to the flow without losing their place.



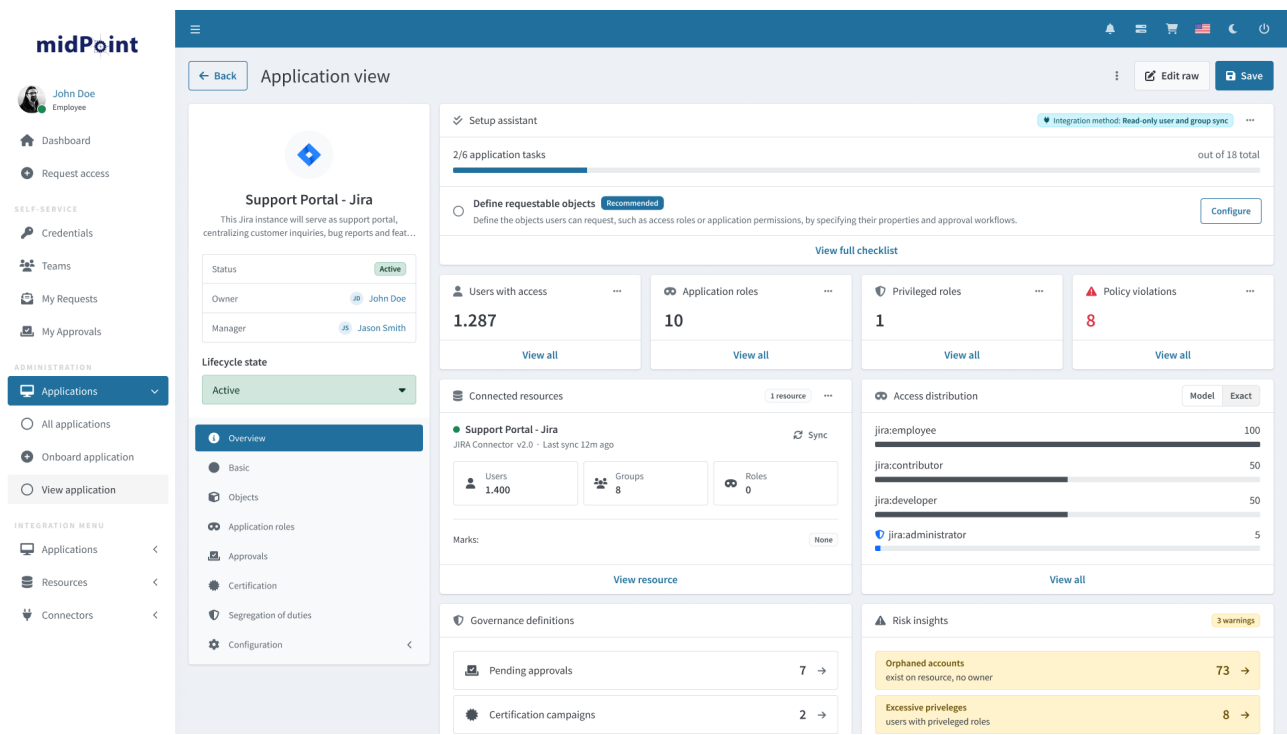
- **Method Selection:** The user then selects an integration method, which defines the intended behavior, required connectors, and setup guidance. This keeps the initial focus purely on the architectural direction.
- **Business Context:** The final step of the initial wizard captures business-centric details like the application instance name, description, owner, and manager.

By restructuring the flow this way, we drastically reduce the cognitive load on the engineer. The decision path is shorter, known applications benefit from highly reusable definitions, and technical complexities are deferred until the business object is actually created. Once confirmed, midPoint creates the application object and smoothly hands the user off to a persistent Setup Assistant to tackle the detailed resource connection—ensuring strong continuity between business onboarding and technical setup.

9.5.3 Application and Resource Dashboards and Setup Assistant

During the project, we identified a crucial operational challenge: navigating users through the complex flow of connecting resources and applications. We learned that artificial intelligence is only half the solution; if the user gets lost in the configuration process, the value of the AI suggestions is severely diminished. To address this, we plan to move away from treating application and resource views merely as dense configuration forms. Instead, we will introduce dedicated Application and Resource Dashboards.

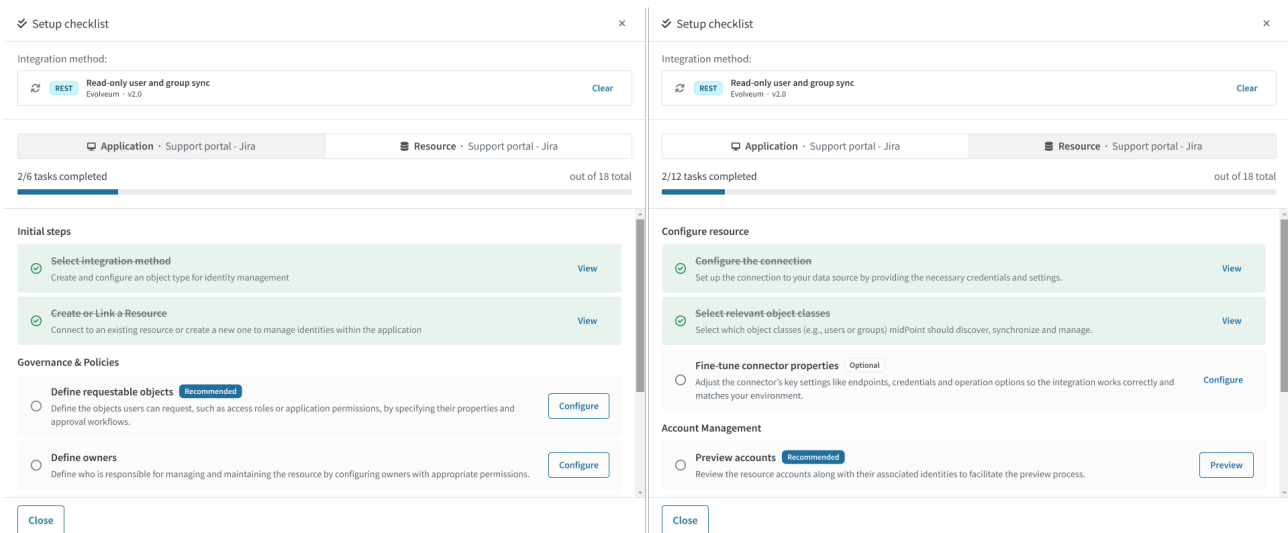
These dashboards are intended to work as object-level dashboards that summarize configuration state, integration readiness, and relevant operational signals in one place. Instead of treating application and resource views only as configuration forms, the dashboard will give administrators and engineers clearer understanding of what the object represents, how complete its setup is, and what requires their attention next.



A key part of this direction is the setup assistant. The setup assistant provides a persistent guidance layer based on the selected integration method. After an application is created and connected to a resource, the integration method can define the expected configuration path for both objects, i.e., the application and the resource. This reduces the cognitive load on engineers, ensuring they do not need to memorize the correct configuration sequence or guess what requires their attention next. It actively bridges the gap between the initial application onboarding and the detailed resource configuration.

To support this guided experience, the user interface will adopt a highly structured two-column object layout. The left side keeps the object identity, basic status, and local navigation visible while the right side serves as the dynamic working area. This keeps users oriented when moving between overview, configuration, and follow-up actions. The setup assistant remains available in the dashboard so that users can return to unfinished work without restarting the onboarding flow.

Importantly, the Setup Assistant is designed as a helpful guide, not a strict requirement. Power users can still configure objects manually or continue without an integration method when needed. This keeps the experience useful for guided onboarding while preserving the flexibility expected in advanced midPoint deployments.



9.5.4 Integration Catalog in MidPoint

Another continuation area is deepening the reuse of delivered artifacts through the integration catalog. Currently, working with the catalog involves some operational friction: it exists as a separate entity outside of midPoint. Engineers must manually browse the external catalog, download integration methods, and then import them into their midPoint environment, which is a disjointed, multi-step process. Our goal for the next iteration is to eliminate this gap and move beyond simply sharing artifacts to fully upstreaming them into the native midPoint experience.

To achieve this, we plan to connect midPoint directly to the integration catalog. Engineers will be able to browse available integrations, read descriptions, and seamlessly download and deploy configurations directly from the midPoint user interface. Furthermore, this direct connection will transform the publication workflow. If a partner or user develops a new AI-assisted connector, they will have a straightforward option to upload and share their work back to the community directly from their midPoint environment, which drastically lowers the barrier to open-source contribution.

Finally, to ensure these shared artifacts are immediately actionable, we will focus on standardizing the formats of sample files within the integration methods. By ensuring these files are natively formatted for midPoint, the system will be able to automatically consume them within the new application onboarding workflows, the setup assistant, and existing resource templates.

9.5.5 Update of Learning Materials

We will share the know-how achieved during the project with our customers and the midPoint community. Our existing midPoint deployment methodologies will be adapted to include AI-assisted configuration to further simplify midPoint deployments. Additional methodologies may be created during this process.

We will also update our existing midPoint training curriculum. The existing deployment training courses will be enriched with AI assistants developed during the project. Additional training courses, or variants of existing ones, may be created during the process. This will improve the user experience with midPoint as well as enable users to evaluate the possibilities of AI-assisted configuration in a safe training environment before deploying them in real projects. The connector development training courses will be extended to include AI-assisted connector development.

We also plan to gradually improve the midPoint certification process by including topics related to the AI-assisted configuration options.

9.5.6 Native Integration of Secure Expression Language

Based on the successful expert analysis of the *secure expression language* conducted during the project, our focus in the upcoming period will be on the full incorporation of this approach in the form of our own MidPoint Expression Language (MEL) into midPilot. The goal is to ensure that artificial intelligence generates configuration scripts and mapping expressions exclusively using this strictly controlled and secure language.

This native integration of MEL into midPilot AI operations fundamentally eliminates security risks associated with unwanted or malicious code injection into generated scripts. It guarantees that all automatically generated artifacts will be secure at the architectural level (security-by-design) and safe for deployment in critical production environments.

9.5.7 Activities in Areas of Compliance and Regulatory Alignment

With the ongoing adoption of AI technologies in the enterprise sector and regulated industries, we will intensively focus on ensuring continuous compliance with evolving European regulatory frameworks. Our activities will primarily target preparing and aligning the product with the Artificial Intelligence Act (EU AI Act), the Cyber Resilience Act (CRA), the cybersecurity directive (NIS2), and maintaining strict adherence to GDPR standards. In the context of the CRA, our focus will be particularly on vulnerability management throughout the product lifecycle and ensuring the security of the software supply chain. The planned activities also include drafting detailed compliance documentation, enhancing processes for the transparency and explainability of AI decisions, and building robust audit trails. This will ensure that the midPilot solution seamlessly passes even the most stringent security audits conducted by our customers.